Marketing Research

Mike Nguyen

2021-03-11

Contents

Chapter 1

Prerequisites

No prerequisites required to read this book

Chapter 2

Introduction

This is not an actual book or guide on marketing research, but more like a placeholder for major constructs in marketing as well as interesting articles in marketing.

Feel free to comment or suggest changes to the content of this book.

Differences between goods and services:

- Intangibility
- Complexity
- Heterogeneity

Dimensions of Qualitative Research:

1. Ontology: "is a philosophical belief system about the nature of social reality- what can be known and how".
2. Epistemology: "is a philosophical belief system about who can be a knower".
3. Methodology (theoretical perspective): "an account of social reality or some component of it that extends further than what has been empirically investigated":
   - Post-positivist: causal relationship can be tested (i.e., disprove)
   - Interpretive: assumes the world is constantly being constructed through group interactions;hence social reality can be understood via the perspectives of social actors enmeshed in meaning-making activities".

- Critical: view social reality as an ongoing construction, and suggest that discourses created in shifting fields of social power shape social reality and its study.

4. Methods: “technique for gathering evidence”.

Chapter 3

Advertising

Prior distribution for TV advertising elasticity for consumer packaged goods can be found in (Shapiro et al., 2018). A substantial portion of products has statistically insignificant or negative estimates for advertising elasticity. TV ads might not be the best vehicle to reach the customer now.

(Gordon et al., 2019) reframe our idea of advertising effect, which should be thought in the sense of incremental effect above the baseline of an ad on consumer behavior.

(Gordon et al., 2017) and (Lewis and Rao, 2015) provided evidence that observational data can hardly detect adverting effect from noise (i.e., we could either over or under-estimate).

The goal standard to measure advertising is random experiment, but it not possible. An improvement is to have panel data from different regions of your market to estimate the baseline for each market. Then, you can uncover the advertising effect.

Chapter 4

Brand Equity

brand and psychological distance: Congruence effect: “information is easier to process, which leads consumers to respond more favorably.” (Connors et al., 2020) “consumer spending higher when distance between consumer and brand matched with construal level of marketing communications.”

Value-added is the difference between a product’s price to consumers and the cost of producing it.

Whereas, brand equity is the difference between a branded product’s price to a generic product’s price.

This generic product can also have value-added in it in other forms (e.g., convenience)

Brand identity offers a value proposition to customers, where “a brand’s value proposition is a statement of the functional, emotional, and self-expressive benefits delivered by the brand that provide value to the customers” (Aaker, 1996, pp.95).

Functional benefits are product attributes that deliver functional utility to customers, while emotional benefits are positive feelings customers feel when using the brand. Ads that provide emotional benefits are higher in effectiveness score compared to functional benefits ads (Aaker, 1996, pp.98).

For example, Evian – “Another day, another chance to feel healthy” ad - which provides emotional benefits of feeling satisfied after a workout. Moreover, self-expressive benefits are when brands provide a means for consumers to express their self-image. A person/ consumer can have multiple roles which associate with multiple self-concepts. Such as a woman, a mother, an author. The difference between emotional benefits and self-expressive benefits can be subtle. Emotional benefits focus on feelings, private setting (e.g., feeling of watching a TV show), past-oriented (memories), transitory, after-use emotional; In contrast, self-expressive benefits focus on self, public settings (e.g., using cars or clothes to signal), aspiration or future-oriented, permanent (i.e., the self links to person's personality), and during-use (wearing fancy clothes to signify a successful person). (cf. (Aaker, 1996, pp.99))

With the advancement in mass production of high -quality products, quality is no longer a dimension of status signal (Holt, 1998)

In the virality framework, the first and foremost driver is social currency. Social currency refers to things worth sharing. People share so that they can let others know that they are in the know. Sharing a new product to assimilate his or her identity with the product and its brand identity, customers are acquiring social currency. Moreover, brand identity is a driver of brand equity, which refers to the value portion that customers are willing to pay above the product's utility value (practical value). Brand identity offers a value proposition to customers, where "a brand's value proposition is a statement of the functional, emotional, and self-expressive benefits delivered by the brand that provide value to the customers" (Aaker, 1996, p. 95). Functional benefits are product attributes that deliver functional utility to customers, while emotional benefits are positive feelings customers feel when using the brand. Ads that provide emotional benefits are higher in effectiveness score compared to functional benefits ads (Aaker, 1996, p. 99). For example, Evian – "Another day, another chance to feel healthy" ad - which provides emotional benefits of feeling satisfied after a workout. Moreover, self-expressive benefits are when brands provide a means for consumers to express their self-image. A person/ consumer can have multiple roles which associate with multiple self-concepts. Such as a woman, a mother, an author. The difference between emotional benefits and self-expressive benefits can be subtle. Emotional benefits focus on feelings, private setting (e.g., feeling of watching a TV show), past-oriented (memories), transitory, after-use emotional; In contrast, self-expressive benefits focus on self, public settings (e.g., using cars or clothes to signal), aspiration or future-oriented, permanent (i.e., the self links to person's personality), and during-use (wearing fancy clothes to signify a successful person). (cf. (Aaker, 1996, p. 99)) Hence, one can see that the brand identity portion that provides self-expressive benefits is social currency while exhaustively different from the brand identity portion that provides functional benefits (i.e., the practical value/ usefulness/ utility) .

We acknowledge that some might view usefulness/ utility as a dimension of brand equity (perceived quality): the perceived quality dimension can signal the true practical value. However, the practical value that drives virality is usually understood, experimented, experienced; while perceived quality is subjective comprehension that is hard to demonstrate. Moreover, quality used to be the status signal. However, with the advancement in mass production of high-quality products, quality is no longer a status signal dimension (Holt, 1998). Thus, we believe that the distinction between the two understandings should be appreciated.

Chapter 5

Celebrity Endorsement

Based on Power Distance Theory, power distance beliefs moderates the positive effect of celebrity endorsers on evaluations of advertising (Winterich et al., 2018)

Chapter 6

Innovation

(Chandrasekaran et al., 2020) introduce a model for successive technologies adoption that accounts for:

- Different rate of technology disengagement
- Different rate of technology adoption

Chapter 7

WOM / Virality

To understand virality, we first need to comprehend its root, which is WOM. Acknowledging that there are subtle differences among WOM, virality, and sharing, we use the three terms interchangeably in this paper. Previous authors provide strong supports for the similarity between virality and WOM (Camarero & San José, 2011; Phelps, Lewis, Mobilio, Perry, & Raman, 2004).

Formally, (Westbrook, 1987, p. 261) defined WOM as "informational communication directed at other consumers about the ownership, usage, or characteristics of particular goods and services or their sellers." In the age of the Internet, WOM evolves to "word of mouse" (or e-WOM). Both virality and WOM take advantages of the network effect to reach wide audiences (Vilpponen, Winter, & Sundqvist, 2006), but virality does it better due to its competitive cost, rapid diffusion, and better targeting (Bampo, Ewing, Mather, Stewart, & Wallace, 2008; Khan & Vong, 2014). Hence, virality is more synonymous with e-WOM.

Virality comes up as a by-product of the Internet, social media platforms, and network theory developments. In network theory, researchers from the domain of sociology or computer science study the structural characteristics/properties of a node (an individual) or a network, such as a node's position in a network or network's cluster. While epidemiologists, management, and marketing researcher study the characteristics of the entity being transmitted, such as virus or message itself. Thus, virality can be driven by content or context of the message (Berger & Milkman, 2012), nature of the spreader, and audience (Hennig-Thurau, Gwinner, Walsh, & Gremler, 2004; Iyengar, van den Bulte, & Valente, 2011), seeding strategies (Kalish, Mahajan, & Muller, 1995; Libai, Muller, & Peres, 2005), and social network structure (Bampo et al., 2008). In this conceptual development, we focus on the content and context drivers of virality.

Virality is defined as the probability of an entity being passed along (Hansen, Arvidsson, Nielsen, Colleoni, & Etter, 2011). The entity can be a message, content, video, knowledge, or disease. Alternatively, (Tucker, 2015) defines virality as “a process whereby an ad is successively shared by viewers.” These two definitions focus on virality as a process; instead, we focus on viral as a construct. Hence, we are in line with (Chandler & Munday, 2016) defining virality (social media virality, social shareability, social media shareability) as “the potential for spreadability of any given content; or particular qualities which are considered to have led to content ‘going viral’.”

For WOM to work, people must share brand-related content, and attention needs to be translated into sales. With these positive associations towards a brand, marketers can increase a customer’s positive brand attitude and later purchase behavior (Faircloth, Capella, & Alford, 2001).

81 percent of a telecom firm indicated that they are likely to refer other to the company; however, only 33 percent actually did. And of those who are referred, only 8 percent became profitable customer. (Kumar et al., 2007)

Even though suffering from the same fate as brand equity, WOM has made it come back under a different alias - virality. The attention that virality has garnered in recent years can be attributed to several reasons:

(1) the dawn of the Internet helps propel marketing into a new paradigm - digital marketing - where earned media is the new king

(2) the introduction of social network platforms such as Facebook, Twitter that facilitate the transmission of information and messages seamlessly and provide an unlimited source of data

(3) The advancement in network theory both theoretically, and empirically.

One platform that can be representative of this digital revolution is Twitter. According to a recent report, Twitter has 145 million monetizable daily active users. Twelve percent of Americans get their news from Twitter. And Twitter has 2 billion video impressions per day (25 Twitter Stats All Marketers Need to Know in 2020. There are more than 500 million Tweets are sent per day. (The 2014 #YearOnTwitter, 2014). One cannot deny the power of online WOM as well as its potential reach; hence, viral marketing has been born with these platforms.

The age group from 18-24 can be more challenging to target since they reported fewer positive responses compared to other age groups (Libert, 2014). Since their threshold is higher due to the overwhelming digital content they encounter, their emotions are harder to activate.

Wide gaps between two opposing opinions can increase future discussion (reviews); however, fluctuation around one opinion does not. Higher informational content moderate positively the relationship between opposing opinions and future reviews. (Nagle and Riedl, 2014)

WOM should be taken under the context of the evolution of conversation. (Berger, 2014)

Valuable virality (Akpinar and Berger, 2017)

Establishes causality of appeals and brand integralness to virality.

- Compared to informative appeal ads (focus on product features), emotional appeal ads (drama, mood, music, emotion-eliciting strategies) are more likely to be shared.

- Informative appeals boost brand evaluations and purchase because the brand is an integral part of the ad content.

- Hence, to combine the best of both worlds, emotional brand-integral ads boost sharing while also bolstering brand-related outcomes (embedded the brand into the stories).

- The mechanism that ads affect brand evaluation through two simultaneous mechanisms: persuasive attempt and brand knowledge.

"keeping viewers involved depends in large part on two emotions: joy and surprise"(Teixeira, 2012) - "Research: The Emotions that Make Marketing Campaigns Go Viral"

"ads that produce stable emotional states generally aren't effective at engaging viewers for very long." And "shock may get people to watch an ad privately, it often works against their desire to share the spot. Like Clothing Drive that try to mimic "Swear Jar" by Bud Light" (Teixeira, 2012)

(Westbrook, 1987, p. 261) defined WOM as "informational communication directed at other consumers about the ownership, usage, or characteristics of particular goods and services or their sellers." WOM evolves to "word of mouse" (or e-WOM).

(Berger, 2014):

WOM is goal-driven and serves five key functions:

- Impression Management

- Emotion regulation

- Information acquisition

- Social bonding

- Persuasion

The motivation behind this process is self -serving and even subconscious level. Hence, we can predict the types of news and information people are most likely to discuss.

- Over 70% of everyday speech is about the self, such as personal experiences or relationships (Dunbar et al., 1997). Consistent with previous research, online behaviors exhibit the same pattern: over 70% of social media posts are about self (Naaman et al., 2010).
- At any given point in time, multiple drivers can be present.

7.1 Impression Management

Consumers buy to signal their derived identities to achieve desired impressions (J. Berger & Heath, 2007)

Interpersonal communication (WOM) influences impression management in 3 ways: (1) self-enhancement, (2) identity -signaling, and (3) filling conversational space.

- **Self-enhancement**: human has a tendency to self-enhance (Markus, 1987). Hence, people like to share things to help them look good (Chung & Darke, 2006; Hennig-Thurau et al., 2004). Since bragging too much about oneself might have an opposite effect. People sometimes engage in" humblebragging": brag but self-deprecating at the same time. (Sezer et al., 2018)

- **Identity-signalling**: they talk about themselves to signal they have certain traits or expertise (G. Packard & Wooten, 2013), such as opinion leaders talk to signal their identity (share to show their knowledge).

- **Filling conversational space**: (small talk) people engage in small talk to avoid pauses and silence in between conversations so that the other party would not make lousy inferences about the person. (J. Berger, 2014)

Hence impression management induces people to share things that are

- **Entertaining** (i.e., interesting, surprising, funny or extreme) so that they shared look more interesting, and in-the-know. Interesting products get more word of mouth (J. Berger & Iyengar, 2013; J. Berger & Milkman, 2012; J. Berger & Schwartz, 2011). Moderate controversy is a catalyst for word of mouth to spread (Chen & Berger, 2013). And sometimes, people even exaggerate stories to make things more interesting (C. Heath, 1996); around 60% of the stories are distorted (Marsh & Tversky, 2004). Things that can be considered interesting are those that are either novel, exciting, or violate previous expectations (i.e., surprise) (Berlyne, 2006; Silvia, 2008). Nevertheless, comprehensibility can be a moderate of whether a novel thing is interesting: novel and comprehensible equals interesting, while novel and incomprehensible equal confusing (Silvia, 2008). Novelty refers to things that are new, surprising, exciting, or complex (Berlyne, 1960). Comprehensibility refers to the fact that the novelty must be understandable.

- **Useful**: because it makes the sharer smart and helpful. Empirical evidence show that useful stories(J. Berger & Milkman, 2012) and higher quality brands (Lovett et al., 2013) are more likely to be shared.

- **Self-concept relevant**: different people see different domain as the optimal signal amplifier such as politics, economics, etc. hence symbolic products are more likely to be shared than utilitarian ones (Chung & Darke, 2006) (even though useful information is helpful, but the signal to be smart and helpful are not appreciated as being interesting and entertaining).

- **Status related**: premium brands are talked about more (Lovett et al., 2013). Knowledge is can also be considered as a status symbol, and people share to signal that they re in the know (Ritson & Elliott, 1999)

- **Unique**: unique products are more likely to be shared, but people with a high need for uniqueness will be less likely to share positive WOM because they do not want people to adopt the product and reduces their uniqueness (Cheema & Kaikati, 2010). Sometimes they even scare others by citing product complexity (Moldovan et al., 2015)

- **Common ground**: sharing common things induces listener to convey that there is interpersonal similarity and facilitates conversation.

- **Emotional valence**: people prefer to share positive things than negative news (J. Berger & Milkman, 2012). However, in some special cases, negative WOM can be perceived as a desired component such as reviewers are seen as more intelligent, (Amabile, 1983). Which is moderated by whether people are refereeing to themselves or others will affect WOM valence (positive vs. negative)(Kamins et al., 1997). Positive WOM when talking oneself conveys a positive image, while negative WOM about others convey that they are relatively better than the other party.

- **Incidental arousal**: unrelated arousal (e.g., running in place) can increase sharing in general (J. Berger, 2011) because of the increase in arousal levels.

- **Accessible things**: products with more cued or triggered will be discussed more (J. Berger & Schwartz, 2011). Hence, more advertised products generate more WOM (Onishi & Manchanda, 2012). “top of mind” and "tip of the tongue." Availability bias means that the easier we can recall something, the more likely we are going to talk bout it. Publicly visible products also have more WOM (J. Berger & Schwartz, 2011)

7.2 Emotion regulation

WOM help consumers regulate their emotions. Emotion regulation refers to "a person's ability to effectively manage and respond to an emotional experience" (Rolston & Lloyd-Richardson, 2017). and sharing with other (WOM) can facilitate emotion regulation by:

- **Generating social support**: sharing can give you comfort and consolation (Rimé, 2009). For example, after a negative emotional experience, sharing can help increase well-being through perceived social support (J. A. Berger & Buechel, 2012)

- **Venting**: WOM (sharing) helps people achieve catharsis when experiencing negative experiences(Alicke et al., 1992): consumers express when they are angry (Wetzer et al., 2007) or dissatisfied (Anderson, 1998).

- **Sensemaking**: "talking can help people understand what they feel and why" (J. Berger, 2014)

- **Reducing dissonance**: even after making a decision, we want to make sure we make the right choice by talking to others tot reduce feelings of doubt (Rosnow, 1980)

- **Taking vengeance:** to punish the company (different from venting, which makes you feel better), consumers share negative consumption experience (Ward & Ostrom, 2006)

- **Rehearsal**: people talk to relive positive experiences (Rimé, 2009)

Emotion regulation drives people to share (1) emotional content (2) influence the valence of the content shared, (3) lead people to share more emotionally arousing content:

- **Emotionality**: more emotional intensity, more sharing (J. Berger & Milkman, 2012), people share more emotional social anecdotes (Peters et al., 2009). But not all emotions increase sharing: for example, shame and guilt decreases sharing (Finkenauer & Rimé, 1998) since sharing those things makes people look bad.

- **Valence**: "Emotion regulation tends to focus on the management of negative emotions" (J. Berger, 2014), but a counter vailing effect is that under impression management, people avoid sharing negative stories since it might reflect on the shares. Sharing negative things can decrease willingness to share (Chen & Berger, 2013)

- **Emotional arousal**:
  - **Negative** side: low arousal emotion (sadness), high arousal (anger or anxiety) should increase the need to vent.
  - **Positive** side: low arousal (contentment), high arousal (awe, excitement or amusement) increase desires for rehearsal.

7.3 Information acquisition

WOM helps acquire information.

- **Seeking advice**: (Hennig-Thurau et al., 2004; Rimé, 2009). Rather than trial and error, or direct observation, gossip serves as another form of learning (Baumeister et al., 2004)

- **Resolving problems**: online forums or other online opinion platforms help customers find a solution quickly from others (Hennig-Thurau et al., 2004)

Hence the process of information acquisition would induce to talk more about

- **Risky, important, complex, or uncertainty-ridden decision**: talking to others can reduce risk, simplify complexity, and increase consumer's confidence (Hennig-Thurau et al., 2004)

- **Lack of (trustworthy) information**:

7.4 Social bonding

"people have a fundamental desire for social relationships (Baumeister & Leary, 1995), and interpersonal communication fills that need (Hennig-Thurau et al., 2004)". Empirically, people in brand communities because want to connect with others like them (Muniz & O'Guinn, 2001). Sharing facilitates social bonding through

- **Reinforce shared views**: group memberships, and one's place in the social hierarchy. WOM (sharing) allows people to fit in as a group member (J. Berger & Heath, 2007; Ritson & Elliott, 1999)

- **Reducing loneliness and social exclusion**: sharing decrease interpersonal distance and decreases loneliness and exclusion (Maner et al., 2007)

Social bonding drives what people share: people share things are :

- **Common ground**: social bonding drives people to talk about things they have in common because it makes them feel socially connected (Clark & Kashima, 2007)
- **Emotionality**: "sharing an emotional story or narrative increases the chance that others will feel similarly" (J. Berger, 2014). Emotional similarity increase group cohesiveness (Barsade & Gibson, 2007). And emotion sharing and social bonding can be a simultaneous process since previous works found that emotion sharing bonds people (Peters & Kashima, 2007) and high arousal emotion increase social bonding needs (Chen & Berger, 2013)

7.5 Persuading others

people use WOM to persuade others to affect their satisfaction or choices (J. Berger, 2014). Persuasive drive people to share things that are

- **Emotionally polarized** (polarized valence): since the goal is to convince something is good or bad (i.e., people share extremely rather than moderately positive(negative) information

- **Arousing content**: "people who want to persuade others may find shared arousing content to incite others to take desired actions" (J. Berger, 2014)

7.6 Moderators

- Two key moderators when different WOM functions have a greater impact:
  - **Communication audience**:
    - Tie strength
    - Audience size
    - Tie status
  - **Communication Channel**
    - Written vs. oral
    - Identifiability: whether communicators are identifiable. (anonymously)

    - Audience salience: whether the audience is salient during communication (online communication, the audience is less salient).

Social motivation: why we share :(why_some_videos_go_viral_2015)

- Impression management:
  - authority (e.g., "I want to demonstrate my knowledge")

  - Social utility (e.g., this could be useful to my friends)

  - coolhunter (e.g., I want to be the first o tell my friends)
  - Zeitgeist (e.g., It's about a current trend or event)
  - Conversation starting (e.g., I want to start an online conversation)
  - Self-expression (e.g., it says something about me)
  - Social good (e.g., It's for a good cause, and I want to help").

- Information acquisition:
  - Opinion seeking (e.g., "I want to see what my friends think)

- Social bonding:
  - Shared passion (e.g, "it lets me connect with my friends bout a shared interest)
  - Social in real life (e.g., it will help me socialize with my friends offline).

More frequent exposure to perceptually or conceptually related cues increases product accessibility and makes the product easier to process. In turn, this increased accessibility influences product evaluation and choice, which are found to vary directly with the frequency of exposure to conceptually related cues. These results support the hypothesis that conceptual priming effects can have a strong impact on real-world consumer judgment" (Berger and Fitzsimons, 2008)

7.7 Social Currency

Different people see different domain as the optimal signal amplifier, such as politics, economics

Even though useful information is helpful, the signal of being smart and helpful is not appreciated as being interesting and entertaining (Berger, 2014).

People talk more about status-related products because they enhance their impression.

In extreme cases, they even scare others by citing product complexity (Moldovan, Steinhart, & Ofen, 2015).

Interesting things are discussed more in online settings (J. Berger & Milkman, 2012), but this effect fails to replicate in the case of face-to-face settings (Berger & Schwartz, 2011). Hence, research has found that written communication (e.g., texting, messaging, posting online) leads people to talk more about interesting products or brands than oral communication because they have more time to think and polish what to say, leading to more prevalent self-enhancement motives (Berger & Iyengar, 2013). In other words, not because people do not have self-enhancement motives, but because synchronous conversations do not allow for much time to think and posish ideas, people might talk about mundane things to fill the gaps between conversational turns (Berger, 2014).

7.8 Accessibility

Accessibility drives ongoing WOM (i.e., publicly visible products are cued more in the environment – top of mind), regardless of how mundane it might be.

From the theory of semantic process (Collins & Loftus, 1975), priming can help activate related perceptual or conceptual constructs in memory spontaneously (Berger & Fitzsimons, 2008). For example, prior exposure affects the brand choice of both target brand and competing brands (Nedungadi, 1990). In other words, the activation of one brand can lead to others more accessible, which increases the chance of remembered brands to be included in the consideration set. (Lee & Labroo, 2004) argues that ease of processing drives our positive evaluation, which is also consistent with the discrepancy -attribution hypothesis. The Discrepancy-attribution hypothesis states that if people cannot easily attribute the ease of processing to a source, they will likely attribute it to the positive quality of the target. Additionally, (Nedungadi, 1990) found that recently exposed brands are more likely to be included and chosen from consumers' consideration set. Repeated exposure to an object increases favorable ratings (e.g., an object, or person) (Zajonc, 1968), which also apply to attractive people (Moreland & Beach, 1992), and brand (Baker, 1999). Behavioral can be influenced by incidental exposure to stimuli because the primes can activate relevant constructs associated in memory (Christian Wheeler & Petty, 2001; Dijksterhuis & Bargh, 2001; Wheeler & Berger, 2007)

7.9 Emotions

(Milkman & Berger, 2014) found that contents that are (1) emotional aroused, (2) useful, or (3) interesting have a higher chance of being shared. Consistently, (Milkman & Berger, 2014) also found that women produce articles that are more likely to be shared because female authors write more comprehensible, useful, and interesting articles.

People share their extreme emotional experience (i.e., either highly satisfied or highly dissatisfied) (Anderson, 1998)

Lastly, people use WOM to persuade others to affect their satisfaction or choice (Berger, 2014). Thus, emotionally polarized (polarized valance) are more likely to be shared due to persuasion reasons. Since the goal is to convince something is good or bad, people share extremely rather moderately positive (negative) information.

Virality is partially driven by physiological arousal (Berger, 2011; Berger & Milkman, 2012). These results hold even when the authors control how surprising, interesting, or practically useful content is (all of which are positively linked to virality) and external drivers of attention (e.g., how prominently content was featured) (Berger & Milkman, 2012). Different emotion evokes different levels of physiological arousal (i.e., activation) (Smith & Ellsworth, 1985). Arousal is an excitatory state that increases action-related behavior, such as helping others (Gaertner & Dovidio, 1977). Hence, we see contents that are physiological arousal get shared more. Specifically, Negative high arousal emotions (e.g., anger or anxiety) are shared more than negative low arousal (e.g., sadness). Moreover, sharing negative experiences can help people vent to regulate their emotions (Berger, 2014). On the other hand, positive emotion high arousal emotions (e.g., awe, excitement, or amusement) are shared more than positive low arousal (e.g., contentment). Furthermore, sharing for the purpose of rehearsal (i.e., people talk to relive positive experiences) helps people regulate their emotions (Rimé, 2009).

Chapter 8

Customer Lifetime Value (CLV)

also known as Lifetime Value (LTV)

"A year's projected business gives a number that is normally about half of a customer's full lifetime value, although that can vary by industry." (Kumar et al., 2007)

(Kumar et al., 2008) CLV model

$$CLV_i = \sum_{j=T+1}^{T+N} \frac{p(Buy_{ij}=1) \times \hat{CM}_{ij}}{(1+r)^{j-T}} - \frac{\hat{MC}_{ij}}{(1-r)^{j-T}}$$

where

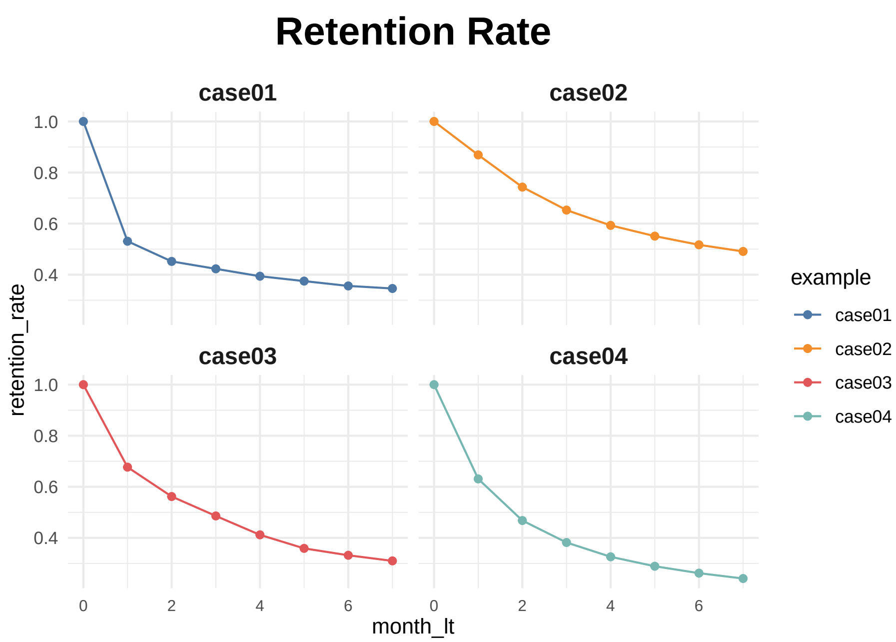
- CLV_i = lifetime value for customer i,
- $p(Buy_{ij})$ = predicted probability that customer i will purchase in period j,
- CM_{ij} = predicted contribution margin provided by customer i in period j,
- MC_{ij} = predicted marketing costs directed toward customer i in period j,
- j = index for time periods (semiannual or annual)
- T = the end of the calibration or observation time frame (usually 1 year projection),

- N = total number of prediction periods,
- r = semiannual discount factor (e.g., 0.07238 (Kumar et al., 2008) = 15% annual rate)

8.1 Example

This example is subscription customers by Sergey Bryl' in which he adapted model Fader and Hardie (2007)

```
library(tidyverse)
library(reshape2)
library(MLmetrics)

# retention rate data
df_ret <- data.frame(month_lt = c(0:7),
                     case01 = c(1, .531, .452, .423, .394, .375, .356, .346),
                     case02 = c(1, .869, .743, .653, .593, .551, .517, .491),
                     case03 = c(1, .677, .562, .486, .412, .359, .332, .310),
                     case04 = c(1, .631, .468, .382, .326, .289, .262, .241)
                     ) %>%
        melt(., id.vars = c('month_lt'), variable.name = 'example', value.name = 'reten

ggplot(df_ret, aes(x = month_lt, y = retention_rate, group = example, color = example))
        theme_minimal() +
        facet_wrap(~ example) +
        scale_color_manual(values = c('#4e79a7', '#f28e2b', '#e15759', '#76b7b2')) +
        geom_line() +
        geom_point() +
        theme(plot.title = element_text(size = 20, face = 'bold', vjust = 2, hjust = 0
              axis.text.x = element_text(size = 8, hjust = 0.5, vjust = .5, face = 'pla
              strip.text = element_text(face = 'bold', size = 12)) +
        ggtitle('Retention Rate')
```



Prediciton when we only have values on certain months

```
# functions for sBG distribution
churnBG <- Vectorize(function(alpha, beta, period) {
        t1 = alpha / (alpha + beta)
        result = t1
        if (period > 1) {
                result = churnBG(alpha, beta, period - 1) * (beta + period - 2) / (alpha + beta +
        }
        return(result)
}, vectorize.args = c("period"))

survivalBG <- Vectorize(function(alpha, beta, period) {
        t1 = 1 - churnBG(alpha, beta, 1)
        result = t1
        if(period > 1){
                result = survivalBG(alpha, beta, period - 1) - churnBG(alpha, beta, period)
        }
        return(result)
}, vectorize.args = c("period"))

MLL <- function(alphabeta) {
        if(length(activeCust) != length(lostCust)) {
                stop("Variables activeCust and lostCust have different lengths: ",
                     length(activeCust), " and ", length(lostCust), ".")
```

```
        }
        t = length(activeCust) # number of periods
        alpha = alphabeta[1]
        beta = alphabeta[2]
        return(-as.numeric(
                sum(lostCust * log(churnBG(alpha, beta, 1:t))) +
                        activeCust[t]*log(survivalBG(alpha, beta, t))
        ))
}


df_ret <- df_ret %>%
        group_by(example) %>%
        mutate(activeCust = 1000 * retention_rate,
               lostCust = lag(activeCust) - activeCust,
               lostCust = ifelse(is.na(lostCust), 0, lostCust)) %>%
        ungroup()


ret_preds01 <- vector('list', 7)
for (i in c(1:7)) {

        df_ret_filt <- df_ret %>%
                filter(between(month_lt, 1, i) == TRUE & example == 'case01')

        activeCust <- c(df_ret_filt$activeCust)
        lostCust <- c(df_ret_filt$lostCust)

        opt <- optim(c(1, 1), MLL)
        retention_pred <- round(c(1, survivalBG(alpha = opt$par[1], beta = opt$par[2],

        df_pred <- data.frame(month_lt = c(0:7),
                              example = 'case01',
                              fact_months = i,
                              retention_pred = retention_pred)
        ret_preds01[[i]] <- df_pred
}
```

```
## Warning in log(churnBG(alpha, beta, 1:t)): NaNs produced

## Warning in log(churnBG(alpha, beta, 1:t)): NaNs produced

## Warning in log(survivalBG(alpha, beta, t)): NaNs produced

## Warning in log(churnBG(alpha, beta, 1:t)): NaNs produced

## Warning in log(survivalBG(alpha, beta, t)): NaNs produced
```

```
## Warning in log(churnBG(alpha, beta, 1:t)): NaNs produced

## Warning in log(survivalBG(alpha, beta, t)): NaNs produced

## Warning in log(churnBG(alpha, beta, 1:t)): NaNs produced

## Warning in log(survivalBG(alpha, beta, t)): NaNs produced
```

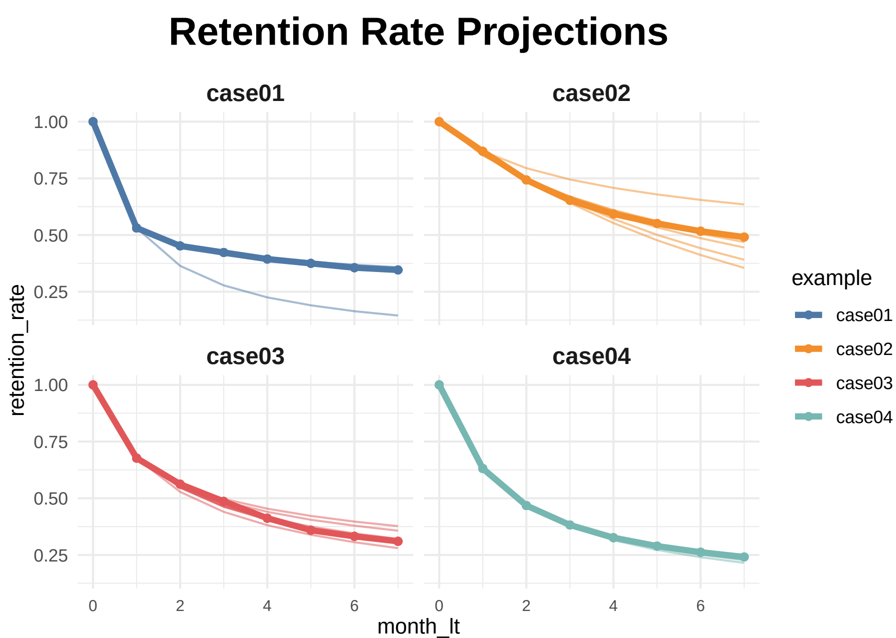
```
ret_preds01 <- as.data.frame(do.call('rbind', ret_preds01))


ret_preds02 <- vector('list', 7)
for (i in c(1:7)) {

        df_ret_filt <- df_ret %>%
                filter(between(month_lt, 1, i) == TRUE & example == 'case02')

        activeCust <- c(df_ret_filt$activeCust)
        lostCust <- c(df_ret_filt$lostCust)

        opt <- optim(c(1, 1), MLL)
        retention_pred <- round(c(1, survivalBG(alpha = opt$par[1], beta = opt$par[2], c(1:7))),

        df_pred <- data.frame(month_lt = c(0:7),
                              example = 'case02',
                              fact_months = i,
                              retention_pred = retention_pred)
        ret_preds02[[i]] <- df_pred
}
```

```
## Warning in log(churnBG(alpha, beta, 1:t)): NaNs produced

## Warning in log(churnBG(alpha, beta, 1:t)): NaNs produced

## Warning in log(churnBG(alpha, beta, 1:t)): NaNs produced

## Warning in log(churnBG(alpha, beta, 1:t)): NaNs produced

## Warning in log(churnBG(alpha, beta, 1:t)): NaNs produced

## Warning in log(churnBG(alpha, beta, 1:t)): NaNs produced

## Warning in log(churnBG(alpha, beta, 1:t)): NaNs produced
```

```
ret_preds02 <- as.data.frame(do.call('rbind', ret_preds02))


ret_preds03 <- vector('list', 7)
for (i in c(1:7)) {
```

```
        df_ret_filt <- df_ret %>%
                filter(between(month_lt, 1, i) == TRUE & example == 'case03')

        activeCust <- c(df_ret_filt$activeCust)
        lostCust <- c(df_ret_filt$lostCust)

        opt <- optim(c(1, 1), MLL)
        retention_pred <- round(c(1, survivalBG(alpha = opt$par[1], beta = opt$par[2],

        df_pred <- data.frame(month_lt = c(0:7),
                              example = 'case03',
                              fact_months = i,
                              retention_pred = retention_pred)
        ret_preds03[[i]] <- df_pred
}

ret_preds03 <- as.data.frame(do.call('rbind', ret_preds03))


ret_preds04 <- vector('list', 7)
for (i in c(1:7)) {

        df_ret_filt <- df_ret %>%
                filter(between(month_lt, 1, i) == TRUE & example == 'case04')

        activeCust <- c(df_ret_filt$activeCust)
        lostCust <- c(df_ret_filt$lostCust)

        opt <- optim(c(1, 1), MLL)
        retention_pred <- round(c(1, survivalBG(alpha = opt$par[1], beta = opt$par[2],

        df_pred <- data.frame(month_lt = c(0:7),
                              example = 'case04',
                              fact_months = i,
                              retention_pred = retention_pred)
        ret_preds04[[i]] <- df_pred
}

ret_preds04 <- as.data.frame(do.call('rbind', ret_preds04))


ret_preds <- bind_rows(ret_preds01, ret_preds02, ret_preds03, ret_preds04)
```

```
df_ret_all <- df_ret %>%
        select(month_lt, example, retention_rate) %>%
        left_join(., ret_preds, by = c('month_lt', 'example'))

ggplot(df_ret_all, aes(x = month_lt, y = retention_rate, group = example, color = example)) +
        theme_minimal() +
        facet_wrap(~ example) +
        scale_color_manual(values = c('#4e79a7', '#f28e2b', '#e15759', '#76b7b2')) +
        geom_line(size = 1.5) +
        geom_point(size = 1.5) +
        geom_line(aes(y = retention_pred, group = fact_months), alpha = 0.5) +
        theme(plot.title = element_text(size = 20, face = 'bold', vjust = 2, hjust = 0.5),
              axis.text.x = element_text(size = 8, hjust = 0.5, vjust = .5, face = 'plain'),
              strip.text = element_text(face = 'bold', size = 12)) +
        ggtitle('Retention Rate Projections')
```



calculate the average LTV for case03 based on two historical months with a forecast horizon of 24 months and a subscription price of $1

```
### LTV prediction ###
df_ltv_03 <- df_ret %>%
        filter(between(month_lt, 1, 2) == TRUE & example == 'case03')

activeCust <- c(df_ltv_03$activeCust)
lostCust <- c(df_ltv_03$lostCust)
```

```
opt <- optim(c(1, 1), MLL)
retention_pred <- round(c(survivalBG(alpha = opt$par[1], beta = opt$par[2], c(3:24))),

df_pred <- data.frame(month_lt = c(3:24),
                      retention_pred = retention_pred)

df_ltv_03 <- df_ret %>%
        filter(between(month_lt, 0, 2) == TRUE & example == 'case03') %>%
        select(month_lt, retention_rate) %>%
        bind_rows(., df_pred) %>%
        mutate(retention_rate_calc = ifelse(is.na(retention_rate), retention_pred, rete
               ltv_monthly = retention_rate_calc * 1,
               ltv_cum = round(cumsum(ltv_monthly), 2))
# average LTV of $9.33. actual data for the observed periods (from 0 to 2nd months) an
```

8.2 Referral value (CRV)

"Referrals made by customers after a referral-incentive marketing campaign can be attributed to that campaign for about a year." (Kumar et al., 2007). Hence, we usually count those referrals made after one year of a campaign.

2 types of referral:

- Type-one referral: Would not join without referral.
- Type-two referral: still join without referral.

(Kumar et al., 2007) estimated that each customer made about half type-one and half type-two referrals.

Referral value = PV(Type-one referrals) + PV(Type-two referrals)

where

- PV(type-two referrals) = the prevent value of the savings in acquisition costs.

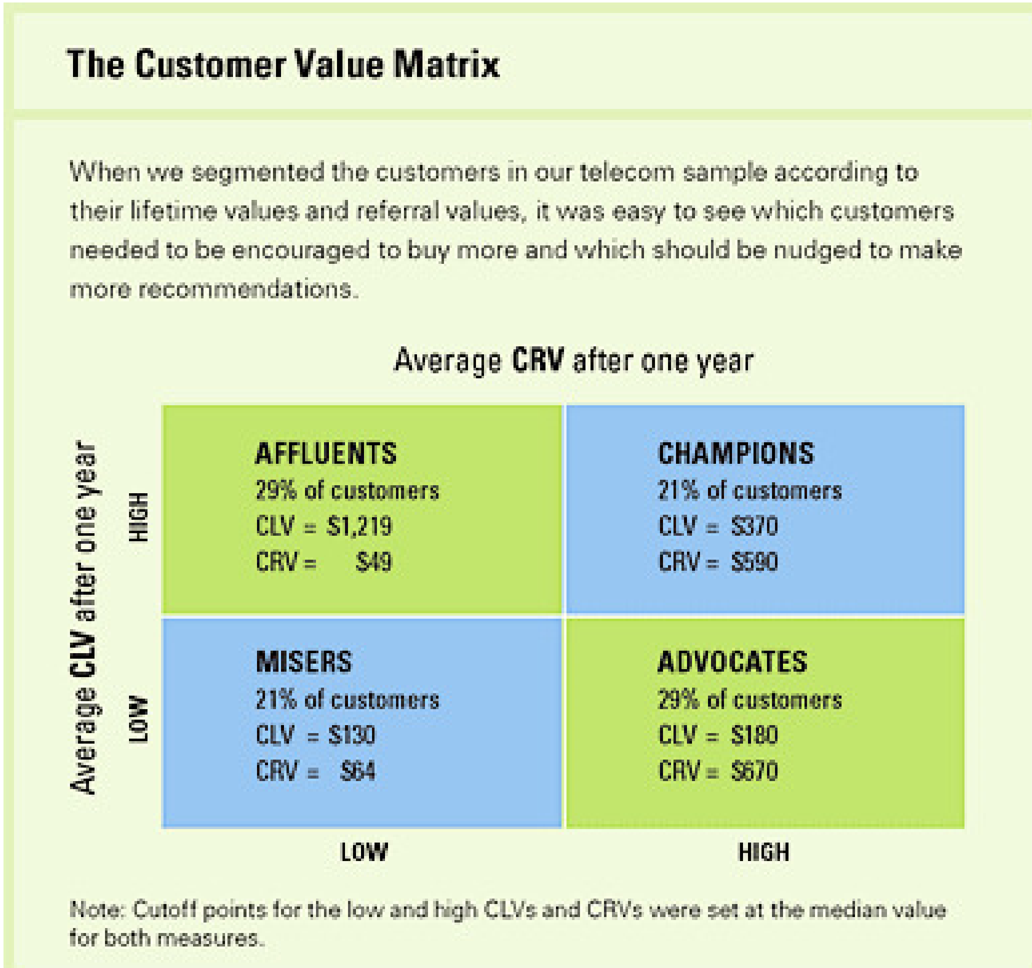
$$CRV_i = \sum_{t=1}^{T}\sum_{y=1}^{n1}\frac{A_{ty} - a_{ty} - M_{ty} + ACQ1_{ty}}{(1+r)^t} + \sum_{t=1}^{T}\sum_{y=n1+1}^{n2}\frac{ACQ2_{ty}}{(1+r)^t}$$

where

- T = the number of periods that will be predicted into the future (e.g., typically 1 year)

- A_{ty} = the gross margin contributed by customer y who other otherwise would not have bought the product,
- a_{ty} = the cost of the referral for the customer y (this is what you set up)
- $n1$ = the number of customers who would not join without the referral,
- $n2 - n1$ = the number of customers who would have joined anyway,
- M_{ty} = the marketing costs needed to retain the referred customers,
- r = semiannual discount factor ((Kumar et al., 2008) used .07238 = 15% annual rate)
- $ACQ1_{ty}$ = the savings in acquisition costs from customers who would not join without the referral,
- $ACQ2_{ty}$ = the savings in acquisition cost from customers who would have joined anyway.

Customers with high CLV does not guarantee high CRV (Kumar et al., 2007). Hence, (Kumar et al., 2007) proposed the customer value matrix


The Customer Value Matrix
When we segmented the customers in our telecom sample according to their lifetime values and referral values, it was easy to see which customers needed to be encouraged to buy more and which should be nudged to make more recommendations.
Average CRV after one year
Average CLV after one year
HIGH
LOW
AFFLUENTS
29% of customers
CLV = $1,219
CRV = $49
CHAMPIONS
21% of customers
CLV = $370
CRV = $590
MISERS
21% of customers
CLV = $130
CRV = $64
ADVOCATES
29% of customers
CLV = $180
CRV = $670
LOW
HIGH
Note: Cutoff points for the low and high CLVs and CRVs were set at the median value for both measures.

Chapter 9

Finance-Marketing Interface

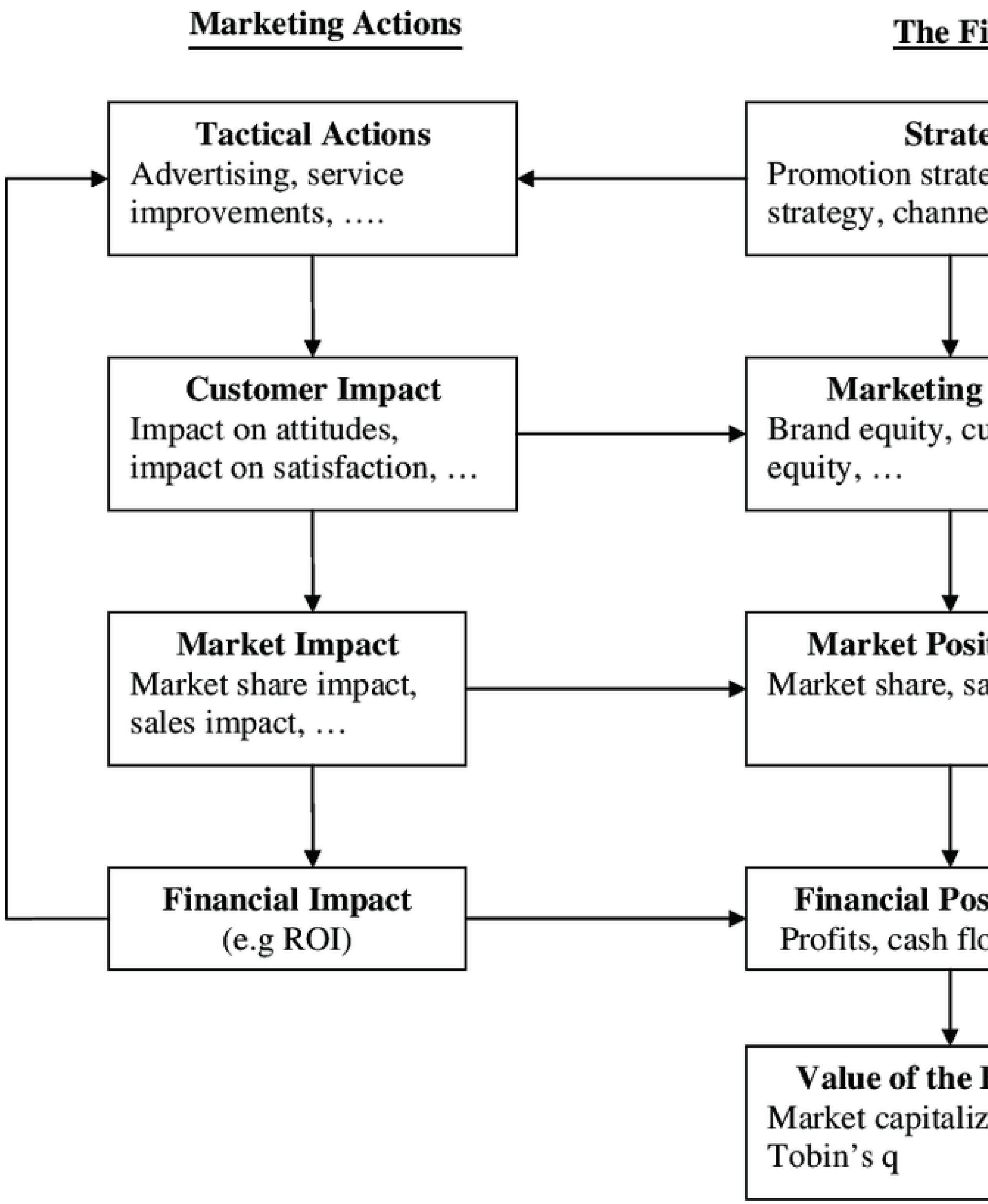
Market-based assets are: relational and intellectual.

Relational market-based assets are “outcomes of the relationship between a firm and key external stakeholders, including distributors, retailers, end customers, other strategic partners”. (Srivastava et al., 1998)

For example, brand equity = relational assets between firms and their customers, while channel equity - relational assets between firms and their channel partners.

Intellectual market-based assets are “the types of knowledge a firm possesses about the environment, such as the emerging and the potential state of market conditions and the entities in it, including competitors, customers, channels, suppliers, and social and political interest group.” (Srivastava et al., 1998)

Marketing actions (e.g, adverting, product innovation) can build long-term assets (e.g., brand equity, customer equity), in turn increase short-term profitability (e.g., increase marketing effectiveness for dollars spent on advertising) (Rust et al., 2004)


Marketing Actions
The Fi
Tactical Actions
Advertising, service improvements, ….
Strate
Promotion strate
strategy, channe
Customer Impact
Impact on attitudes, impact on satisfaction, …
Marketing
Brand equity, cu
equity, …
Market Impact
Market share impact, sales impact, …
Market Posi
Market share, sa
Financial Impact
(e.g ROI)
Financial Pos
Profits, cash flo
Value of the
Market capitaliz
Tobin's q

The Chain of Marketing Productivity (source (Rust et al., 2004))

Relationship between marketing and finance (Srivastava et al., 1998)

Stage 1: Market-based Assets

- Customer relationships: Brands, installed base.
- Partner Relationship: channels, co-branding network.

Stage 2:

- Faster Market penetration
- Price premium
- share premium
- extensions
- reduce sales/service costs
- Loyalty/ Retention

Stage 3: Shareholder value:

- Accelerate Cash Flows
- Enhance Cash Flows
- Reduce Volatility and Vulnerability of Cash flows
- Enhance Residual Value of Cash Flows.

(Grewal et al., 2010; McAlister et al., 2016; Morgan and Rego, 2009) used Tobin's Q in marketing research. since it is "forward-looking, risk-adjusted, and less easily manipulated by managers" (Wies et al., 2019). When probing factors affecting Tobin's Q, it was suggested to control for Financial Leverage,Cash flows

9.1 Tobin's q

Retrieve data from WRDS

```
# to set up connection from R to WRDS (https://wrds-www.wharton.upenn.edu/pages/support/programm
library(RPostgres)
library(dplyr)

# I've set up wrds connection before hand. Please use your username and password here.
```

```
# wrds <- dbConnect(Postgres(),
#                   host='wrds-pgdata.wharton.upenn.edu',
#                   port=9737,
#                   dbname='wrds',
#                   sslmode='require',
#                   user='')

# Check variables (column headers) in COMP ANNUAL FUNDAMENTAL
#uses the already-established wrds connection to prepare the SQL query string and save
# check avaiable databases: https://wrds-web.wharton.upenn.edu/wrds/tools/variable.cfm
res <- dbSendQuery(wrds, "select column_name
                   from information_schema.columns
                   where table_schema='compa'
                   and table_name='funda'
                   order by column_name")
data <- dbFetch(res, n=-1) # fetches the data that results from running the query res 
dbClearResult(res) # closes the connection
head(data)
```

```
##   column_name
## 1       acchg
## 2        acco
## 3       accrt
## 4     acctchg
## 5     acctstd
## 6        acdo
```

```
# select everything
res <- dbSendQuery(wrds, "select * from compa.funda")

# from compa.funda

# only select the following variables
res <- dbSendQuery(wrds, "select gvkey, datadate, fyear, indfmt, consol, popsrc, datafr
```

```
## Warning in result_create(conn@ptr, statement): Closing open result set,
## cancelling previous query
```

```
data1 <- dbFetch(res, n=-1)
dbClearResult(res)

data = data1 %>%
    distinct(gvkey,datadate,fyear,tic,conm,.keep_all = T)
```

Calculate Tobin's Q

```
tobin_q_dt = data %>%
    mutate(mkvalt = coalesce(mkvalt,0),
           dltt = coalesce(dltt,0),
           at = coalesce(at,0),
           dlc = coalesce(dlc,0),
           act = coalesce(act,0)) %>%
    mutate(tobin_q = (mkvalt + ifelse((dlc - act) >=0,as.numeric(dlc-act),0) + dltt)/at ) %>%  #j
    select(tobin_q,gvkey,datadate,fyear,conm)
head(tobin_q_dt)
```

```
##     tobin_q  gvkey   datadate fyear                  conm
## 1       Inf 001000 1961-12-31  1961 A & E PLASTIK PAK INC
## 2       NaN 001000 1962-12-31  1962 A & E PLASTIK PAK INC
## 3       Inf 001000 1963-12-31  1963 A & E PLASTIK PAK INC
## 4 0.3686441 001000 1964-12-31  1964 A & E PLASTIK PAK INC
## 5 0.4995671 001000 1965-12-31  1965 A & E PLASTIK PAK INC
## 6 0.4563786 001000 1966-12-31  1966 A & E PLASTIK PAK INC
```

9.2 Fama and French Three-factor model

(Fama and French, 1993)

An expansion from the capital asset pricing model (CAPM) to measure portfolio performance and future returns

Accounting for **company size**, and **value**

$$R_{it} - R_{ft} = \alpha_{it} + \beta_1 (R_{Mt} - R_{ft}) + \beta_2 SMB_t + \beta_3 HML_t$$

where

- R_{it} = total return of a stock or portfolio i at time t
- R_{ft} = risk free rate of return at time t
- R_{Mt} = total market portfolio return at time t
- $R_{it} - R_{ft}$ = expected excess return
- $R_{Mt} - R_{ft}$ = excess return on the market portfolio (index)
- SMB_t = size premium (small - big)
- HML_t = value premium (high - low), also known as value premum (i.e., companies with high book-to-market stocks are called value stocks)
- $\beta_{1,2,3}$ = factor coefficients

9.2.1 Example 1

Example from Calculating Investor

```
# Fama-French Regression example in R

#download factors from https://mba.tuck.dartmouth.edu/pages/faculty/ken.french/data_li

# Load CSV file into R
ff_data <- read.table("images/ffdata.csv",header=TRUE,sep=",")

# Extract Fama-French Factors and Fund Returns
rmrf <- ff_data[,2]/100
smb <- ff_data[,3]/100
hml <- ff_data[,4]/100
rf <- ff_data[,5]/100
fund <- ff_data[,6]/100

# Calculate Excess Returns for Target fund
fund.xcess <- fund - rf

# Run Fama-French Regression
ffregression <- lm(fund.xcess ~ rmrf + smb + hml)

# Print summary of regression results
print(summary(ffregression))
```

```
##
## Call:
## lm(formula = fund.xcess ~ rmrf + smb + hml)
##
## Residuals:
##       Min        1Q    Median        3Q       Max
## -0.043944 -0.008162 -0.000333  0.009172  0.042174
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) -0.002922   0.001946  -1.502  0.13881
## rmrf         1.210644   0.041743  29.002  < 2e-16 ***
## smb          0.151090   0.086468   1.747  0.08606 .
## hml         -0.298928   0.073488  -4.068  0.00015 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.01491 on 56 degrees of freedom
## Multiple R-squared:  0.9509, Adjusted R-squared:  0.9483
## F-statistic: 361.7 on 3 and 56 DF,  p-value: < 2.2e-16
```

This is monthly data, and we can see α term is negative means that the portfolio underperformed compared to the market.

9.2.2 Example 2

Example from R View

```
library(tidyquant)
library(tidyverse)
library(timetk)
library(broom)
library(glue)

symbols <- c("SPY","EFA", "IJS", "EEM","AGG")



prices <-
  getSymbols(symbols, src = 'yahoo',
             from = "2012-12-31",
             to = "2017-12-31",
             auto.assign = TRUE, warnings = FALSE) %>%
  map(~Ad(get(.))) %>%
  reduce(merge) %>%
  `colnames<-`(symbols)

w <- c(0.25, 0.25, 0.20, 0.20, 0.10)

# + SPY (S&P500 fund) weighted 25%
# + EFA (a non-US equities fund) weighted 25%
# + IJS (a small-cap value fund) weighted 20%
# + EEM (an emerging-mkts fund) weighted 20%
# + AGG (a bond fund) weighted 10%

asset_returns_long <-
  prices %>%
  to.monthly(indexAt = "lastof", OHLC = FALSE) %>%
  tk_tbl(preserve_index = TRUE, rename_index = "date") %>%
  gather(asset, returns, -date) %>%
  group_by(asset) %>%
  mutate(returns = (log(returns) - log(lag(returns)))) %>%
  na.omit()

portfolio_returns_tq_rebalanced_monthly <-
  asset_returns_long %>%
  tq_portfolio(assets_col  = asset,
```

```
                returns_col = returns,
                weights     = w,
                col_rename  = "returns",
                rebalance_on = "months")
```

```
temp <- tempfile()

base <- "http://mba.tuck.dartmouth.edu/pages/faculty/ken.french/ftp/"

factor <-  "Global_3_Factors"

format<- "_CSV.zip"

full_url <- glue(base,
                 factor,
                 format,
                 sep = "")

download.file(full_url,
              temp,
              quiet = TRUE)


# Global_3_Factors <-
#   read_csv(unz(temp,
#                 "Global_3_Factors.csv"))
#
# head(Global_3_Factors)

# skip 6 rows so we can have the right format
Global_3_Factors <-
  read_csv(unz(temp,
               "Global_3_Factors.csv"),
    skip = 6)
```

```
## Warning: Missing column names filled in: 'X1' [1]
```

```
## Warning: 1 parsing failure.
## row col   expected    actual         file
## 349  -- 5 columns 1 columns <connection>
```

```
head(Global_3_Factors)
```

```
## # A tibble: 6 x 5
##   X1     `Mkt-RF` SMB   HML   RF
##   <chr>  <chr>    <chr> <chr> <chr>
## 1 199007 0.77     0.55  -0.40 0.68
```

```
## 2 199008 -10.77   -1.49 0.53  0.66
## 3 199009 -11.88   1.35  0.71  0.60
## 4 199010 9.36     -7.57 -4.16 0.68
## 5 199011 -3.72    1.37  1.13  0.57
## 6 199012 1.11     -0.72 -1.52 0.60
```

```
# X1 = formatted dates
# Mkt-Rf for the market returns above the risk-free rate
# SMB for the size factor
# HML for the value factor
# RF for the risk-free rate

#check data type
map(Global_3_Factors, class)
```

```
## $X1
## [1] "character"
##
## $`Mkt-RF`
## [1] "character"
##
## $SMB
## [1] "character"
##
## $HML
## [1] "character"
##
## $RF
## [1] "character"
```

```
Global_3_Factors <-
  read_csv(unz(temp,
               "Global_3_Factors.csv"),
           skip = 6) %>%
  rename(date = X1) %>%
  mutate_at(vars(-date), as.numeric) %>%
  mutate(date =
           rollback(ymd(parse_date_time(date, "%Y%m") + months(1)))) %>%
  filter(
    date >=
      first(portfolio_returns_tq_rebalanced_monthly$date) & date <=
      last(portfolio_returns_tq_rebalanced_monthly$date)
  )
```

```
## Warning: Missing column names filled in: 'X1' [1]

## Warning: 1 parsing failure.
```

```
## row col  expected    actual         file
## 349  -- 5 columns 1 columns <connection>

## Warning in mask$eval_all_mutate(quo): NAs introduced by coercion

## Warning in mask$eval_all_mutate(quo): NAs introduced by coercion

## Warning in mask$eval_all_mutate(quo): NAs introduced by coercion

## Warning in mask$eval_all_mutate(quo): NAs introduced by coercion

## Warning: 29 failed to parse.
```

```
head(Global_3_Factors)
```

```
## # A tibble: 6 x 5
##   date       `Mkt-RF`   SMB   HML    RF
##   <date>        <dbl> <dbl> <dbl> <dbl>
## 1 2013-01-31     5.46  0.17  2.01     0
## 2 2013-02-28     0.09  0.35 -0.76     0
## 3 2013-03-31     2.29  0.85 -2.02     0
## 4 2013-04-30     3.02 -1.13  0.9      0
## 5 2013-05-31     0.56 -0.66  0.97     0
## 6 2013-06-30    -2.52  0.21 -0.06     0
```

9.3 Carhart Four-factor model

(Carhart, 1997)

$$R_{it} - R_{ft} = \alpha_{it} + \beta_1(R_{Mt} - R_{ft}) + \beta_2 SMB_t + \beta_3 HML_t + \beta_4 UMD_t$$

where

- R_{it} = total return of a stock or portfolio i at time t
- R_{ft} = risk free rate of return at time t
- R_{Mt} = total market portfolio return at time t
- $R_{it} - R_{ft}$ = expected excess return
- $R_{Mt} - R_{ft}$ = excess return on the market portfolio (index)
- SMB_t = size premium (small - big)
- HML_t = value premium (high - low)

- UMD_t = premium on winners minus losers.
- $\beta_{1,2,3}$ = factor coefficients

9.4 Fama and French Five-factor model

(Fama and French, 2015) Accounting additionally for profitability and investment factors

$$R_{it}-R_{ft} = \alpha_{it}+\beta_1(R_{Mt}-R_{ft})+\beta_2 SMB_t+\beta_3 HML_t+\beta_4 RMW_t+\beta_5 CMA_t+e_{it}$$

where

- R_{it} = total return of a stock or portfolio i at time t
- R_{ft} = risk free rate of return at time t
- R_{Mt} = total market portfolio return at time t
- $R_{it} - R_{ft}$ = expected excess return
- $R_{Mt} - R_{ft}$ = excess return on the market portfolio (index)
- SMB_t = size premium (small - big)
- HML_t = value premium (high - low)
- $\beta_{1,2,3}$ = factor coefficients
- RMW_t is the return spread of the most profitable firms - the least profitable
- CMA_t is the return spread of firms that invest conservatively minus aggressively .

9.5 Multi-Factor Model

$$R_i = \alpha_i + \beta_{i(m)} Rm + \beta_{i(1)} F_1 + ... + \beta_{i(n)} F_n + \epsilon_i$$

where

- R_i = return of security
- Rm = market return

- $F(1,...N)$ = factors used

Factors can be

- Macroeconomic: employment, inflation, interest.
- Fundamental: (underlying financials): earnings, market cap, debt levels.
- Statistical: historical data modeling.

Type of Multi-Factor Models

- Bomicnaiton mdoel:

9.6 Event study

9.6.1 Abnormal Return

https://rpubs.com/gregoryarobinson/315096

https://www.eventstudytools.com/introduction-event-study-methodology

```
# install.packages("EventStudy")
library(EventStudy)
```

Another package

https://cran.r-project.org/web/packages/eventstudies/vignettes/replications.pdf

```
library(eventstudies)
```

Replication of (Chen and Siems, 2004)

```
data(TerrorIndiceReturns)
data(TerrorAttack)
TerrorIndiceCAR <- lapply(1: ncol(TerrorIndiceReturns), function(x){
# 10-day window around the event
event <- phys2eventtime(na.omit(TerrorIndiceReturns[ , x,
drop = FALSE]),
TerrorAttack,
10)
# Estimate ARs
esMean <- constantMeanReturn(event$z.e[which(attributes(event$z.e)$index
%in% -30:-11), ],
residual = FALSE)
ar <- event$z.e - esMean
ar <- window(ar, start = 0, end = 10)
# CAR
car <- remap.cumsum(ar, base = as.numeric(ar[1, 1]))
```

```
names(car) <- colnames(TerrorIndiceReturns[ , x,
drop = FALSE])
return(car)
})
names(TerrorIndiceCAR) <- colnames(TerrorIndiceReturns)
# Compile for all indices
TerrorIndiceCAR <- do.call(cbind, TerrorIndiceCAR)
# 11-day CAR
TerrorIndiceCAR[11, ]
```

```
##       S.P500       Dow      NYSE    NASDAQ   Toronto    Mexico    London
## 10 -3.832794 -7.897366 -3.981837 -9.993519 -9.110412 -6.53973 -8.602633
##      Germany Eurostoxx50    France     Spain Switzerland   Austria     Italy
## 10 -10.45718   -7.596024 -9.518721 -10.6724   -7.292201 -7.894399 -15.21462
##      Belgium Amsterdam Portugal  Helsinki    Norway    Sweden     Tokyo
## 10 -9.219016 -10.82756 1.698079 -5.500048 -12.39281 -4.691903 -3.603373
##     HongKong   S.Korea     India   Jakarta Singapore Kualalampur Australia
## 10 -5.231521 -16.64571 -17.02777 -9.933654 -17.98219   -14.55417 -8.597988
##          NZL  Pakistan     Saudi    Israel     Jberg
## 10 -5.635766 -12.42189 -11.32466 -7.81159 -12.59022
```

```
TerrorIndiceCAR[6, ] # 6-day CAR
```

```
##     S.P500       Dow      NYSE    NASDAQ  Toronto    Mexico    London   Germany
## 5 -7.72379 -10.56695 -8.094739 -10.13781 -7.01292 -13.17444 -4.533527 -8.063819
##   Eurostoxx50    France     Spain Switzerland   Austria     Italy   Belgium
## 5   -5.940704 -8.707865 -9.426443   -5.967055 -4.428614 -13.98185 -8.510382
##   Amsterdam  Portugal  Helsinki    Norway    Sweden      Tokyo  HongKong
## 5 -8.521101 -6.875878 -4.802975 -9.894391 -4.96208 -0.5636291 -5.570385
##     S.Korea     India   Jakarta Singapore Kualalampur Australia       NZL
## 5 -11.81901 -11.77841 -4.91951 -13.74157   -10.50214 -6.814062 -7.101218
##   Pakistan     Saudi    Israel     Jberg
## 5 -10.3741 -5.320904 -11.03399 -12.09548
```

Chapter 10

Advertising

People don't like intrusive ads, especially when they are vulnerable (Tybout and calkins, 2005, p.143)

Chapter 11

Communication

11.1 Information Theory

11.1.1 Entropy

Most of the examples are taken from `philentropy` package in R, which can be accessed via this link

11.1.1.1 Discrete Entropy

If $\mathbf{p}_X = (p_{x1}, p_{x2}, ..., p_{xn})$ is the probability distribution of the discrete random variable X with $P(X = x) = p_{xi}$. Then, the **entropy** of the distribution is (Shannon, 1948)

$$H(\mathbf{p}_X) = -\sum_{x_i} p_{xi} \log_2 p_{xi}$$

where

- entropy unit is **bits**, and $H(p) \in [0,1]$ and max at $p = 0.5$
- or the unit is **nats** (log_e)
- or the unit is **dits** (log_{10})

In general, if you have different bases, you can convert between units. Given bases a and b

$$\log_a k = \log_a b \times log_b k$$

where $\log_a b$ is a constant.

Generalize

$$H_b(X) = log_b a \times H_a(X)$$

Equivalently,

$$H(X) = E(\frac{1}{\log p(x)})$$

Entropy is the measure of how much information gained from learning the value of X (Zwillinger, 1995, pp.262). Entropy depends on the distribution of X.

Example:

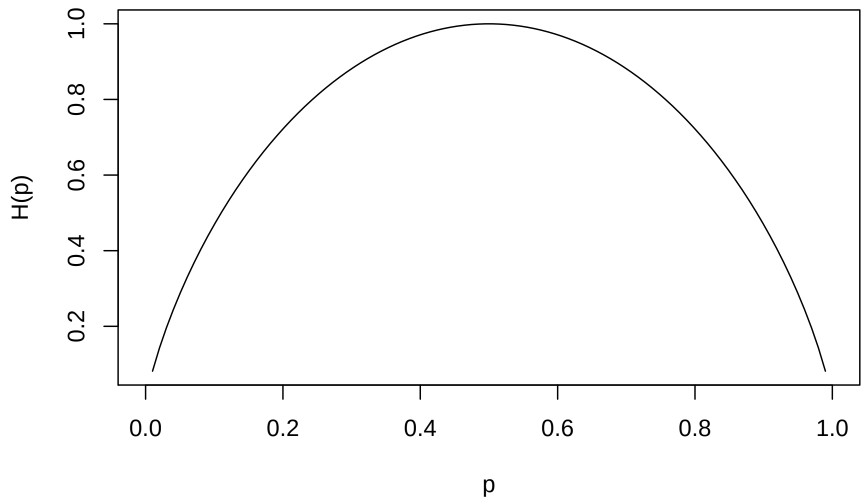
If X has 2 values, then $\mathbf{p} = (p, 1-p)$ and

$$H(\mathbf{p}_X) = H(p, 1-p) = -p\log_2 p - (1-p)\log_2(1-p)$$

which is known as binary entropy function

Plot of p vs. H(p) where max of $H(\mathbf{p}_X) = \log_2 n$ when $X \sim U$

```
hb <- function(gamma) {
    -gamma * log2(gamma) - (1 - gamma) * log2(1 - gamma) # output in bits
}

xs <- seq(0, 1, len = 100)
plot(xs,
     hb(xs),
     type = 'l',
     xlab = "p",
     ylab = "H(p)")
```




```
library("philentropy")
Prob <- 1:10/sum(1:10) # probabilities P(X)
H(Prob) # Shannon's Entropy
```

```
## [1] 3.103643
```

11.1.1.2 Mutual Information

If X and Y are two discrete random variables and $\mathbf{p}_{X \times Y}$ is the their joint distribution, the **mutual information** (i.e., learning of a value of X gives information about the value of Y) of X and Y is

$$I(X,Y) = I(Y,X) = H(\mathbf{p}_X) + H(\mathbf{p}_Y) - H(\mathbf{p}_{X \times Y})$$

where $I(X,Y) \geq 0$ and $I(X,Y) = 0$ iff X and Y are independent (information about X gives no information about Y).

Equivalently,

$$MI(X,Y) = \sum_{i=1}^{n} \sum_{j=1}^{m} P(x_i, y_j) \times \log_b\left(\frac{P(x_i, y_j)}{P(x_i) \times P(y_j)}\right)$$

Properties:

- $I(X;X) = H(X)$ known as self-information

- $I(X;Y) \geq 0$ with equality iff $X \perp Y$

```
P_x <- 1:10/sum(1:10) # distribution P(X)
P_y <- 20:29/sum(20:29) # distribution P(Y)
P_xy <- 1:10/sum(1:10) # joint distribution P(X,Y)
MI(P_x, P_y, P_xy) # Mutual Information
```

```
## [1] 3.311973
```

11.1.1.3 Relative Entropy

The measure of the distance between distribution p and q is called **relative entropy**, denoted by $D(p||q)$

Equivalently, it measures the inefficiency of assuming distribution q when the true distribution is p.

The relative entropy between p(x) and q(x) is

$$D(p||q) = \sum_x p(x) \log \frac{p(x)}{q(X)}$$

where

- $D(p||q) \geq 0$ and equality iff $p = q$

11.1.1.4 Joint-Entropy

$$H(X,Y) = -\sum_{i=1}^{n} \sum_{j=1}^{m} P(x_i, y_j) \times \log_b(P(x_i, y_j))$$

```
P_xy <- 1:10/sum(1:10) # joint distribution P(X,Y)
JE(P_xy) # Joint-Entropy
```

```
## [1] 3.103643
```

11.1.1.5 Conditional Entropy

$$H(Y|X) = -\sum_{i=1}^{n} \sum_{j=1}^{m} P(x_i, y_j) \times \log_b(\frac{P(x_i)}{P(x_i, y_j)})$$

```
P_x <- 1:10/sum(1:10) # distribution P(X)
P_y <- 1:10/sum(1:10) # distribution P(Y)
CE(P_x, P_y) # Joint-Entropy
```

```
## [1] 0
```

Relation among entropy measures

$$H(X,Y) = H(X) + H(Y|X) = H(Y) + H(X|Y)$$

Properties:

- Chain rule: $H(X_1, ..., X_n) = \sum_{i=1}^{n} H(X_i|X_{i-1}, ..., X_1)$
- $0 \leq H(Y|X) \leq H(Y)$ Information on X does not increase the entropy measure of Y
- If $X \perp Y$, then $H(Y|X) = H(Y); H(X,Y) = H(X) + H(Y)$
- $H(X) \leq log|n_X|$ where n_X = number of elements in X
- $H(X) = \log(n_X)$ iff $X \sim U$
- $H(X_1, ..., X_n) \leq \sum_i^n H(X_i)$
- $H(X_1, ..., X_n) = \sum_i^n H(X_i)$ iff X_i's are mutually independent.

11.1.1.6 Continuous Entropy

If X is a continuous variable, its entropy is

$$h(\mathbf{X}) = -\int_{R^d} p(\mathbf{x}) \log p(\mathbf{x}) dx$$

where d is the dimension of X.

If X and Y are continuous random variables with density functions $p(\mathbf{x})$ and $q(\mathbf{y})$, the relative entropy is

$$H(X,Y) = \int_{R^d} p(x) \log \frac{p(x)}{q(x)} dx$$

More advance analysis can be access here

11.1.2 Divergence

Three metrics for divergence can be found here:

- Kullback-Leibler
- Jensen-Shannon
- Generalized Jensen-Shannon

11.1.3 Channel Capacity

The information channel capacity of a discrete memoryless channel is

$$C = \max_{p(x)} I(X;Y)$$

which is the highest rate use at which information can be sent without much error.
More information can be found here

Chapter 12

Metrics

12.1 Return on Investment (ROI)

$$ROI = \frac{\text{Net Profit}}{\text{Investment}}$$

Similarly, return on marketing investment

$$ROIM = \frac{IRAM - CM}{MS}$$

where

- IRAM = Incremental Revenue Attributable to Marketing
- CM = Cost of the Marketing Investment
- MS = Marketing Spending

If ROMI is positive, then firm is on doing well.

12.2 Economic Value Added

- also known as economic profit.
- is a measure based on the residual wealth calculated by deducting its cost of capital from its operating profit, adjusted for taxes on a cash basis.
- measures the value a company generates from its invested funds.

- EVA relies heavily on invested capital. Hence, more suitable for asset-rich companies, whereas companies with intangible assets, such as technology businesses, may not be good candidates.

$$EVA = NOPAT - (InvestedCapital * WACC)$$

where

- NOPAT = Net Operating profit after taxes = Operating Profit x (1-Tax Rate)

- Invested capital = Debt + capital leases + shareholders' equity = Equity + long-term debt at the beginning of the period

- WACC = Weighted average cost of capital (average rate of return a company expects to pay its investors).
  - $WACC = \frac{Ke \times E}{E+D} + \frac{Kd \times (1-t) \times D}{E+D}$
    * Ke = required return on equity

    * Kd(1-t) = after tax return on debt.

- (WACC* capital invested) is also known as a **finance charge**

Invested capital can also be calculated as (Total Assets - Current Liabilities). Hence, the modified version of EVA is:

$$EVA = NOPAT - (\text{total assets} - \text{current liabilities}) * WACC$$

12.3 Market Value Added

- is the amount of wealth that a company is able to create for its stakeholders since its foundation.

- (the current market value of the company's stock - the initial invested capital)

$$MVA = \text{market value of shares (or enterprise value)} - \text{book value of shareholders' equity}$$

12.4 Unexpected size-adjusted advertising investments

(Chakravarty and Grewal, 2016; Kim and McAlister, 2011; Liu et al., 2017) used

12.5 Shareholder Complaints

(Wies et al., 2019) used data from RiskMetrics

12.6 Profitability

(Grewal et al., 2008; McAlister et al., 2016)

$$\text{profitability} = \frac{\text{operating income before depreciation}}{\text{total assets}}$$

12.7 Firm Size

(Grewal et al., 2010; McAlister et al., 2016; Nezami et al., 2018) used log of total asset in million as firm size

$$\text{firm size} = \log(\text{total assets})$$

Sales Growth (Grewal et al., 2010; Nezami et al., 2018; Rao et al., 2004)

Percentage change in gross sales = sales growth

12.8 Financial Flexibility

12.8.1 Cash flows

(Chakravarty and Grewal, 2011; Malshe and Agarwal, 2015) Cash flows = log(operating cash flow in mil)

12.8.2 Financial Leverage

(Kashmiri and Mahajan, 2017; Chakravarty and Grewal, 2011)

$$\text{financial leverage} = \frac{\text{long-term debt}}{\text{book value of assets}}$$

12.9 Stock return

(Markovitch et al., 2005; Chakravarty and Grewal, 2011, 2016) used dummy if stock returns exceed industry averaged stock returns

12.10 Financial Flexibility

12.11 Net Contribution

Net contribution = gross margin x sales - cost of marketing

$$NC = m \times S(a) - ka$$

Chapter 13

Data

13.1 WRDS

Retrieve data from WRDS

```
# to set up connection from R to WRDS (https://wrds-www.wharton.upenn.edu/pages/support/programm
library(RPostgres)
library(dplyr)

# I've set up wrds connection before hand. Please use your username and password here.

# wrds <- dbConnect(Postgres(),
#                   host='wrds-pgdata.wharton.upenn.edu',
#                   port=9737,
#                   dbname='wrds',
#                   sslmode='require',
#                   user='')

# Check variables (column headers) in COMP ANNUAL FUNDAMENTAL
#uses the already-established wrds connection to prepare the SQL query string and save the query
# check avaiable databases: https://wrds-web.wharton.upenn.edu/wrds/tools/variable.cfm?vendor_id=
res <- dbSendQuery(wrds, "select column_name
                   from information_schema.columns
                   where table_schema='compa'
                   and table_name='funda'
                   order by column_name")
data <- dbFetch(res, n=-1) # fetches the data that results from running the query res against wr
dbClearResult(res) # closes the connection
head(data)
```

```
##   column_name
```

```
## 1       acchg
## 2        acco
## 3       accrt
## 4     acctchg
## 5     acctstd
## 6        acdo
```

```
# select everything
res <- dbSendQuery(wrds, "select * from compa.funda")

# from compa.funda

# only select the following variables
res <- dbSendQuery(wrds, "select gvkey, datadate, fyear, indfmt, consol, popsrc, datafm
```

```
## Warning in result_create(conn@ptr, statement): Closing open result set,
## cancelling previous query
```

```
# res <- dbSendQuery(wrds, "select gvkey, gp from compa.funda") #check variables from
data1 <- dbFetch(res, n=-1)
dbClearResult(res)

data = data1 %>%
    distinct(gvkey,datadate,fyear,tic,conm,.keep_all = T)
```

13.2 YouTube

- YouTube Ads
- YouTube Analytics and Reporting APIs: All YouTube Analytics and YouTube Reporting API requests must be authorized by the channel or content owner that owns the requested data

13.2.1 OAuth

Go to link to access your API

```
library(tuber)
app_id = "YOUR APP ID"
app_secret = "YOUR APP SECRET"
yt_oauth(app_id, app_secret)


get_stats(video_id = "N708P-A45D0")
```

Note for Ubuntu user

```
httr::set_config(httr::config( ssl_verifypeer = 0L ) )
```

Get info about a video

```
get_video_details(video_id="N708P-A45D0")
```

Get caption of a video

```
get_captions(video_id="yJXTXN4xrI8")
```

Search Videos

```
res <- yt_search(term = "test")
head(res[, 1:3])
```

Get Comments of a video

```
res <- get_comment_threads(c(video_id="N708P-A45D0"))
head(res)
```

Get All the Comments Including Replies

```
get_all_comments(video_id = "a-UQz7fqR3w")
```

Get statistics of all the videos in a channel

```
a <- list_channel_resources(filter = c(channel_id = "UCT5Cx1l4IS3wHkJXNyuj4TA"), part="contentDet

# Uploaded playlists:
playlist_id <- a$items[[1]]$contentDetails$relatedPlaylists$uploads

# Get videos on the playlist
vids <- get_playlist_items(filter= c(playlist_id=playlist_id))

# Video ids
vid_ids <- as.vector(vids$contentDetails.videoId)

# Function to scrape stats for all vids
get_all_stats <- function(id) {
  get_stats(id)
}

# Get stats and convert results to data frame
res <- lapply(vid_ids, get_all_stats)
res_df <- do.call(rbind, lapply(res, data.frame))

head(res_df)
```

13.2.2 API

```
require(curl)
require(jsonlite)
library(kableExtra)
library(dplyr)
library(ggplot2)
library(plotly)
library(reshape2)

API_key = "YOUR API KEY"

getstats_video<-function(video_id,API_key){

  url=paste0("https://www.googleapis.com/youtube/v3/videos?part=snippet,statistics&id="
  result <- fromJSON(txt=url)
  salida=list()
  return(data.frame(name=result$items$snippet$channelTitle, result$items$statistics,ti
}

get_playlist_canal<-function(id,API_key,topn=15){

  url=paste0('https://www.googleapis.com/youtube/v3/playlistItems?part=contentDetails&p

  result=fromJSON(txt=url)
  return(data.frame(result$items$contentDetails))
}

getstats_canal<-function(id,API_key){

url=paste0('https://www.googleapis.com/youtube/v3/channels?part=snippet%2CcontentDetai

result <- fromJSON(txt=url)
return(data.frame(name=result$items$snippet$title,result$items$statistics,pl_list_id=re

}

getall_channels<-function(ids,API_key,topn=5){

  videos=lapply(ids,FUN=get_playlist_canal,API_key=API_key,topn=topn) %>% bind_rows()

  stats=lapply(videos[,1],FUN=getstats_video,API_key=API_key)
  stats=bind_rows(stats)
  stats$vid_id=videos[,1]
  return(stats)
```

```
}
```

Statistics per Channel

```
can_st=lapply(comp_data$cha_id,FUN = getstats_canal,API_key=API_key)
can_st=bind_rows(can_st)
can_st$viewCount=as.numeric(can_st$viewCount)
can_st[,1:6] %>% kable() %>%kable_styling()
```

```
can_st$viewCount=round(as.numeric(can_st$viewCount)/1000000,2)
p1=can_st %>% ggplot(aes(x=reorder(name,viewCount),y=viewCount,fill=name))+
geom_bar(stat="sum")+guides(size=F)+coord_flip()+scale_fill_manual(values = c("red", "darkblue",
geom_text(inherit.aes = T,aes(label=paste(viewCount,"M")),nudge_y =0,angle = 90)+
labs(x="Total Visualizations(Millions)",y="Visualizations",fill="")+
theme(legend.position = "top")+mytheme3

  ggplotly(p1,tooltip=c("name","viewCount"))  %>%
    layout(legend = list(orientation = "h",x = 0.01, y = -0.1,autosize=F))
```

Information on Individual Videos per Channel

```
var_to_see="dislikeCount" #favoriteCount or commentCount

info=getall_channels(ids = can_st$pl_list_id.uploads,API_key = API_key,topn =20)


 datacond=melt(info[,c(1:6,8)],id.vars = c("name","date"))
 datacond$date=as.Date(datacond$date)
 datacond$value=as.numeric(datacond$value)



ggplot(filter(datacond,variable==var_to_see),aes(x=as.Date(date),y=value,fill=name))+
geom_bar(stat="sum")+labs(x=var_to_see,y="",fill="")+guides(size=FALSE)+scale_fill_manual(values
  scale_x_date(limits =as.Date(c(as.Date(min(datacond$date)),as.Date(Sys.time()))),date_breaks  =
```

13.2.3 Python

Pytube

```
# from pytube import YouTube
# YouTube('https://youtube.com/watch?v=2lAe1cqCOXo').streams.first().download()
```

```
import requests
import json
r = requests.get("http://gdata.youtube.com/feeds/api/standardfeeds/top_rated?v=2&alt=jsonc")
#r.text
data = json.loads(r.text)
data
```

```
# for item in data['data']['items']:
#   print "Video Title: %s" % (item['title'])
#   print "Video Category: %s" % (item['category'])
#   print "Video ID: %s" % (item['id'])
#   print "Video Rating: %f" % (item['rating'])
#   print "Embed URL: %s" % (item['player']['default'])
```

Using `BeautifulSoup`

```
from requests_html import HTMLSession
from bs4 import BeautifulSoup as bs # importing BeautifulSoup

# sample youtube video url
video_url = "https://www.youtube.com/watch?v=jNQXAC9IVRw"
# init an HTML Session
session = HTMLSession()
# get the html content
response = session.get(video_url)
# execute Java-script
response.html.render(sleep=1)
# create bs object to parse HTML
soup = bs(response.html.html, "html.parser")
# write all HTML code into a file
open("video.html", "w", encoding='utf8').write(response.html.html)
```

Generate random video

```
import json
import urllib.request
import string
import random

count = 50
API_KEY = 'your_key'
random = ''.join(random.choice(string.ascii_uppercase + string.digits) for _ in range(3

urlData = "https://www.googleapis.com/youtube/v3/search?key={}&maxResults={}&part=snipp
webURL = urllib.request.urlopen(urlData)
data = webURL.read()
encoding = webURL.info().get_content_charset('utf-8')
results = json.loads(data.decode(encoding))

for data in results['items']:
    videoId = (data['id']['videoId'])
    print(videoId)
    #store your ids
```

Random used by YouTube API, test

```
# -*- coding: utf-8 -*-

# Sample Python code for youtube.search.list
# See instructions for running these code samples locally:
# https://developers.google.com/explorer-help/guides/code_samples#python

import os

import google_auth_oauthlib.flow
import googleapiclient.discovery
import googleapiclient.errors

scopes = ["https://www.googleapis.com/auth/youtube.force-ssl"]

def main():
    # Disable OAuthlib's HTTPS verification when running locally.
    # *DO NOT* leave this option enabled in production.
    os.environ["OAUTHLIB_INSECURE_TRANSPORT"] = "1"

    api_service_name = "youtube"
    api_version = "v3"
    client_secrets_file = "YOUR_CLIENT_SECRET_FILE.json"

    # Get credentials and create an API client
    flow = google_auth_oauthlib.flow.InstalledAppFlow.from_client_secrets_file(
        client_secrets_file, scopes)
    credentials = flow.run_console()
    youtube = googleapiclient.discovery.build(
        api_service_name, api_version, credentials=credentials)

    request = youtube.search().list(
        part="snippet",
        maxResults=25,
        q="surfing"
    )
    response = request.execute()

    print(response)

if __name__ == "__main__":
    main()
```

Another way, but this is a dirty crawler, not for production

```
import re, urllib
from random import randint

def random_str(str_size):
    res = ""
    for i in xrange(str_size):
        x = randint(0,25)
        c = chr(ord('a')+x)
        res += c
    return res

def find_watch(text,pos):
    start = text.find("watch?v=",pos)
    if (start<0):
        return None,None
    end = text.find(" ",start)
    if (end<0):
        return None,None

    if (end-start > 200): #silly heuristics, probably not a must
        return None,None

    return text[start:end-1], start


def find_instance_links():
    base_url = 'https://www.youtube.com/results?search_query='
    url = base_url+random_str(3)
    #print url

    r = urllib.urlopen(url).read()

    links = {}

    pos = 0
    while True:
        link,pos =  find_watch(r,pos)
        if link == None or pos == None:
            break
        pos += 1
        #print link
        if (";" in link):
            continue
        links[link] = 1
```

```
    items_list = links.items()

    list_size = len(items_list)
    selected = randint(list_size/2,list_size-1)
    return items_list[selected][0]


for i in xrange(1000):
    sleep(randint(7,20)) # pause randomly between 7 and 20 seconds
    link = find_instance_links()
    print link
```

Chapter 14

Modeling Application

14.1 Transformation

14.1.1 Log-transformation

(Manchanda et al., 2004; Wies et al., 2019) used 1 in place of 0 for log-transformation. And also use 0.5 and 0.0001 for sensitivity analysis.

To control for firm size effects, (Wies et al., 2019) scale advertising investments by the firm's total assets in the given year

Chapter 15

Models

This theoretical/analytical model part of this section comes mostly from professor Murali Mantrala's Marketing Model Seminar.

Marketing models consists of

1. Analytical Model: pure mathematical-based research
2. Empirical Model: data analysis.

"A model is a representation of the most important elements of a perceived real-world system".

Marketing model improves decision-making

- Econometric models
  - Description
  - Prediction
  - Simulation
- Optimization models
  - maximize profit using market response model, cost functions, or any constraints.
- Quasi- and Field experimental analyses
- Conjoint Choice Experiments.

"A decision calculus will be defined as a model-based set of procedures for processing data and judgments to assist a manager in his decision making"(Little, 1976):

- simple
- robust
- easy to control

- adaptive
- as complete as possible
- easy to communicate with

15.1 Market Response Model

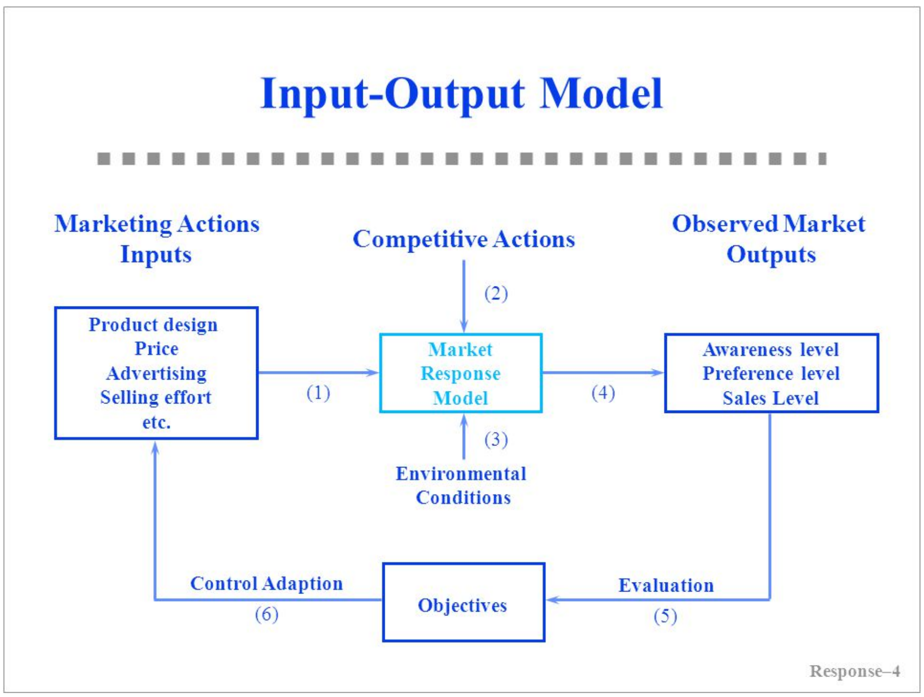
Marketing Inputs:

- Selling effort
- advertising spending
- promotional spending

↓

Marketing Outputs:

- sales
- share
- profit
- awareness



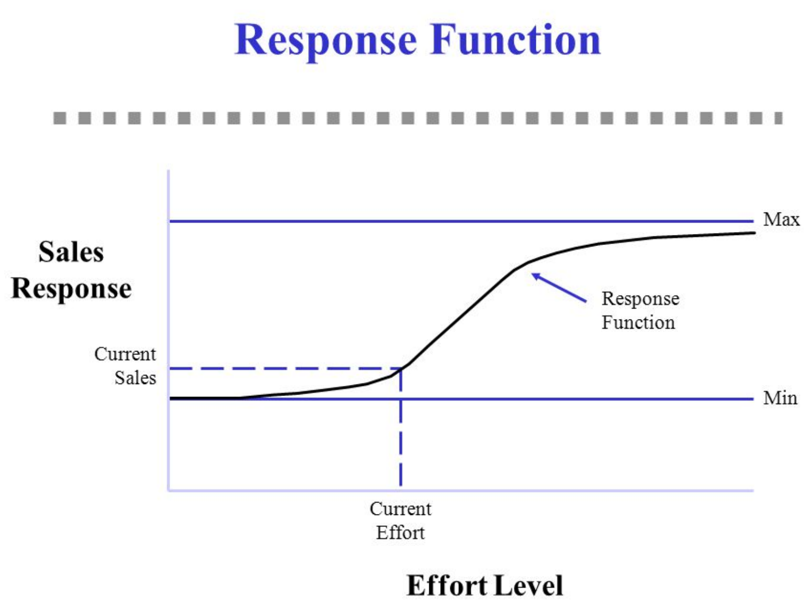
Give phenomena for a good model:

- P1: Dynamic sales response involves a sales **growth rate** and a sales **decay rate** that are different

- P2: Steady-state response can be **concave or S-shaped**. Positive sales at 0 adverting.
- P3: **Competitive effects**
- P4: Advertising effectiveness dynamics due to changes in media, copy, and other factors.
- P5: Sales still increase or fall off even as advertising is held constant.

Saunder (1987) phenomena

- P1: Output = 0 when Input = 0
- P2: The relationship between input and output is linear
- P3: Returns decrease as the scale of input increases (i.e., additional unit of input gives less output)
- P4: Output cannot exceed some level (i.e., saturation)
- P5: Returns increase as scale of input increases (i.e., additional unit of input gives more output)
- P6: Returns first increase and then decrease as input increases (i.e., S-shaped return)
- P7: Input must exceed some level before it produces any output (i.e., threshold)
- P8: Beyond some level of input, output declines (i.e., supersaturation point)



Aggregate Response Models

- Linear model: $Y = a + bX$
  - Through origin
  - can only handle constant returns to scale (i.e., can't handle concave, convex, and S-shape)
- The Power Series/Polynomial model: $Y = a + bX + cX^2 + dX^3 + ...$
  - can't handle saturation and threshold
- Fraction root model/ Power model: $Y = a + bX^c$ where c is prespecified
  - c = 1/2, called **square root model**
  - c = -1, called **reciprocal model**
  - c can be interpreted as elasticity if a = 0.
  - c = 1, linear
  - c <1, decreasing return
  - c>1, increasing returns
- Semilog model: $Y = a + b \ln X$
  - Good when constant percentage increase in marketing effort (X) result in constant absolute increase in sales (Y)
- Exponential model: $Y = ae^{bX}$ where X >0
  - b > 0, increasing returns and convex
  - b < 0, decreasing returns and saturation
- Modified exponential model: $Y = a(1 - e^{-bX}) + c$
  - Decreasing returns and saturation
  - upper bound = a + c
  - lower bound = c
  - typically used in selling effort
- Logistic model: $Y = \frac{a}{a+e^{-(b+cX)}} + d$
  - increasing return followed by decreasing return to scale, S-shape
  - saturation = a + d
  - good with saturation and s-shape
- Gompertz model
- ADBUDG model (Little, 1970) : $Y = b + (a - b)\frac{X^c}{d+X^c}$
  - c > 1, S-shaped

  - 0 < c < 1
    * Concave
    * saturation effect
    * upper bound at a
    * lower bound at b
  - typically used in advertising and selling effort.
  - can handle, through origin, concave, saturation, S-shape
- Additive model for handling multiple Instruments: $Y = af(X_1) + bg(X_2)$
- Multiplicative model for handling multiple instruments: $Y = aX_1^b X_2^c$ where c and c are elasticities. More generally, $Y = af(X_1) \times bg(X_2)$
- Multiplicative and additive model: $Y = af(X_1) + bg(X_2) + cf(X_1)g(X_2)$
- Dynamic response model: $Y_t = a_0 + a_1 X_t + \lambda Y_{t-1}$ where a_1 = current effect, λ = carry-over effect

Dynamic Effects

- Carry-over effect: current marketing expenditure influences future sales
  - Advertising adstock/ advertising carry-over is the same thing: lagged effect of advertising on sales
- Delayed-response effect: delays between when marketing investments and their impact
- Customer holdout effects
- Hysteresis effect
- New trier and wear-out effect
- Stocking effect

Simple Decay-effect model:

$$A_t = T_t + \lambda T_{t-1}, t = 1, ...,$$

where

- A_t = Adstock at time t
- T_t = value of advertising spending at time t
- λ = decay/ lag weight parameter

Response Models can be characterized by:

1. The number of marketing variables
2. whether they include competition or not

3. the nature of the relationship between the input variables
    1. Linear vs. S-shape
4. whether the situation is static vs. dynamic
5. whether the models reflect individual or aggregate response
6. the level of demand analyzed
    1. sales vs. market share

Market Share Model and Competitive Effects: $Y = M \times V$ where

- Y = Brand sales models
- V = product class sales models
- M = market-share models

Market share (attraction) models

$$M_i = \frac{A_i}{A_1 + .. + A_n}$$

where A_i attractiveness of brand i

Individual Response Model:

Multinomial logit model representing the probability of individual `i` choosing brand `l` is

$$P_{il} = \frac{e^{A_{il}}}{\sum_j e^{A_{ij}}}$$

where

- A_{ij} = attractiveness of product j for individual i $A_{ij} = \sum_k w_k b_{ijk}$
- b_{ijk} = individual i's evaluation of product j on product attribute k, where the summation is over all the products that individual `i` is considering to purchase
- w_k = importance weight associated with attribute k in forming product preferences.

15.2 Marketing Resource Allocation Models

This section is based on (Mantrala et al., 1992)

15.2.1 Case study 1

Concave sales response function

- Optimal vs. proportional at different investment levels
- Profit maximization perspective of aggregate function

$$s_i = k_i(1 - e^{-b_i x_i})$$

where

- s_i = current-period sales response (dollars / period)
- x_i = amount of resource allocated to submarket i
- b_i = rate at which sales approach saturation
- k_i = sales potential

Allocation functions

- Fixed proportion
  - R_i = Investment level (dollars/period)
  - w_i = fixed proportion or weights

$$\hat{x}_i = w_i R; \sum_{t=1}^{2} w_t = 1; 0 < w_t < 1$$

- Informed allocator
  - optimal allocations via marginal analysis (maximize profits)

$$maxC = m \sum_{i=1}^{2} k_i(1-e^{-b_i x_i}) x_1 + x_2 \leq R; x_i \geq 0 \text{ for } i = 1, 2 x_1 = \frac{1}{(b_1 + b_2)(b_2 R + \ln(\frac{k_1 b_1}{k_2 b_2})} x_2 = \frac{1}{(b_1 + b_2)(b_2 R + \ln(\frac{k_2 b_2}{k_1 b_1}}$$

15.2.2 Case study 2

S-shaped sales response function:

- Optimal vs. proportional at different investment levels
- Profit maximization perspective of aggregate function

15.2.3 Case study 3

Quadratic-form stochastic response function

- Optimal allocation only with risk averse and risk neutral investors.

15.3 Meta-analyses of Econometric Marketing Models

15.4 Dynamic Advertising Effects and Spending Models

15.5 Marketing Mix Optimization Models

Check this post for implementation in Python

15.6 New Product Diffusion Models

15.7 Two-sided Platform Marketing Models

Example of Marketing Mix Model in practice: link

15.8 Attribution Models

15.8.1 Ordered Shapley

Based on (Zhao et al., 2018) (to access paper) Cooperative game theory: look at the marginal contribution of each player in the game, where **Shapley value** (i..e, the credit assigned to each individual) is the expected value value of the marginal contribution over all possible permutations (e.g., all possible sequences) of the players.

Shapely value considered:

- marginal contribution of each player (i.e., channel)
- sequence of joining the coalition (i.e., customer journey).

Typically, we can't apply the Shapley Value method due to computational burden (you need all possible permutations). And a drawback is that all the credit must be divided among your channels, if you have missing channels, then it will distort the estimates of other channels' estimates.

It's hard to use Shapley value model for model comparison since we have no "ground truth"

Marketing application:

- Pardot Einstein features and more

```
library("GameTheory")
```

packages reference

15.8.2 Markov Model

Markov chains maps the movement and gives a probability distribution, for moving from one state to another state. A Markov Chain has three properties:

- **State space** – set of all the states in which process could potentially exist
- **Transition operator** –the probability of moving from one state to other state
- **Current state probability distribution** – probability distribution of being in any one of the states at the start of the process

In mathematically sense

$$w_{ij} = P(X_t = s_j | X_{t-1} = s_i), 0 \leq w_{ij} \leq 1, \sum_{j=1}^{N} w_{ij} = 1 \forall i$$

where

- The Transition Probability (w_{ij}) = The Probability of the Previous State (X_{t-1}) Given the Current State (X_t)
- The Transition Probability (w_{ij}) is No Less Than 0 and No Greater Than 1
- The Sum of the Transition Probabilities Equals 1 (i.e., Everyone Must Go Somewhere)

To examine a particular node in the Markov graph, we use **removal effect** (s_i) to see its contribution to a conversion. In another word, the Removal Effect is the probability of converting when a step is completely removed; all sequences that had to go through that step are now sent directly to the null node

Each node is called **transition states**
The probability of moving from one channel to another channel is called **transition probability**.

first-order or "memory-free" Markov graph is called "memory-free" because the probability of reaching one state depends only on the previous state visited.

- Order 0: Do not care about where the you came from or what step the you are on, only the probability of going to any state.
- Order 1: Looks back zero steps. You are currently at a state. The probability of going anywhere is based on being at that step.
- Order 2: Looks back one step. You came from state A and are currently at state B. The probability of going anywhere is based on where you were and where you are.
- Order 3: Looks back two steps. You came from state A after state B and are currently at state C. The probability of going anywhere is based on where you were and where you are.

- Order 4: Looks back three steps. You came from state A after B after C and are currently at state D. The probability of going anywhere is based on where you were and where you are.

15.8.2.1 Example 1

This section is by Analytics Vidhya

data link

```
# #Install the libraries
# install.packages("ChannelAttribution")
# install.packages("ggplot2")
# install.packages("reshape")
# install.packages("dplyr")
# install.packages("plyr")
# install.packages("reshape2")
# install.packages("markovchain")
# install.packages("plotly")

#Load the libraries
library("ChannelAttribution")
library("ggplot2")
library("reshape")
library("dplyr")
library("plyr")
library("reshape2")
library("markovchain")
library("plotly")

#Read the data into R
channel = read.csv("images/Channel_attribution.csv", header = T) %>% select(-c(Output)
head(channel, n = 2)
```

```
##   R05A.01 R05A.02 R05A.03 R05A.04 R05A.05 R05A.06 R05A.07 R05A.08 R05A.09
## 1      16       4       3       5      10       8       6       8      13
## 2       2       1       9      10       1       4       3      21      NA
##   R05A.10 R05A.11 R05A.12 R05A.13 R05A.14 R05A.15 R05A.16 R05A.17 R05A.18
## 1      20      21      NA      NA      NA      NA      NA      NA      NA
## 2      NA      NA      NA      NA      NA      NA      NA      NA      NA
##   R05A.19 R05A.20
## 1      NA      NA
## 2      NA      NA
```

The number represents:

- 1-19 are various channels
- 20 – customer has decided which device to buy;

- 21 – customer has made the final purchase, and;
- 22 – customer hasn't decided yet.

Pre-processing

```
for (row in 1:nrow(channel)){
    if (21 %in% channel[row,]){
        channel$convert = 1
    }
}

column = colnames(channel)
channel$path = do.call(paste, c(channel, sep = " > "))
head(channel$path)
```

```
## [1] "16 > 4 > 3 > 5 > 10 > 8 > 6 > 8 > 13 > 20 > 21 > NA > NA > NA > NA > NA > NA > NA > NA >
## [2] "2 > 1 > 9 > 10 > 1 > 4 > 3 > 21 > NA > NA > NA > NA > NA > NA > NA > NA > NA > NA > NA >
## [3] "9 > 13 > 20 > 16 > 15 > 21 > NA > NA > NA > NA > NA > NA > NA > NA > NA > NA > NA > NA >
## [4] "8 > 15 > 20 > 21 > NA > NA > NA > NA > NA > NA > NA > NA > NA > NA > NA > NA > NA > NA >
## [5] "16 > 9 > 13 > 20 > 21 > NA > NA > NA > NA > NA > NA > NA > NA > NA > NA > NA > NA > NA >
## [6] "1 > 11 > 8 > 4 > 9 > 21 > NA > NA > NA > NA > NA > NA > NA > NA > NA > NA > NA > NA > NA
```

```
for(row in 1:nrow(channel)){
  channel$path[row] = strsplit(channel$path[row], " > 21")[[1]][1]
}
channel_fin = channel[,c(22,21)]
channel_fin = ddply(channel_fin,~path,summarise, conversion= sum(convert))
head(channel_fin)
```

```
##                            path conversion
## 1                1 > 1 > 1 > 20          1
## 2               1 > 1 > 12 > 12          1
## 3     1 > 1 > 14 > 13 > 12 > 20          1
## 4       1 > 1 > 3 > 13 > 3 > 20          1
## 5           1 > 1 > 3 > 17 > 17          1
## 6 1 > 1 > 6 > 1 > 12 > 20 > 12          1
```

```
Data = channel_fin
head(Data)
```

```
##                            path conversion
## 1                1 > 1 > 1 > 20          1
## 2               1 > 1 > 12 > 12          1
## 3     1 > 1 > 14 > 13 > 12 > 20          1
## 4       1 > 1 > 3 > 13 > 3 > 20          1
## 5           1 > 1 > 3 > 17 > 17          1
## 6 1 > 1 > 6 > 1 > 12 > 20 > 12          1
```

heuristic model

```r
H <- heuristic_models(Data, 'path', 'conversion', var_value='conversion')
H
```

```
##    channel_name first_touch_conversions first_touch_value
## 1             1                     130               130
## 2            20                       0                 0
## 3            12                      75                75
## 4            14                      34                34
## 5            13                     320               320
## 6             3                     168               168
## 7            17                      31                31
## 8             6                      50                50
## 9             8                      56                56
## 10           10                     547               547
## 11           11                      66                66
## 12           16                     111               111
## 13            2                     199               199
## 14            4                     231               231
## 15            7                      26                26
## 16            5                      62                62
## 17            9                     250               250
## 18           15                      22                22
## 19           18                       4                 4
## 20           19                      10                10
##    last_touch_conversions last_touch_value linear_touch_conversions
## 1                      18               18                73.773661
## 2                    1701             1701               473.998171
## 3                      23               23                76.127863
## 4                      25               25                56.335744
## 5                      76               76               204.039552
## 6                      21               21               117.609677
## 7                      47               47                76.583847
## 8                      20               20                54.707124
## 9                      17               17                53.677862
## 10                     42               42               211.822393
## 11                     33               33               107.109048
## 12                     95               95               156.049086
## 13                     18               18                94.111668
## 14                     88               88               250.784033
## 15                     15               15                33.435991
## 16                     23               23                74.900402
## 17                     71               71               194.071690
## 18                     47               47                65.159225
## 19                      2                2                 5.026587
## 20                     10               10                12.676375
```

```
##    linear_touch_value
## 1           73.773661
## 2          473.998171
## 3           76.127863
## 4           56.335744
## 5          204.039552
## 6          117.609677
## 7           76.583847
## 8           54.707124
## 9           53.677862
## 10         211.822393
## 11         107.109048
## 12         156.049086
## 13          94.111668
## 14         250.784033
## 15          33.435991
## 16          74.900402
## 17         194.071690
## 18          65.159225
## 19           5.026587
## 20          12.676375
```

- First Touch Conversion: credit is given to the first touch point.
- Last Touch Conversion: credit is given to the last touch point.
- Linear Touch Conversion: All channels/touch points are given equal credit in the conversion.

Markov model

```
M <- markov_model(Data, 'path', 'conversion', var_value='conversion', order = 1)
```

```
##
## Number of simulations: 100000 - Convergence reached: 2.05% < 5.00%
##
## Percentage of simulated paths that successfully end before maximum number of steps (17) is rea
```

```
M
```

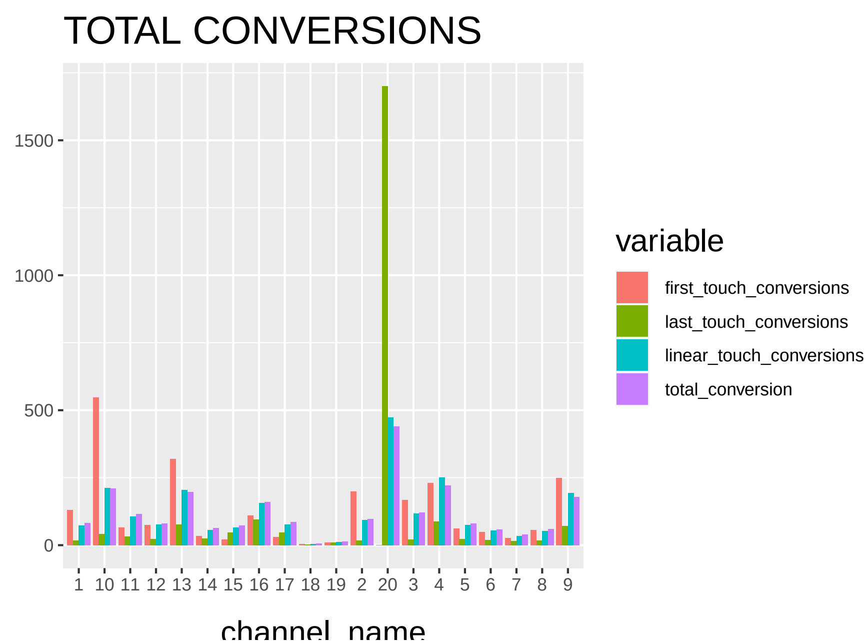
```
##    channel_name total_conversion total_conversion_value
## 1             1        82.805970              82.805970
## 2            20       439.582090             439.582090
## 3            12        81.253731              81.253731
## 4            14        64.238806              64.238806
## 5            13       197.791045             197.791045
## 6             3       122.328358             122.328358
## 7            17        86.985075              86.985075
## 8             6        58.985075              58.985075
```

```
## 9              8        60.656716              60.656716
## 10            10       209.850746             209.850746
## 11            11       115.402985             115.402985
## 12            16       159.820896             159.820896
## 13             2        97.074627              97.074627
## 14             4       222.149254             222.149254
## 15             7        40.597015              40.597015
## 16             5        80.537313              80.537313
## 17             9       178.865672             178.865672
## 18            15        72.358209              72.358209
## 19            18         6.567164               6.567164
## 20            19        14.149254              14.149254
```

combine the two models

```
# Merges the two data frames on the "channel_name" column.
R <- merge(H, M, by='channel_name')

# Select only relevant columns
R1 <- R[, (colnames(R) %in% c('channel_name', 'first_touch_conversions', 'last_touch_co

# Transforms the dataset into a data frame that ggplot2 can use to plot the outcomes
R1 <- melt(R1, id='channel_name')
```

```
# Plot the total conversions
ggplot(R1, aes(channel_name, value, fill = variable)) +
  geom_bar(stat='identity', position='dodge') +
  ggtitle('TOTAL CONVERSIONS') +
  theme(axis.title.x = element_text(vjust = -2)) +
  theme(axis.title.y = element_text(vjust = +2)) +
  theme(title = element_text(size = 16)) +
  theme(plot.title=element_text(size = 20)) +
  ylab("")
```



and then check the final results.

15.8.2.2 Example 2

Example code by Sergey Bryl'

```
library(dplyr)
library(reshape2)
library(ggplot2)
library(ggthemes)
library(ggrepel)
library(RColorBrewer)
library(ChannelAttribution)
library(markovchain)

##### simple example #####
# creating a data sample
df1 <- data.frame(path = c('c1 > c2 > c3', 'c1', 'c2 > c3'), conv = c(1, 0, 0), conv_null = c(0,

# calculating the model
mod1 <- markov_model(df1,
                     var_path = 'path',
                     var_conv = 'conv',
                     var_null = 'conv_null',
                     out_more = TRUE)
```

```
# extracting the results of attribution
df_res1 <- mod1$result

# extracting a transition matrix
df_trans1 <- mod1$transition_matrix
df_trans1 <- dcast(df_trans1, channel_from ~ channel_to, value.var = 'transition_probal

### plotting the Markov graph ###
df_trans <- mod1$transition_matrix

# adding dummies in order to plot the graph
df_dummy <- data.frame(channel_from = c('(start)', '(conversion)', '(null)'),
                       channel_to = c('(start)', '(conversion)', '(null)'),
                       transition_probability = c(0, 1, 1))
df_trans <- rbind(df_trans, df_dummy)

# ordering channels
df_trans$channel_from <- factor(df_trans$channel_from,levels = c('(start)','(conversio
df_trans$channel_to <- factor(df_trans$channel_to,levels = c('(start)', '(conversion)'
df_trans <- dcast(df_trans, channel_from ~ channel_to, value.var ='transition_probabil

# creating the markovchain object
trans_matrix <- matrix(data = as.matrix(df_trans[, -1]),nrow = nrow(df_trans[, -1]), n
trans_matrix[is.na(trans_matrix)] <- 0
# trans_matrix1 <- new("markovchain", transitionMatrix = trans_matrix)
#
# # plotting the graph
# plot(trans_matrix1, edge.arrow.size = 0.35)
```

```
# simulating the "real" data
set.seed(354)
df2 <- data.frame(client_id = sample(c(1:1000), 5000, replace = TRUE),
                  date = sample(c(1:32), 5000, replace = TRUE),
                  channel = sample(c(0:9), 5000, replace = TRUE,
                                   prob = c(0.1, 0.15, 0.05, 0.07, 0.11, 0.07, 0.13, 0
df2$date <- as.Date(df2$date, origin = "2015-01-01")
df2$channel <- paste0('channel_', df2$channel)

# aggregating channels to the paths for each customer
df2 <- df2 %>%
        arrange(client_id, date) %>%
        group_by(client_id) %>%
        summarise(path = paste(channel, collapse = ' > '),
                  # assume that all paths were finished with conversion
                  conv = 1,
```

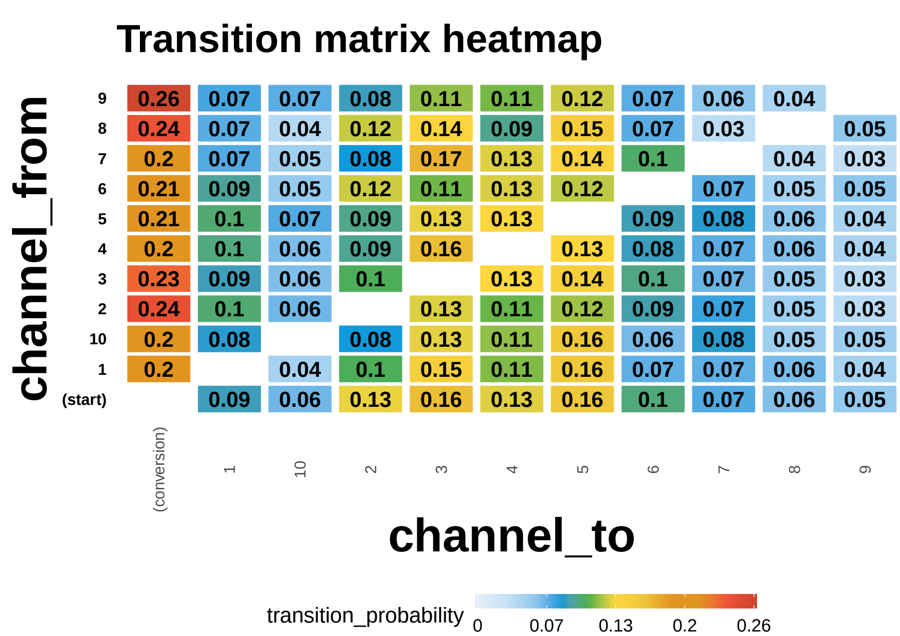
```
                  conv_null = 0) %>%
        ungroup()

# calculating the models (Markov and heuristics)
mod2 <- markov_model(df2,
                     var_path = 'path',
                     var_conv = 'conv',
                     var_null = 'conv_null',
                     out_more = TRUE)
```

```
##
## Number of simulations: 100000 - Convergence reached: 1.40% < 5.00%
##
## Percentage of simulated paths that successfully end before maximum number of steps (13) is rea
```

```
# heuristic_models() function doesn't work for me, therefore I used the manual calculations
# instead of:
#h_mod2 <- heuristic_models(df2, var_path = 'path', var_conv = 'conv')

df_hm <- df2 %>%
        mutate(channel_name_ft = sub('>.*', '', path),
               channel_name_ft = sub(' ', '', channel_name_ft),
               channel_name_lt = sub('.*>', '', path),
               channel_name_lt = sub(' ', '', channel_name_lt))
# first-touch conversions
df_ft <- df_hm %>%
        group_by(channel_name_ft) %>%
        summarise(first_touch_conversions = sum(conv)) %>%
        ungroup()
# last-touch conversions
df_lt <- df_hm %>%
        group_by(channel_name_lt) %>%
        summarise(last_touch_conversions = sum(conv)) %>%
        ungroup()

h_mod2 <- merge(df_ft, df_lt, by.x = 'channel_name_ft', by.y = 'channel_name_lt')

# merging all models
all_models <- merge(h_mod2, mod2$result, by.x = 'channel_name_ft', by.y = 'channel_name')
colnames(all_models)[c(1, 4)] <- c('channel_name', 'attrib_model_conversions')
```

```
library("RColorBrewer")
library("ggthemes")
library("ggrepel")
############## visualizations ##############
# transition matrix heatmap for "real" data
```

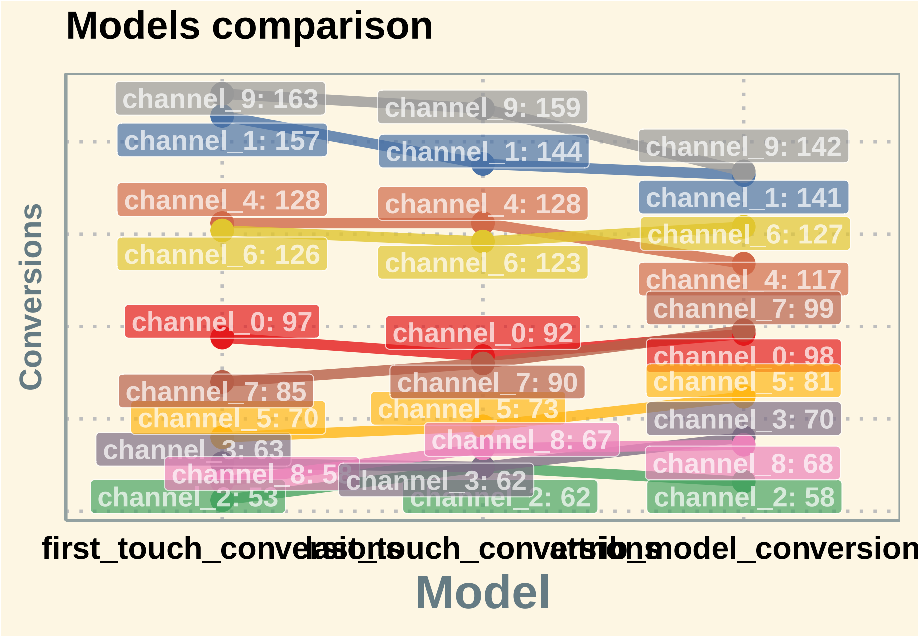
```
df_plot_trans <- mod2$transition_matrix

cols <- c("#e7f0fa", "#c9e2f6", "#95cbee", "#0099dc", "#4ab04a", "#ffd73e", "#eec73a",
          "#e29421", "#e29421", "#f05336", "#ce472e")
t <- max(df_plot_trans$transition_probability)

ggplot(df_plot_trans, aes(y = channel_from, x = channel_to, fill = transition_probabili
        theme_minimal() +
        geom_tile(colour = "white", width = .9, height = .9) +
        scale_fill_gradientn(colours = cols, limits = c(0, t),
                             breaks = seq(0, t, by = t/4),
                             labels = c("0", round(t/4*1, 2), round(t/4*2, 2), round(t/
                             guide = guide_colourbar(ticks = T, nbin = 50, barheight =
        geom_text(aes(label = round(transition_probability, 2)), fontface = "bold", si
        theme(legend.position = 'bottom',
              legend.direction = "horizontal",
              panel.grid.major = element_blank(),
              panel.grid.minor = element_blank(),
              plot.title = element_text(size = 20, face = "bold", vjust = 2, color = 'l
              axis.title.x = element_text(size = 24, face = "bold"),
              axis.title.y = element_text(size = 24, face = "bold"),
              axis.text.y = element_text(size = 8, face = "bold", color = 'black'),
              axis.text.x = element_text(size = 8, angle = 90, hjust = 0.5, vjust = 0.5
        ggtitle("Transition matrix heatmap")
```



```
# models comparison
all_mod_plot <- reshape2::melt(all_models, id.vars = 'channel_name', variable.name = 'conv_type')
all_mod_plot$value <- round(all_mod_plot$value)
# slope chart
pal <- colorRampPalette(brewer.pal(10, "Set1"))
```

```
## Warning in brewer.pal(10, "Set1"): n too large, allowed maximum for palette Set1 is 9
## Returning the palette you asked for with that many colors
```

```
ggplot(all_mod_plot, aes(x = conv_type, y = value, group = channel_name)) +
        theme_solarized(base_size = 18, base_family = "", light = TRUE) +
        scale_color_manual(values = pal(10)) +
        scale_fill_manual(values = pal(10)) +
        geom_line(aes(color = channel_name), size = 2.5, alpha = 0.8) +
        geom_point(aes(color = channel_name), size = 5) +
        geom_label_repel(aes(label = paste0(channel_name, ': ', value), fill = factor(channel_nam
                         alpha = 0.7,
                         fontface = 'bold', color = 'white', size = 5,
                         box.padding = unit(0.25, 'lines'), point.padding = unit(0.5, 'lines'),
                         max.iter = 100) +
        theme(legend.position = 'none',
              legend.title = element_text(size = 16, color = 'black'),
              legend.text = element_text(size = 16, vjust = 2, color = 'black'),
              plot.title = element_text(size = 20, face = "bold", vjust = 2, color = 'black', lin
              axis.title.x = element_text(size = 24, face = "bold"),
              axis.title.y = element_text(size = 16, face = "bold"),
              axis.text.x = element_text(size = 16, face = "bold", color = 'black'),
              axis.text.y = element_blank(),
              axis.ticks.x = element_blank(),
              axis.ticks.y = element_blank(),
              panel.border = element_blank(),
              panel.grid.major = element_line(colour = "grey", linetype = "dotted"),
              panel.grid.minor = element_blank(),
              strip.text = element_text(size = 16, hjust = 0.5, vjust = 0.5, face = "bold", color
              strip.background = element_rect(fill = "#f0b35f")) +
        labs(x = 'Model', y = 'Conversions') +
        ggtitle('Models comparison') +
        guides(colour = guide_legend(override.aes = list(size = 4)))
```



Additional concerns:

```
library(tidyverse)
library(reshape2)
library(ggthemes)
library(ggrepel)
library(RColorBrewer)
library(ChannelAttribution)
library(markovchain)
library(visNetwork)
library(expm)
library(stringr)
library(purrr)
library(purrrlyr)


##### simulating the "real" data #####
set.seed(454)
df_raw <- data.frame(customer_id = paste0('id', sample(c(1:20000), replace = TRUE)), da
        group_by(customer_id) %>%
        mutate(conversion = sample(c(0, 1), n(), prob = c(0.975, 0.025), replace = TRUE
        ungroup() %>%
        dmap_at(c(1, 3), as.character) %>%
        arrange(customer_id, date)
```

```
df_raw <- df_raw %>%
        mutate(channel = ifelse(channel == 'channel_2', NA, channel))
head(df_raw, n = 2)
```

```
## # A tibble: 2 x 4
##   customer_id date       channel   conversion
##   <chr>       <date>     <chr>          <dbl>
## 1 id1         2016-01-02 channel_7          0
## 2 id1         2016-01-09 channel_4          0
```

15.8.2.2.1 1. Customers will be at different stage of purchase journey after each conversion. First-time buyer's journey will look different from n-times buyer's (e.g., he will not start at website)

You can create your own code to split data into customers in different stages.

```
##### splitting paths #####
df_paths <- df_raw %>%
        group_by(customer_id) %>%
        mutate(path_no = ifelse(is.na(lag(cumsum(conversion))), 0, lag(cumsum(conversion))) + 1)
        ungroup()
head(df_paths)
```

```
## # A tibble: 6 x 5
##   customer_id date       channel   conversion path_no
##   <chr>       <date>     <chr>          <dbl>   <dbl>
## 1 id1         2016-01-02 channel_7          0       1
## 2 id1         2016-01-09 channel_4          0       1
## 3 id1         2016-01-18 channel_5          1       1
## 4 id1         2016-01-20 channel_4          1       2
## 5 id100       2016-01-01 channel_0          0       1
## 6 id100       2016-01-01 channel_0          0       1
```

attribution path for first-time buyers:

```
df_paths_1 <- df_paths %>%
        filter(path_no == 1) %>%
        select(-path_no)
```

15.8.2.2.2 2. Handle missing data We might have missing data on the channel or do not want to attribute a path (e.g., Direct Channel). We can either

- Remove NA/Channel
- Use the previous channel in its place.

In the first-order Markov chains, the results are unchanged because duplicated channels don't affect the calculation.

```
##### replace some channels #####
df_path_1_clean <- df_paths_1 %>%
        # removing NAs
        filter(!is.na(channel)) %>%

        # adding order of channels in the path
        group_by(customer_id) %>%
        mutate(ord = c(1:n()),
               is_non_direct = ifelse(channel == 'channel_6', 0, 1),
               is_non_direct_cum = cumsum(is_non_direct)) %>%

        # removing Direct (channel_6) when it is the first in the path
        filter(is_non_direct_cum != 0) %>%

        # replacing Direct (channel_6) with the previous touch point
        mutate(channel = ifelse(channel == 'channel_6', channel[which(channel != 'chann

        ungroup() %>%
        select(-ord, -is_non_direct, -is_non_direct_cum)
```

15.8.2.2.3 3. one vs. multi-channel paths We need to calculate the weighted importance for each channel because the sum of the Removal Effects doesn't equal to 1. In case we have a path with a unique channel, the Removal Effect and importance of this channel for that exact path is 1. However, weighting with other multi-channel paths will decrease the importance of one-channel occurrences. That means that, in case we have a channel that occurs in one-channel paths, usually it will be underestimated if attributed with multi-channel paths.

There is also a pretty straight logic behind splitting – for one-channel paths, we definitely know the channel that brought a conversion and we don't need to distribute that value into other channels.

To account for one-channel path:

1. Split data for paths with one or more unique channels
2. Calculate total conversions for one-channel paths and compute the Markov model for multi-channel paths
3. Summarize results for each channel.

```
##### one- and multi-channel paths #####
df_path_1_clean <- df_path_1_clean %>%
        group_by(customer_id) %>%
        mutate(uniq_channel_tag = ifelse(length(unique(channel)) == 1, TRUE, FALSE)) %
        ungroup()

df_path_1_clean_uniq <- df_path_1_clean %>%
```

```
        filter(uniq_channel_tag == TRUE) %>%
        select(-uniq_channel_tag)

df_path_1_clean_multi <- df_path_1_clean %>%
        filter(uniq_channel_tag == FALSE) %>%
        select(-uniq_channel_tag)

### experiment ###
# attribution model for all paths
df_all_paths <- df_path_1_clean %>%
        group_by(customer_id) %>%
        summarise(path = paste(channel, collapse = ' > '),
                  conversion = sum(conversion)) %>%
        ungroup() %>%
        filter(conversion == 1)

mod_attrib <- markov_model(df_all_paths,
                           var_path = 'path',
                           var_conv = 'conversion',
                           out_more = TRUE)
```

```
##
## Number of simulations: 100000 - Convergence reached: 1.28% < 5.00%
##
## Percentage of simulated paths that successfully end before maximum number of steps (19) is rea
```

```
mod_attrib$removal_effects
```

```
##   channel_name removal_effects
## 1    channel_7       0.2812250
## 2    channel_4       0.4284428
## 3    channel_5       0.6056845
## 4    channel_0       0.5367294
## 5    channel_1       0.3820056
## 6    channel_3       0.2535028
```

```
mod_attrib$result
```

```
##   channel_name total_conversions
## 1    channel_7          192.8653
## 2    channel_4          293.8279
## 3    channel_5          415.3811
## 4    channel_0          368.0913
## 5    channel_1          261.9811
## 6    channel_3          173.8533
```

```
d_all <- data.frame(mod_attrib$result)

# attribution model for splitted multi and unique channel paths
df_multi_paths <- df_path_1_clean_multi %>%
        group_by(customer_id) %>%
        summarise(path = paste(channel, collapse = ' > '),
                  conversion = sum(conversion)) %>%
        ungroup() %>%
        filter(conversion == 1)

mod_attrib_alt <- markov_model(df_multi_paths,
                               var_path = 'path',
                               var_conv = 'conversion',
                               out_more = TRUE)
```

```
##
## Number of simulations: 100000 - Convergence reached: 1.21% < 5.00%
##
## Percentage of simulated paths that successfully end before maximum number of steps
```

```
mod_attrib_alt$removal_effects
```

```
##   channel_name removal_effects
## 1    channel_7       0.3265696
## 2    channel_4       0.4844802
## 3    channel_5       0.6526369
## 4    channel_0       0.5814164
## 5    channel_1       0.4343546
## 6    channel_3       0.2898041
```

```
mod_attrib_alt$result
```

```
##   channel_name total_conversions
## 1    channel_7          150.9460
## 2    channel_4          223.9350
## 3    channel_5          301.6599
## 4    channel_0          268.7406
## 5    channel_1          200.7661
## 6    channel_3          133.9524
```

```
# adding unique paths
df_uniq_paths <- df_path_1_clean_uniq %>%
        filter(conversion == 1) %>%
        group_by(channel) %>%
        summarise(conversions = sum(conversion)) %>%
        ungroup()
```

```r
d_multi <- data.frame(mod_attrib_alt$result)

d_split <- full_join(d_multi, df_uniq_paths, by = c('channel_name' = 'channel')) %>%
        mutate(result = total_conversions + conversions)

sum(d_all$total_conversions)
```

```
## [1] 1706
```

```r
sum(d_split$result)
```

```
## [1] 1706
```

15.8.2.2.4 4. Higher Order Markov Chains Since the transition matrix stays the same in the first order Markov, having duplicates will not affect the result. But starting from the second order order Markov, you will have different results when skipping duplicates. In order to check the effect of skipping duplicates in the first-order Markov chain, we will use my script for "manual" calculation because the package skips duplicates automatically.

```r
##### Higher order of Markov chains and consequent duplicated channels in the path #####

# computing transition matrix - 'manual' way
df_multi_paths_m <- df_multi_paths %>%
        mutate(path = paste0('(start) > ', path, ' > (conversion)'))
m <- max(str_count(df_multi_paths_m$path, '>')) + 1 # maximum path length

df_multi_paths_cols <- reshape2::colsplit(string = df_multi_paths_m$path, pattern = ' > ', names
colnames(df_multi_paths_cols) <- paste0('ord_', c(1:m))
df_multi_paths_cols[df_multi_paths_cols == ''] <- NA

df_res <- vector('list', ncol(df_multi_paths_cols) - 1)

for (i in c(1:(ncol(df_multi_paths_cols) - 1))) {

        df_cache <- df_multi_paths_cols %>%
                select(num_range("ord_", c(i, i+1))) %>%
                na.omit() %>%
                group_by_(.dots = c(paste0("ord_", c(i, i+1)))) %>%
                summarise(n = n()) %>%
                ungroup()

        colnames(df_cache)[c(1, 2)] <- c('channel_from', 'channel_to')
        df_res[[i]] <- df_cache
}
```

```
## Warning: `group_by_()` was deprecated in dplyr 0.7.0.
```

```
## Please use `group_by()` instead.
## See vignette('programming') for more help
```

```
df_res <- do.call('rbind', df_res)

df_res_tot <- df_res %>%
        group_by(channel_from, channel_to) %>%
        summarise(n = sum(n)) %>%
        ungroup() %>%
        group_by(channel_from) %>%
        mutate(tot_n = sum(n),
               perc = n / tot_n) %>%
        ungroup()

df_dummy <- data.frame(channel_from = c('(start)', '(conversion)', '(null)'),
                       channel_to = c('(start)', '(conversion)', '(null)'),
                       n = c(0, 0, 0),
                       tot_n = c(0, 0, 0),
                       perc = c(0, 1, 1))

df_res_tot <- rbind(df_res_tot, df_dummy)

# comparing transition matrices
trans_matrix_prob_m <- dcast(df_res_tot, channel_from ~ channel_to, value.var = 'perc'
trans_matrix_prob <- data.frame(mod_attrib_alt$transition_matrix)
trans_matrix_prob <- dcast(trans_matrix_prob, channel_from ~ channel_to, value.var = '

# computing attribution - 'manual' way
channels_list <- df_path_1_clean_multi %>%
        filter(conversion == 1) %>%
        distinct(channel)
channels_list <- c(channels_list$channel)

df_res_ini <- df_res_tot %>% select(channel_from, channel_to)
df_attrib <- vector('list', length(channels_list))

for (i in c(1:length(channels_list))) {

        channel <- channels_list[i]

        df_res1 <- df_res %>%
                mutate(channel_from = ifelse(channel_from == channel, NA, channel_from)
                       channel_to = ifelse(channel_to == channel, '(null)', channel_to)
                na.omit()

        df_res_tot1 <- df_res1 %>%
```

```
                group_by(channel_from, channel_to) %>%
                summarise(n = sum(n)) %>%
                ungroup() %>%

                group_by(channel_from) %>%
                mutate(tot_n = sum(n),
                       perc = n / tot_n) %>%
                ungroup()

        df_res_tot1 <- rbind(df_res_tot1, df_dummy) # adding (start), (conversion) and (null) sta

        df_res_tot1 <- left_join(df_res_ini, df_res_tot1, by = c('channel_from', 'channel_to'))
        df_res_tot1[is.na(df_res_tot1)] <- 0

        df_trans1 <- dcast(df_res_tot1, channel_from ~ channel_to, value.var = 'perc', fun.aggreg

        trans_matrix_1 <- df_trans1
        rownames(trans_matrix_1) <- trans_matrix_1$channel_from
        trans_matrix_1 <- as.matrix(trans_matrix_1[, -1])

        inist_n1 <- dcast(df_res_tot1, channel_from ~ channel_to, value.var = 'n', fun.aggregate
        rownames(inist_n1) <- inist_n1$channel_from
        inist_n1 <- as.matrix(inist_n1[, -1])
        inist_n1[is.na(inist_n1)] <- 0
        inist_n1 <- inist_n1['(start)', ]

        res_num1 <- inist_n1 %*% (trans_matrix_1 %^% 100000)

        df_cache <- data.frame(channel_name = channel,
                               conversions = as.numeric(res_num1[1, 1]))

        df_attrib[[i]] <- df_cache
}

df_attrib <- do.call('rbind', df_attrib)

# computing removal effect and results
tot_conv <- sum(df_multi_paths_m$conversion)

df_attrib <- df_attrib %>%
        mutate(tot_conversions = sum(df_multi_paths_m$conversion),
               impact = (tot_conversions - conversions) / tot_conversions,
               tot_impact = sum(impact),
               weighted_impact = impact / tot_impact,
               attrib_model_conversions = round(tot_conversions * weighted_impact)
```

```
        ) %>%
        select(channel_name, attrib_model_conversions)
```

Since with different transition matrices, the removal effects and attribution results stay the same, in practice we skip duplicates.

15.8.2.2.5 5. Non-conversion paths We incorporate null paths in this analysis.

```
##### Generic Probabilistic Model #####
df_all_paths_compl <- df_path_1_clean %>%
        group_by(customer_id) %>%
        summarise(path = paste(channel, collapse = ' > '),
                  conversion = sum(conversion)) %>%
        ungroup() %>%
        mutate(null_conversion = ifelse(conversion == 1, 0, 1))

mod_attrib_complete <- markov_model(
        df_all_paths_compl,
        var_path = 'path',
        var_conv = 'conversion',
        var_null = 'null_conversion',
        out_more = TRUE
)
```

```
##
## Number of simulations: 100000 - Convergence reached: 4.05% < 5.00%
##
## Percentage of simulated paths that successfully end before maximum number of steps
```

```
trans_matrix_prob <- mod_attrib_complete$transition_matrix %>%
        dmap_at(c(1, 2), as.character)

##### viz #####
edges <-
        data.frame(
                from = trans_matrix_prob$channel_from,
                to = trans_matrix_prob$channel_to,
                label = round(trans_matrix_prob$transition_probability, 2),
                font.size = trans_matrix_prob$transition_probability * 100,
                width = trans_matrix_prob$transition_probability * 15,
                shadow = TRUE,
                arrows = "to",
                color = list(color = "#95cbee", highlight = "red")
        )

nodes <- data_frame(id = c( c(trans_matrix_prob$channel_from), c(trans_matrix_prob$chan
```

```
        distinct(id) %>%
        arrange(id) %>%
        mutate(
                label = id,
                color = ifelse(
                        label %in% c('(start)', '(conversion)'),
                        '#4ab04a',
                        ifelse(label == '(null)', '#ce472e', '#ffd73e')
                ),
                shadow = TRUE,
                shape = "box"
        )
```

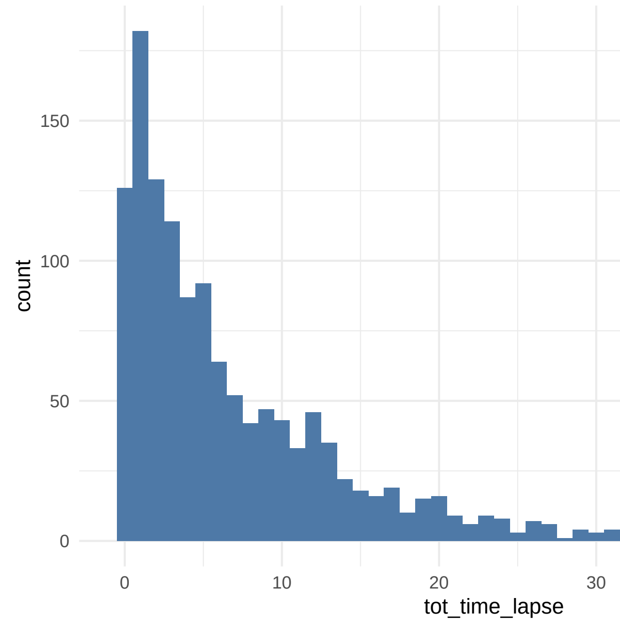
```
## Warning: `data_frame()` was deprecated in tibble 1.1.0.
## Please use `tibble()` instead.
```

```
visNetwork(nodes,
           edges,
           height = "2000px",
           width = "100%",
           main = "Generic Probabilistic model's Transition Matrix") %>%
        visIgraphLayout(randomSeed = 123) %>%
        visNodes(size = 5) %>%
        visOptions(highlightNearest = TRUE)
```


Generic Probabilistic model's Transition Matrix
(null)
(conversion)
(start)
5
1
6
4
3
2
0.18
0.18
0.19
0.21
0.23
0.25
0.27
0.15
0.23
0.28
0.17
0.14

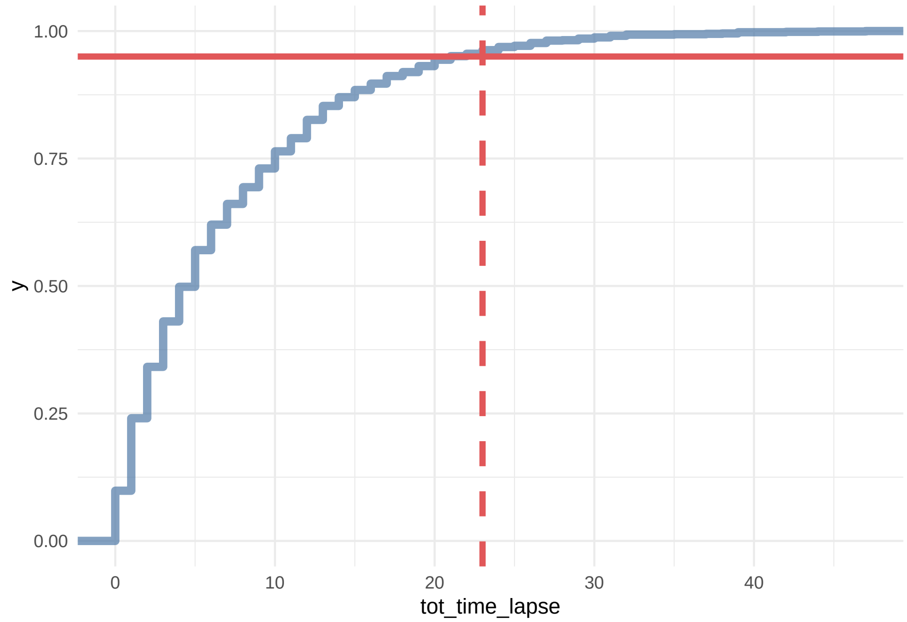
```
##### modeling states and conversions #####
# transition matrix preprocessing
trans_matrix_complete <- mod_attrib_complete$transition_matrix
trans_matrix_complete <- rbind(trans_matrix_complete, df_dummy %>%
                                       mutate(transition_probability = perc) %>%
                                       select(channel_from, channel_to, transition_probability))
trans_matrix_complete$channel_to <- factor(trans_matrix_complete$channel_to, levels = c(levels(tr
trans_matrix_complete <- dcast(trans_matrix_complete, channel_from ~ channel_to, value.var = 'tra
trans_matrix_complete[is.na(trans_matrix_complete)] <- 0
rownames(trans_matrix_complete) <- trans_matrix_complete$channel_from
trans_matrix_complete <- as.matrix(trans_matrix_complete[, -1])


# creating empty matrix for modeling
model_mtrx <- matrix(data = 0,
                     nrow = nrow(trans_matrix_complete), ncol = 1,
                     dimnames = list(c(rownames(trans_matrix_complete)), '(start)'))
# adding modeling number of visits
model_mtrx['channel_5', ] <- 1000

c(model_mtrx) %*% (trans_matrix_complete %^% 5) # after 5 steps
c(model_mtrx) %*% (trans_matrix_complete %^% 100000) # after 100000 steps
```

```
##### Customer journey duration #####
# computing time lapses from the first contact to conversion/last contact
df_multi_paths_tl <- df_path_1_clean_multi %>%
        group_by(customer_id) %>%
        summarise(path = paste(channel, collapse = ' > '),
                  first_touch_date = min(date),
                  last_touch_date = max(date),
                  tot_time_lapse = round(as.numeric(last_touch_date - first_touch_date)),
                  conversion = sum(conversion)) %>%
        ungroup()

# distribution plot
ggplot(df_multi_paths_tl %>% filter(conversion == 1), aes(x = tot_time_lapse)) +
        theme_minimal() +
        geom_histogram(fill = '#4e79a7', binwidth = 1)
```

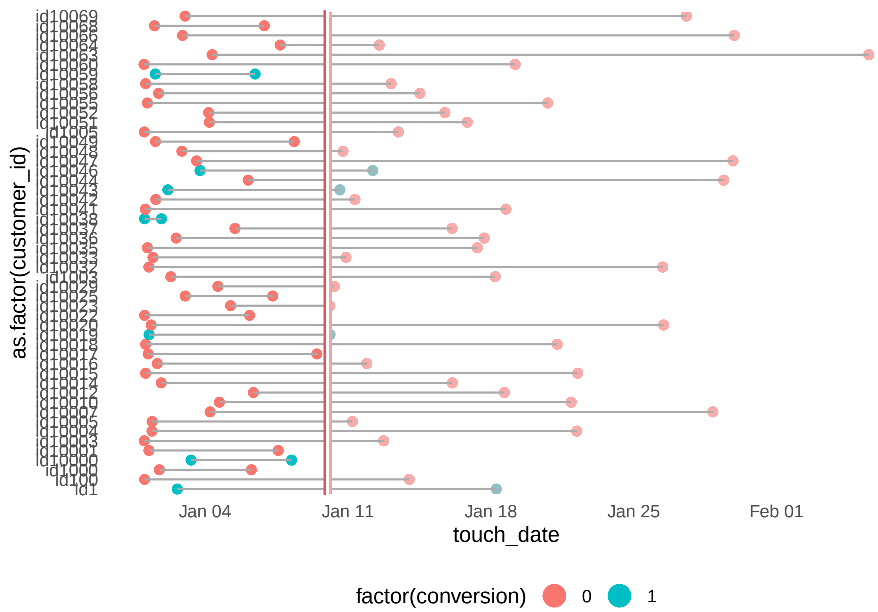


15.8.2.2.6 6. Customer Journey Duration

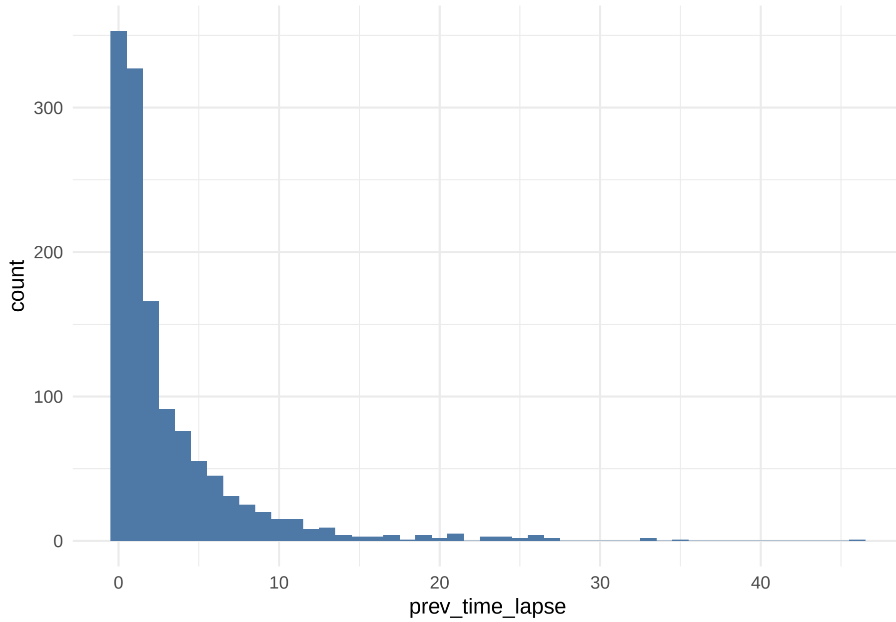
```
# cumulative distribution plot
ggplot(df_multi_paths_tl %>% filter(conversion == 1), aes(x = tot_time_lapse)) +
        theme_minimal() +
        stat_ecdf(geom = 'step', color = '#4e79a7', size = 2, alpha = 0.7) +
        geom_hline(yintercept = 0.95, color = '#e15759', size = 1.5) +
        geom_vline(xintercept = 23, color = '#e15759', size = 1.5, linetype = 2)
```



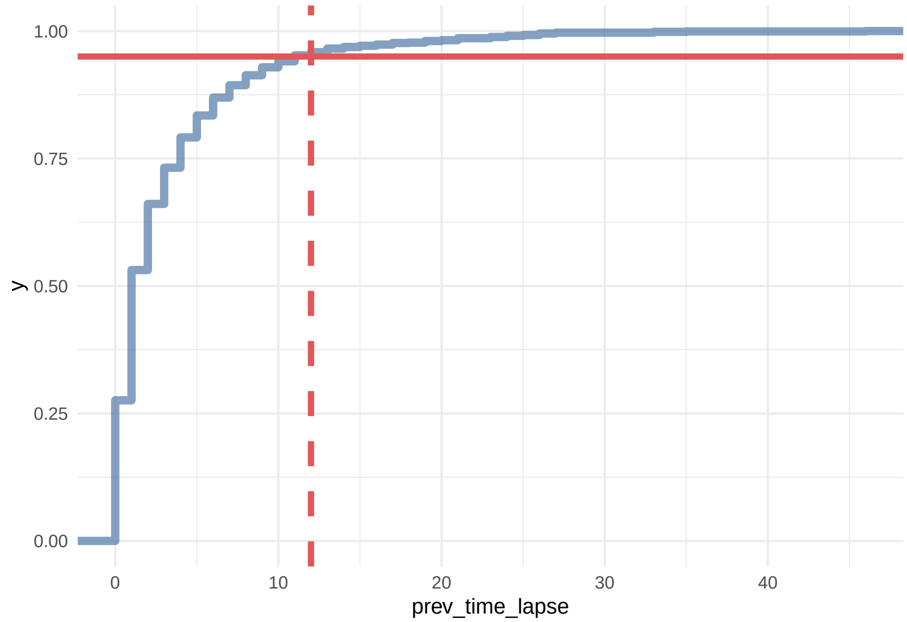

```
### for generic probabilistic model ###
df_multi_paths_tl_1 <- reshape2::melt(df_multi_paths_tl[c(1:50), ] %>% select(customer_id, first_
                    id.vars = c('customer_id', 'conversion'),
                    value.name = 'touch_date') %>%
        arrange(customer_id)
rep_date <- as.Date('2016-01-10', format = '%Y-%m-%d')

ggplot(df_multi_paths_tl_1, aes(x = as.factor(customer_id), y = touch_date, color = factor(conver
        theme_minimal() +
        coord_flip() +
        geom_point(size = 2) +
        geom_line(size = 0.5, color = 'darkgrey') +
        geom_hline(yintercept = as.numeric(rep_date), color = '#e15759', size = 2) +
        geom_rect(xmin = -Inf, xmax = Inf, ymin = as.numeric(rep_date), ymax = Inf, alpha = 0.01,
        theme(legend.position = 'bottom',
              panel.border = element_blank(),
              panel.grid.major = element_blank(),
              panel.grid.minor = element_blank(),
              axis.ticks.x = element_blank(),
              axis.ticks.y = element_blank()) +
        guides(colour = guide_legend(override.aes = list(size = 5)))
```




```
df_multi_paths_tl_2 <- df_path_1_clean_multi %>%
        group_by(customer_id) %>%
        mutate(prev_touch_date = lag(date)) %>%
        ungroup() %>%
        filter(conversion == 1) %>%
        mutate(prev_time_lapse = round(as.numeric(date - prev_touch_date)))

# distribution
ggplot(df_multi_paths_tl_2, aes(x = prev_time_lapse)) +
        theme_minimal() +
        geom_histogram(fill = '#4e79a7', binwidth = 1)
```




```
# cumulative distribution
ggplot(df_multi_paths_tl_2, aes(x = prev_time_lapse)) +
        theme_minimal() +
        stat_ecdf(geom = 'step', color = '#4e79a7', size = 2, alpha = 0.7) +
        geom_hline(yintercept = 0.95, color = '#e15759', size = 1.5) +
        geom_vline(xintercept = 12, color = '#e15759', size = 1.5, linetype = 2)
```



In conclusion, we say that if a customer made contact with a marketing channel the first time for more than 23 days and/or hasn't made contact with a marketing channel for the last 12 days, then it is a fruitless path.

```
# extracting data for generic model
df_multi_paths_tl_3 <- df_path_1_clean_multi %>%
        group_by(customer_id) %>%
        mutate(prev_time_lapse = round(as.numeric(date - lag(date)))) %>%
        summarise(path = paste(channel, collapse = ' > '),
                  tot_time_lapse = round(as.numeric(max(date) - min(date))),
                  prev_touch_tl = prev_time_lapse[which(max(date) == date)],
                  conversion = sum(conversion)) %>%
        ungroup() %>%
        mutate(is_fruitless = ifelse(conversion == 0 & tot_time_lapse > 20 & prev_touc
        filter(conversion == 1 | is_fruitless == TRUE)
```

15.8.2.2.7 7. Channel Comparisons We can use `markov_model` with `var_value` to compare the gross margin among channels.

15.8.2.3 Example 3

This example is from Bounteus

```
# Install these libraries (only do this once)
# install.packages("ChannelAttribution")
# install.packages("reshape")
```

```r
# install.packages("ggplot2")

# Load these libraries (every time you start RStudio)
library(ChannelAttribution)
library(reshape)
library(ggplot2)

# This loads the demo data. You can load your own data by importing a dataset or reading in a fi
data(PathData)
```

- Path Variable – The steps a user takes across sessions to comprise the sequences.
- Conversion Variable – How many times a user converted.
- Value Variable – The monetary value of each marketing channel.
- Null Variable – How many times a user exited.

Build the simple heuristic models (First Click / first_touch, Last Click / last_touch, and Linear Attribution / linear_touch):

```r
H <- heuristic_models(Data, 'path', 'total_conversions', var_value='total_conversion_value')
```

Markov model

```r
M <- markov_model(Data, 'path', 'total_conversions', var_value='total_conversion_value', order =
```

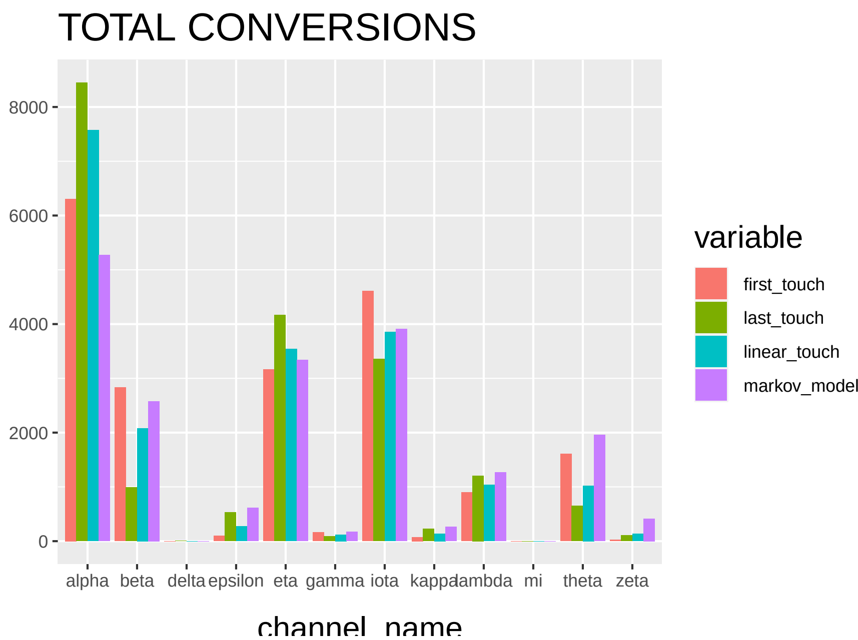
```
##
## Number of simulations: 100000 - Convergence reached: 1.46% < 5.00%
##
## Percentage of simulated paths that successfully end before maximum number of steps (46) is rea
```

```r
# Merges the two data frames on the "channel_name" column.
R <- merge(H, M, by='channel_name')

# Selects only relevant columns
R1 <- R[, (colnames(R)%in%c('channel_name', 'first_touch_conversions', 'last_touch_conversions',

# Renames the columns
colnames(R1) <- c('channel_name', 'first_touch', 'last_touch', 'linear_touch', 'markov_model')

# Transforms the dataset into a data frame that ggplot2 can use to graph the outcomes
R1 <- melt(R1, id='channel_name')
```

Plot the total conversions

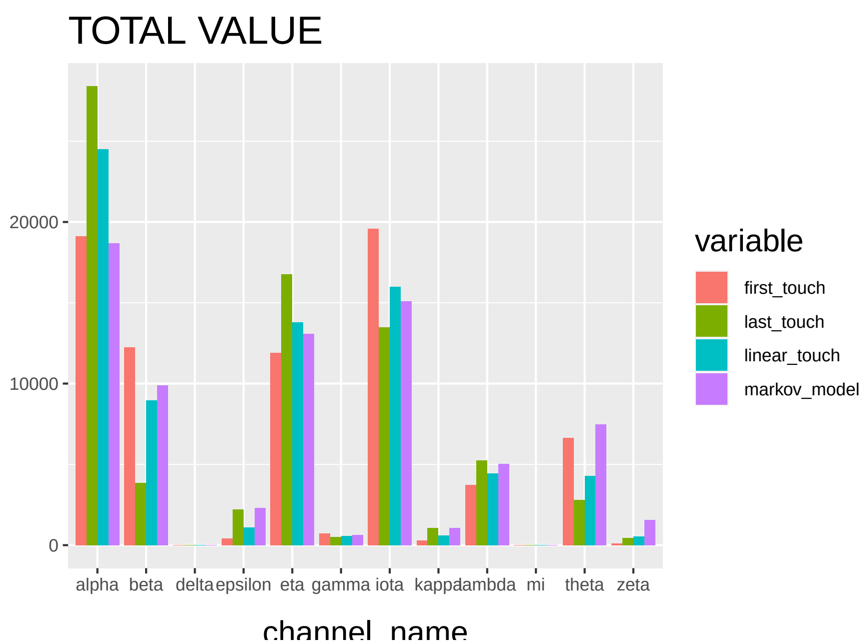
```r
ggplot(R1, aes(channel_name, value, fill = variable)) +
  geom_bar(stat='identity', position='dodge') +
  ggtitle('TOTAL CONVERSIONS') +
  theme(axis.title.x = element_text(vjust = -2)) +
  theme(axis.title.y = element_text(vjust = +2)) +
```

```
  theme(title = element_text(size = 16)) +
  theme(plot.title=element_text(size = 20)) +
  ylab("")
```



The "Total Conversions" bar chart shows you how many conversions were attributed to each channel (i.e. alpha, beta, etc.) for each method (i.e. first_touch, last_touch, etc.).

```
R2 <- R[, (colnames(R)%in%c('channel_name', 'first_touch_value', 'last_touch_value', '

colnames(R2) <- c('channel_name', 'first_touch', 'last_touch', 'linear_touch', 'markov

R2 <- melt(R2, id='channel_name')

ggplot(R2, aes(channel_name, value, fill = variable)) +
  geom_bar(stat='identity', position='dodge') +
  ggtitle('TOTAL VALUE') +
  theme(axis.title.x = element_text(vjust = -2)) +
  theme(axis.title.y = element_text(vjust = +2)) +
  theme(title = element_text(size = 16)) +
  theme(plot.title=element_text(size = 20)) +
  ylab("")
```



The "Total Conversion Value" bar chart shows you monetary value that can be attributed to each channel from a conversion.

15.9 Sales Funnel

15.9.1 Example 1

This example is based on Sergey Bryl

$$Awareness \rightarrow Interest \rightarrow Desire \rightarrow Action$$

Step in the funnel:

- 0 step (necessary condition) – customer visits a site for the first time
- 1st step (awareness) – visits two site's pages
- 2nd step (interest) – reviews a product page
- 3rd step (desire) – adds a product to the shopping cart
- 4th step (action) – completes purchase

Simulate data

```
library(tidyverse)
library(purrrlyr)
library(reshape2)

##### simulating the "real" data #####
```

```
set.seed(454)
df_raw <-
  data.frame(
    customer_id = paste0('id', sample(c(1:5000), replace = TRUE)),
    date = as.POSIXct(
      rbeta(10000, 0.7, 10) * 10000000,
      origin = '2017-01-01',
      tz = "UTC"
    ),
    channel = paste0('channel_', sample(
      c(0:7),
      10000,
      replace = TRUE,
      prob = c(0.2, 0.12, 0.03, 0.07, 0.15, 0.25, 0.1, 0.08)
    )),
    site_visit = 1
  ) %>%

  mutate(
    two_pages_visit = sample(c(0, 1), 10000, replace = TRUE, prob = c(0.8, 0.2)),
    product_page_visit = ifelse(
      two_pages_visit == 1,
      sample(
        c(0, 1),
        length(two_pages_visit[which(two_pages_visit == 1)]),
        replace = TRUE,
        prob = c(0.75, 0.25)
      ),
      0
    ),
    add_to_cart = ifelse(
      product_page_visit == 1,
      sample(
        c(0, 1),
        length(product_page_visit[which(product_page_visit == 1)]),
        replace = TRUE,
        prob = c(0.1, 0.9)
      ),
      0
    ),
    purchase = ifelse(add_to_cart == 1,
                      sample(
                        c(0, 1),
                        length(add_to_cart[which(add_to_cart == 1)]),
                        replace = TRUE,
```

```
                        prob = c(0.02, 0.98)
                      ),
                      0)
  ) %>%
  dmap_at(c('customer_id', 'channel'), as.character) %>%
  arrange(date) %>%
  mutate(session_id = row_number()) %>%
  arrange(customer_id, session_id)
df_raw <-
  reshape2::melt(
    df_raw,
    id.vars = c('customer_id', 'date', 'channel', 'session_id'),
    value.name = "trigger",
    variable.name = 'event'
  ) %>%
  filter(trigger == 1) %>%
  select(-trigger) %>%
  arrange(customer_id, date)
```

Preprocessing

```
### removing not first events ###
df_customers <- df_raw %>%
  group_by(customer_id, event) %>%
  filter(date == min(date)) %>%
  ungroup()
```

Assumption: all customers are first-time buyers. Hence, every next purchase as an event will be removed with the above code.

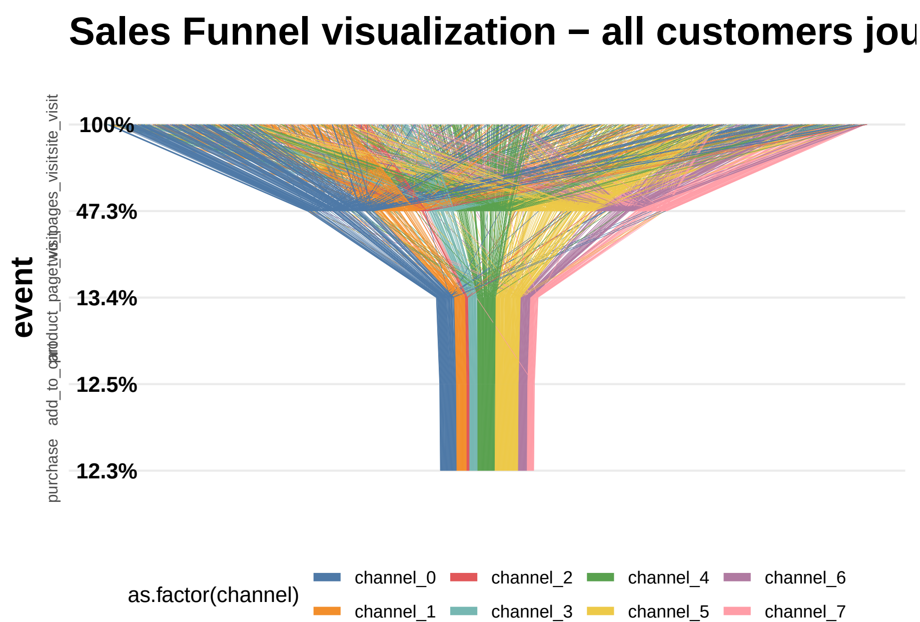
Calculate channel probability

```
### Sales Funnel probabilities ###
sf_probs <- df_customers %>%

    group_by(event) %>%
    summarise(customers_on_step = n()) %>%
    ungroup() %>%

    mutate(
        sf_probs = round(customers_on_step / customers_on_step[event == 'site_visit'], 3),
        sf_probs_step = round(customers_on_step / lag(customers_on_step), 3),
        sf_probs_step = ifelse(is.na(sf_probs_step) == TRUE, 1, sf_probs_step),
        sf_importance = 1 - sf_probs_step,
        sf_importance_weighted = sf_importance / sum(sf_importance)
    )
```

Visualization

```
### Sales Funnel visualization ###
df_customers_plot <- df_customers %>%

    group_by(event) %>%
    arrange(channel) %>%
    mutate(pl = row_number()) %>%
    ungroup() %>%

    mutate(
        pl_new = case_when(
            event == 'two_pages_visit' ~ round((max(pl[event == 'site_visit']) - max(pl
            event == 'product_page_visit' ~ round((max(pl[event == 'site_visit']) - ma
            event == 'add_to_cart' ~ round((max(pl[event == 'site_visit']) - max(pl[eve
            event == 'purchase' ~ round((max(pl[event == 'site_visit']) - max(pl[event
            TRUE ~ 0
        ),
        pl = pl + pl_new
    )

df_customers_plot$event <-
    factor(
        df_customers_plot$event,
        levels = c(
            'purchase',
            'add_to_cart',
            'product_page_visit',
            'two_pages_visit',
            'site_visit'
        )
    )

# color palette
cols <- c(
    '#4e79a7',
    '#f28e2b',
    '#e15759',
    '#76b7b2',
    '#59a14f',
    '#edc948',
    '#b07aa1',
    '#ff9da7',
    '#9c755f',
    '#bab0ac'
)
```

```
ggplot(df_customers_plot, aes(x = event, y = pl)) +
    theme_minimal() +
    scale_colour_manual(values = cols) +
    coord_flip() +
    geom_line(aes(group = customer_id, color = as.factor(channel)), size = 0.05) +
    geom_text(
        data = sf_probs,
        aes(
            x = event,
            y = 1,
            label = paste0(sf_probs * 100, '%')
        ),
        size = 4,
        fontface = 'bold'
    ) +
    guides(color = guide_legend(override.aes = list(size = 2))) +
    theme(
        legend.position = 'bottom',
        legend.direction = "horizontal",
        panel.grid.major.x = element_blank(),
        panel.grid.minor = element_blank(),
        plot.title = element_text(
            size = 20,
            face = "bold",
            vjust = 2,
            color = 'black',
            lineheight = 0.8
        ),
        axis.title.y = element_text(size = 16, face = "bold"),
        axis.title.x = element_blank(),
        axis.text.x = element_blank(),
        axis.text.y = element_text(
            size = 8,
            angle = 90,
            hjust = 0.5,
            vjust = 0.5,
            face = "plain"
        )
    ) +
    ggtitle("Sales Funnel visualization - all customers journeys")
```



Calculate attribution

```
### computing attribution ###
df_attrib <- df_customers %>%
    # removing customers without purchase
    group_by(customer_id) %>%
    filter(any(as.character(event) == 'purchase')) %>%
    ungroup() %>%

    # joining step's importances
    left_join(., sf_probs %>% select(event, sf_importance_weighted), by = 'event') %>%

    group_by(channel) %>%
    summarise(tot_attribution = sum(sf_importance_weighted)) %>%
    ungroup()
```

15.9.2 Example 2

Code from Sergey Bryl

```
library(dplyr)
library(ggplot2)
library(reshape2)

# creating a data samples
# content
```

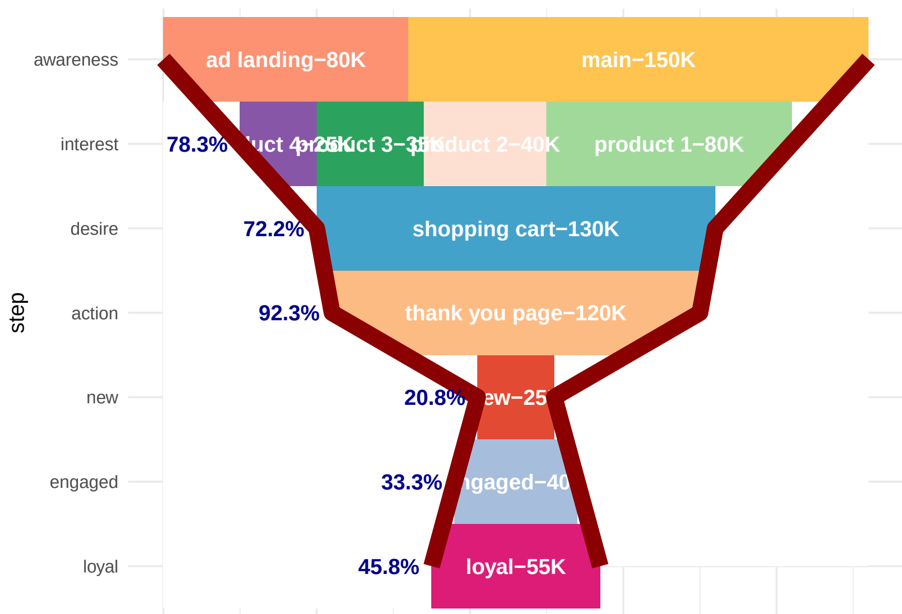
```
df.content <- data.frame(
    content = c(
        'main',
        'ad landing',
        'product 1',
        'product 2',
        'product 3',
        'product 4',
        'shopping cart',
        'thank you page'
    ),
    step = c(
        'awareness',
        'awareness',
        'interest',
        'interest',
        'interest',
        'interest',
        'desire',
        'action'
    ),
    number = c(150000, 80000,
               80000, 40000, 35000, 25000,
               130000,
               120000)
)
# customers
df.customers <- data.frame(
    content = c('new', 'engaged', 'loyal'),
    step = c('new', 'engaged', 'loyal'),
    number = c(25000, 40000, 55000)
)
# combining two data sets
df.all <- rbind(df.content, df.customers)

# calculating dummies, max and min values of X for plotting
df.all <- df.all %>%
    group_by(step) %>%
    mutate(totnum = sum(number)) %>%
    ungroup() %>%
    mutate(dum = (max(totnum) - totnum) / 2,
           maxx = totnum + dum,
           minx = dum)

# data frame for plotting funnel lines
```

```
df.lines <- df.all %>%
    distinct(step, maxx, minx)

# data frame with dummies
df.dum <- df.all %>%
    distinct(step, dum) %>%
    mutate(content = 'dummy',
           number = dum) %>%
    select(content, step, number)

# data frame with rates
conv <- df.all$totnum[df.all$step == 'action']

df.rates <- df.all %>%
    distinct(step, totnum) %>%
    mutate(
        prevnum = lag(totnum),
        rate = ifelse(
            step == 'new' | step == 'engaged' | step == 'loyal',
            round(totnum / conv, 3),
            round(totnum / prevnum, 3)
        )
    ) %>%
    select(step, rate)
df.rates <- na.omit(df.rates)

# creting final data frame
df.all <- df.all %>%
    select(content, step, number)

df.all <- rbind(df.all, df.dum)

# defining order of steps
df.all$step <-
    factor(
        df.all$step,
        levels = c(
            'loyal',
            'engaged',
            'new',
            'action',
            'desire',
            'interest',
            'awareness'
        )
```

```
    )
df.all <- df.all %>%
    arrange(desc(step))
list1 <- df.all %>% distinct(content) %>%
    filter(content != 'dummy')
df.all$content <-
    factor(df.all$content, levels = c(as.character(list1$content), 'dummy'))

# calculating position of labels
df.all <- df.all %>%
    arrange(step, desc(content)) %>%
    group_by(step) %>%
    mutate(pos = cumsum(number) - 0.5 * number) %>%
    ungroup()

# creating custom palette with 'white' color for dummies
cols <- c(
    "#fec44f",
    "#fc9272",
    "#a1d99b",
    "#fee0d2",
    "#2ca25f",
    "#8856a7",
    "#43a2ca",
    "#fdbb84",
    "#e34a33",
    "#a6bddb",
    "#dd1c77",
    "#ffffff"
)

# plotting chart
ggplot() +
    theme_minimal() +
    coord_flip() +
    scale_fill_manual(values = cols) +
    geom_bar(
        data = df.all,
        aes(x = step, y = number, fill = content),
        stat = "identity",
        width = 1
    ) +
    geom_text(
        data = df.all[df.all$content != 'dummy',],
        aes(
```

```
            x = step,
            y = pos,
            label = paste0(content, '-', number / 1000, 'K')
        ),
        size = 4,
        color = 'white',
        fontface = "bold"
    ) +
    geom_ribbon(data = df.lines,
                aes(
                    x = step,
                    ymax = max(maxx),
                    ymin = maxx,
                    group = 1
                ),
                fill = 'white') +
    geom_line(
        data = df.lines,
        aes(x = step, y = maxx, group = 1),
        color = 'darkred',
        size = 4
    ) +
    geom_ribbon(data = df.lines,
                aes(
                    x = step,
                    ymax = minx,
                    ymin = min(minx),
                    group = 1
                ),
                fill = 'white') +
    geom_line(
        data = df.lines,
        aes(x = step, y = minx, group = 1),
        color = 'darkred',
        size = 4
    ) +
    geom_text(
        data = df.rates,
        aes(
            x = step,
            y = (df.lines$minx[-1]),
            label = paste0(rate * 100, '%')
        ),
        hjust = 1.2,
        color = 'darkblue',
```

```
        fontface = "bold"
    ) +
    theme(
        legend.position = 'none',
        axis.ticks = element_blank(),
        axis.text.x = element_blank(),
        axis.title.x = element_blank()
    )
```



15.10 RFM

RFM is calculated as:

- A recency score is assigned to each customer based on date of most recent purchase.
- A frequency ranking is assigned based on frequency of purchases
- Monetary score is assigned based on the total revenue generated by the customer in the period under consideration for the analysis

```
library("rfm")
rfm_data_customer
```

```
## # A tibble: 39,999 x 5
##    customer_id revenue most_recent_visit number_of_orders recency_days
##          <dbl>   <dbl> <date>                       <dbl>        <dbl>
```

```
## 1        22086     777 2006-05-14                            9           232
## 2         2290    1555 2006-09-08                           16           115
## 3        26377     336 2006-11-19                            5            43
## 4        24650    1189 2006-10-29                           12            64
## 5        12883    1229 2006-12-09                           12            23
## 6         2119     929 2006-10-21                           11            72
## 7        31283    1569 2006-09-11                           17           112
## 8        33815     778 2006-08-12                           11           142
## 9        15972     641 2006-11-19                            9            43
## 10       27650     970 2006-08-23                           10           131
## # ... with 39,989 more rows
```

```
# a unique customer id
# number of transaction/order
# total revenue from the customer
# number of days since the last visit


rfm_data_orders # to generate data_orders, use rfm_table_order()
```

```
## # A tibble: 4,906 x 3
##    customer_id         order_date revenue
##    <chr>               <date>       <dbl>
##  1 Mr. Brion Stark Sr. 2004-12-20      32
##  2 Ethyl Botsford      2005-05-02      36
##  3 Hosteen Jacobi      2004-03-06     116
##  4 Mr. Edw Frami       2006-03-15      99
##  5 Josef Lemke         2006-08-14      76
##  6 Julisa Halvorson    2005-05-28      56
##  7 Judyth Lueilwitz    2005-03-09     108
##  8 Mr. Mekhi Goyette   2005-09-23     183
##  9 Hansford Moen PhD   2005-09-07      30
## 10 Fount Flatley       2006-04-12      13
## # ... with 4,896 more rows
```

```
# unique customer id
# date of transaction
# and amount
# customer_id: name of the customer id column
# order_date: name of the transaction date column
# revenue: name of the transaction amount column
# analysis_date: date of analysis
# recency_bins: number of rankings for recency score (default is 5)
# frequency_bins: number of rankings for frequency score (default is 5)
# monetary_bins: number of rankings for monetary score (default is 5)
```

```r
analysis_date <- lubridate::as_date('2007-01-01')
rfm_result <-
    rfm_table_customer(
        rfm_data_customer,
        customer_id,
        number_of_orders,
        recency_days,
        revenue,
        analysis_date
    )
rfm_result
```

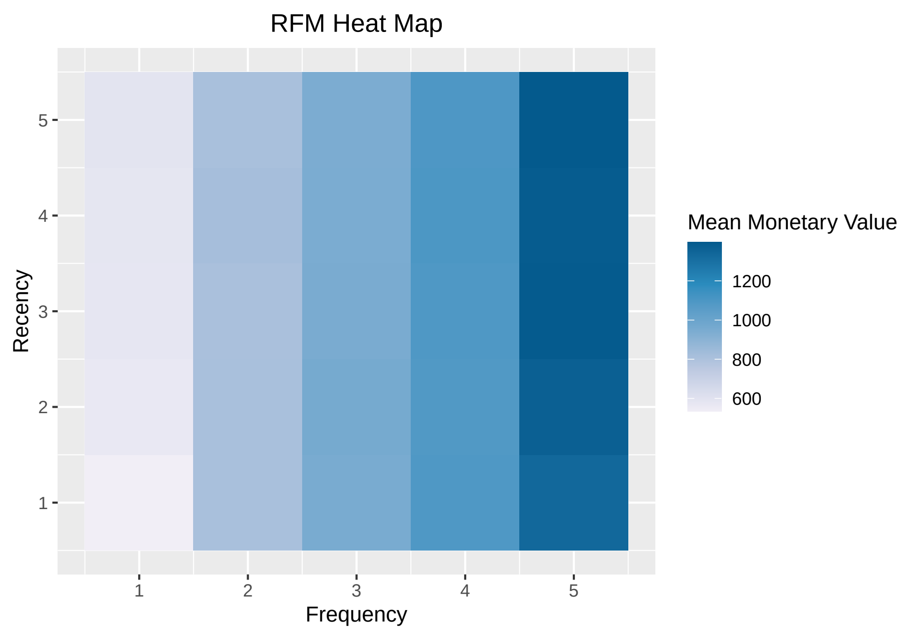
```
## # A tibble: 39,999 x 8
##    customer_id recency_days transaction_count amount recency_score
##          <dbl>        <dbl>             <dbl>  <dbl>         <int>
##  1       22086          232                 9    777             2
##  2        2290          115                16   1555             4
##  3       26377           43                 5    336             5
##  4       24650           64                12   1189             5
##  5       12883           23                12   1229             5
##  6        2119           72                11    929             5
##  7       31283          112                17   1569             4
##  8       33815          142                11    778             3
##  9       15972           43                 9    641             5
## 10       27650          131                10    970             3
## # ... with 39,989 more rows, and 3 more variables: frequency_score <int>,
## #   monetary_score <int>, rfm_score <dbl>
```

```r
# customer_id: unique customer id
# date_most_recent: date of most recent visit
# recency_days: days since the most recent visit
# transaction_count: number of transactions of the customer
# amount: total revenue generated by the customer
# recency_score: recency score of the customer
# frequency_score: frequency score of the customer
# monetary_score: monetary score of the customer
# rfm_score: RFM score of the customer
```

15.10.1 Visualization

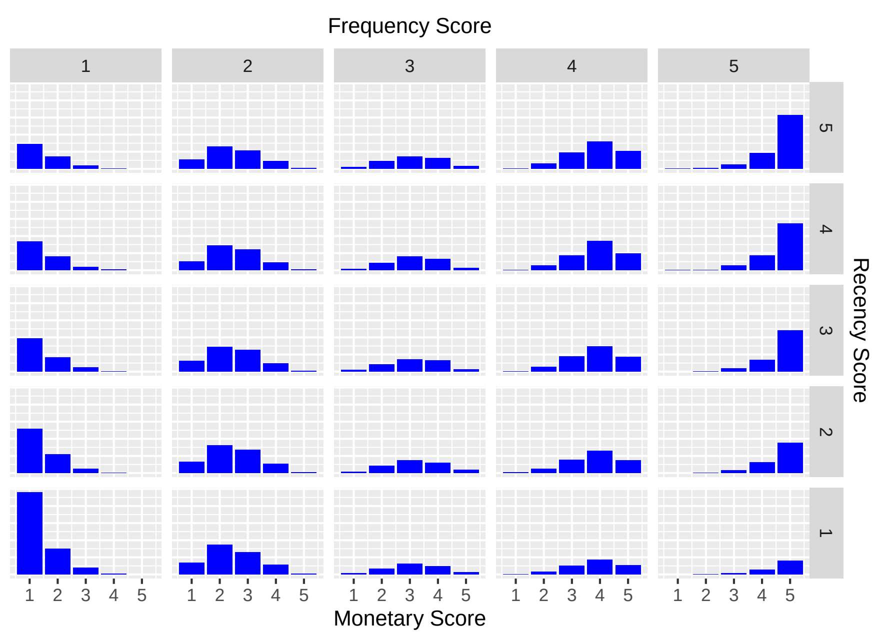
heat map shows the average monetary value for different categories of recency and frequency scores

```
rfm_heatmap(rfm_result)
```



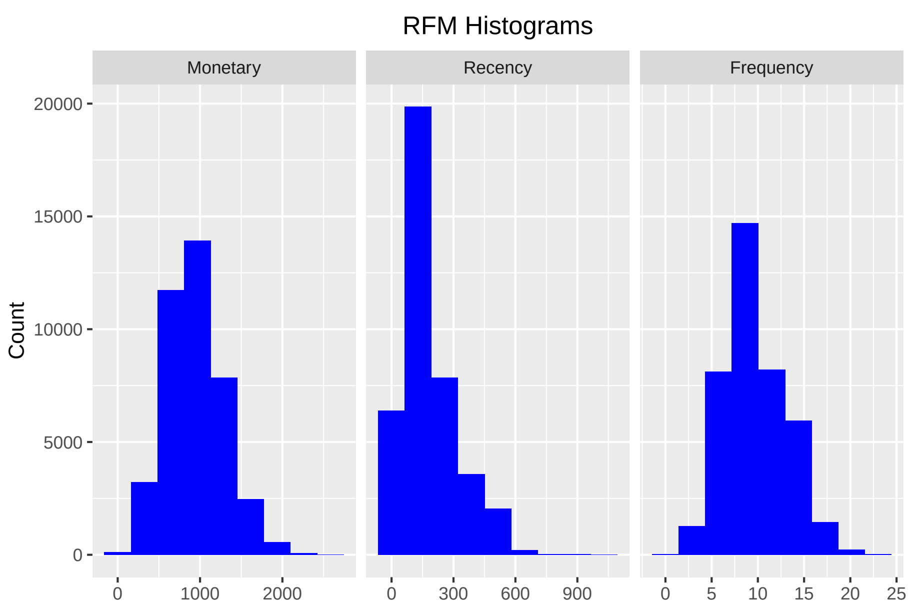
bar chart

```
rfm_bar_chart(rfm_result)
```



histogram

```
rfm_histograms(rfm_result)
```

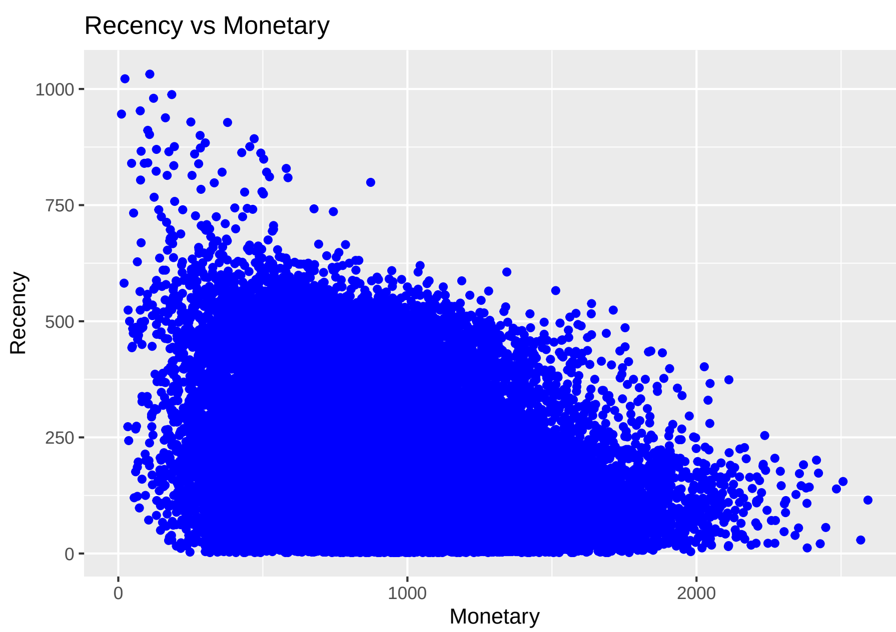


Customers by Orders

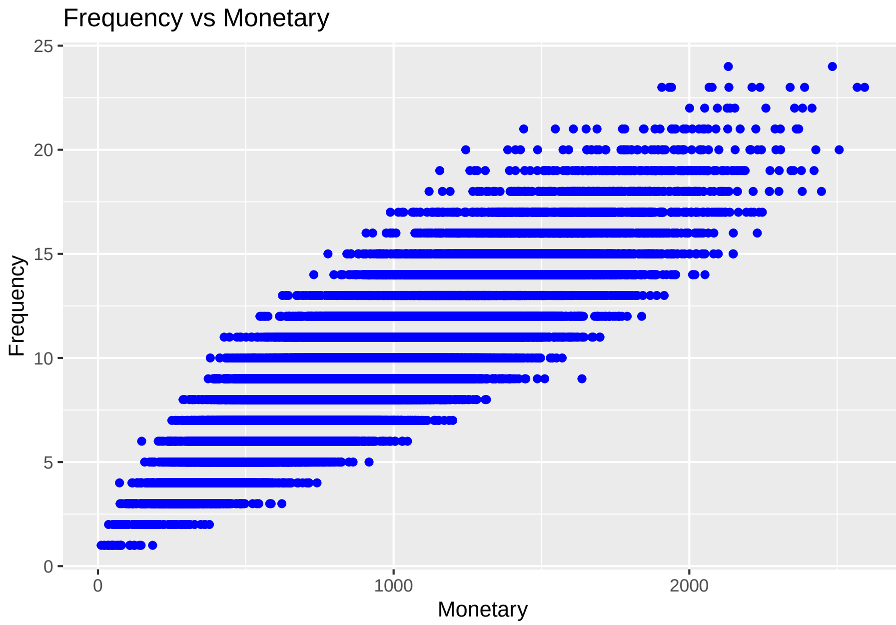
```
rfm_order_dist(rfm_result)
```

Scatter Plots

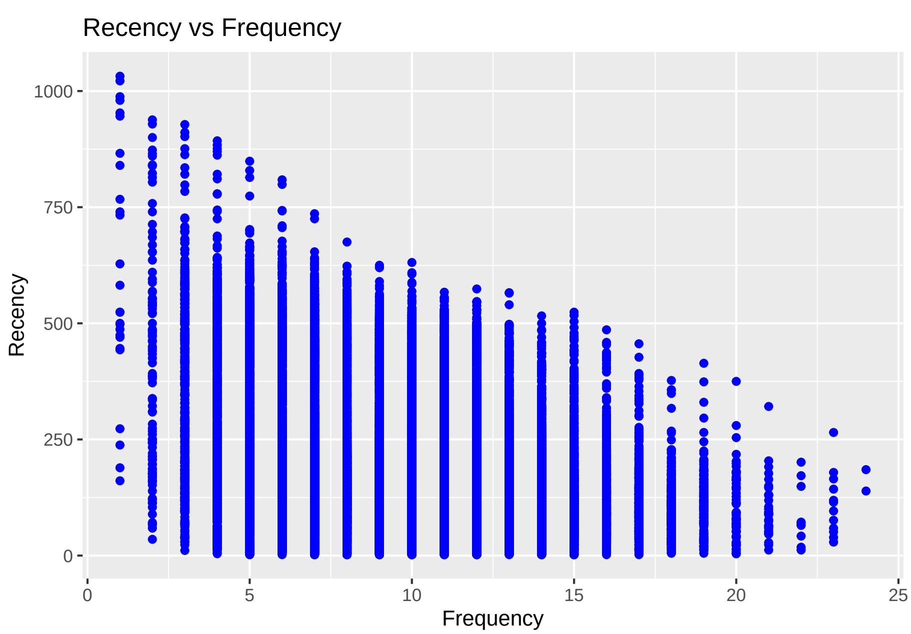
```
rfm_rm_plot(rfm_result)
```



```
rfm_fm_plot(rfm_result)
```




```
rfm_rf_plot(rfm_result)
```



15.10.2 RFMC

(C) clumpiness is defined as the degree of nonconformity to equal spacing (Zhang et al., 2015)

In finance, clumpiness can indicate high growth potential but large risk, Hence, it can be incorporated into firm acquisition decision. Originated from sports phenomenon - hot hand effect - where success leads to more success.

In statistics, clumpiness is the serial dependence or "non-constant propensity, specifically temporary elevations of propensity— i.e. periods during which one event is more likely to occur than the average level." (Zhang et al., 2013)

Properties of clumpiness:

- Min (max) if events are equally spaced (close to one another)
- Continuity
- Convergence

15.11 Customer Segmentation

15.11.1 Example 1

Continue from the RFM

```
segment_names <-
    c(
        "Premium",
        "Loyal Customers",
        "Potential Loyalist",
        "New Customers",
        "Promising",
        "Need Attention",
        "About To Churn",
        "At Risk",
        "High Value Churners/Resurrection",
        "Low Value Churners"
    )

recency_lower <- c(4, 2, 3, 4, 3, 2, 2, 1, 1, 1)
recency_upper <- c(5, 5, 5, 5, 4, 3, 3, 2, 1, 2)
frequency_lower <- c(4, 3, 1, 1, 1, 2, 1, 2, 4, 1)
frequency_upper <- c(5, 5, 3, 1, 1, 3, 2, 5, 5, 2)
monetary_lower <- c(4, 3, 1, 1, 1, 2, 1, 2, 4, 1)
monetary_upper <- c(5, 5, 3, 1, 1, 3, 2, 5, 5, 2)

rfm_segments <-
    rfm_segment(
```

```
        rfm_result,
        segment_names,
        recency_lower,
        recency_upper,
        frequency_lower,
        frequency_upper,
        monetary_lower,
        monetary_upper
    )

head(rfm_segments, n = 5)


rfm_segments %>%
    count(rfm_segments$segment) %>%
    arrange(desc(n)) %>%
    rename(Count = n)

# median recency
rfm_plot_median_recency(rfm_segments)

# median frequency
rfm_plot_median_frequency(rfm_segments)

# Median Monetary Value
rfm_plot_median_monetary(rfm_segments)
```

15.11.2 Example 2

Example by Sergey

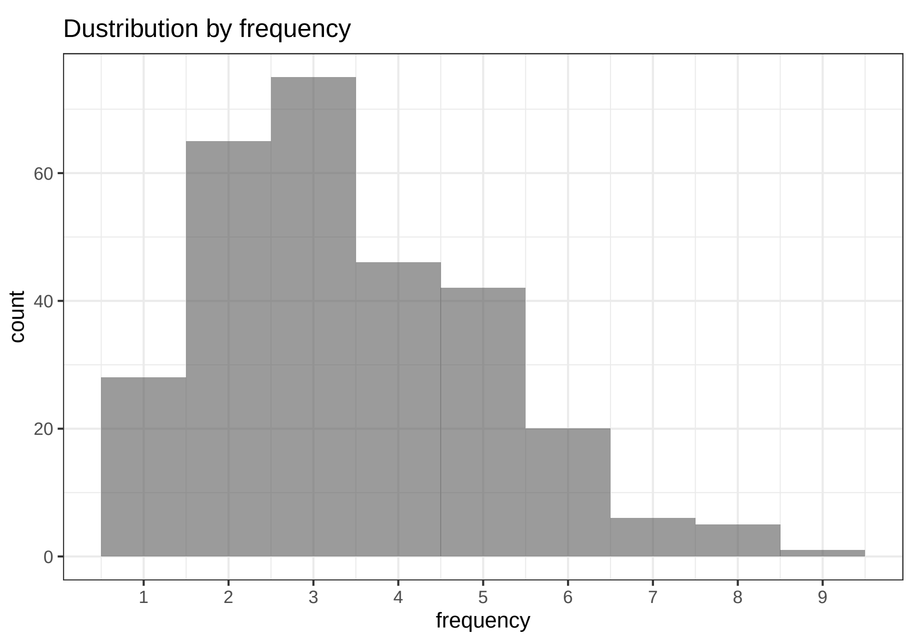
15.11.2.1 LifeCycle Grids

```
# loading libraries
library(dplyr)
library(reshape2)
library(ggplot2)

# creating data sample
set.seed(10)
data <- data.frame(
    orderId = sample(c(1:1000), 5000, replace = TRUE),
    product = sample(
        c('NULL', 'a', 'b', 'c'),
        5000,
```

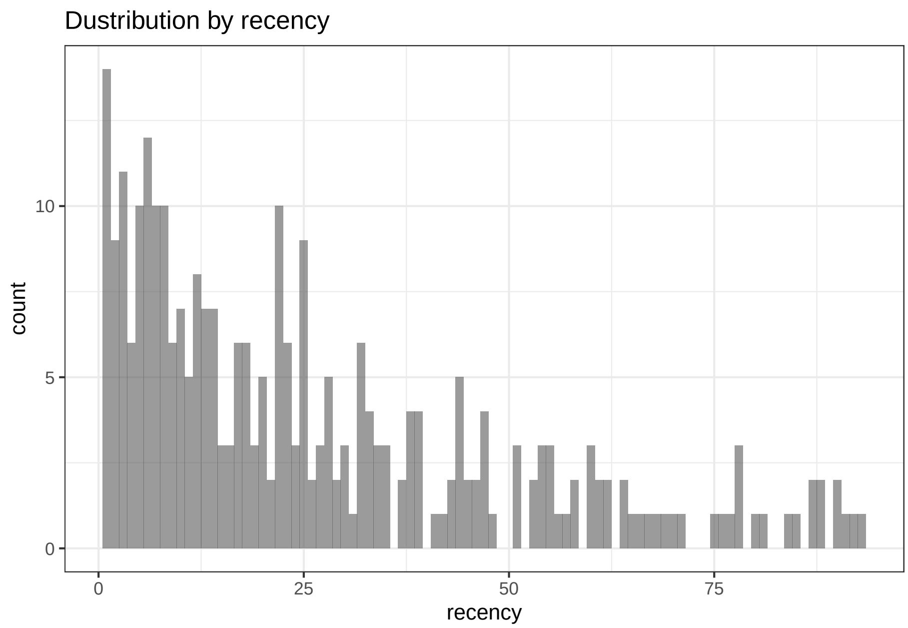
```
        replace = TRUE,
        prob = c(0.15, 0.65, 0.3, 0.15)
    )
)
order <- data.frame(orderId = c(1:1000),
                    clientId = sample(c(1:300), 1000, replace = TRUE))
gender <- data.frame(clientId = c(1:300),
                     gender = sample(
                         c('male', 'female'),
                         300,
                         replace = TRUE,
                         prob = c(0.40, 0.60)
                     ))
date <- data.frame(orderId = c(1:1000),
                   orderdate = sample((1:100), 1000, replace = TRUE))
orders <- merge(data, order, by = 'orderId')
orders <- merge(orders, gender, by = 'clientId')
orders <- merge(orders, date, by = 'orderId')
orders <- orders[orders$product != 'NULL',]
orders$orderdate <- as.Date(orders$orderdate, origin = "2012-01-01")
rm(data, date, order, gender)
```

```
# reporting date
today <- as.Date('2012-04-11', format = '%Y-%m-%d')

# processing data
orders <-
    dcast(
        orders,
        orderId + clientId + gender + orderdate ~ product,
        value.var = 'product',
        fun.aggregate = length
    )

orders <- orders %>%
    group_by(clientId) %>%
    mutate(frequency = n(),
           recency = as.numeric(today - orderdate)) %>%
    filter(orderdate == max(orderdate)) %>%
    filter(orderId == max(orderId)) %>%
    ungroup()

# exploratory analysis
ggplot(orders, aes(x = frequency)) +
    theme_bw() +
```

```
    scale_x_continuous(breaks = c(1:10)) +
    geom_bar(alpha = 0.6, width = 1) +
    ggtitle("Dustribution by frequency")
```



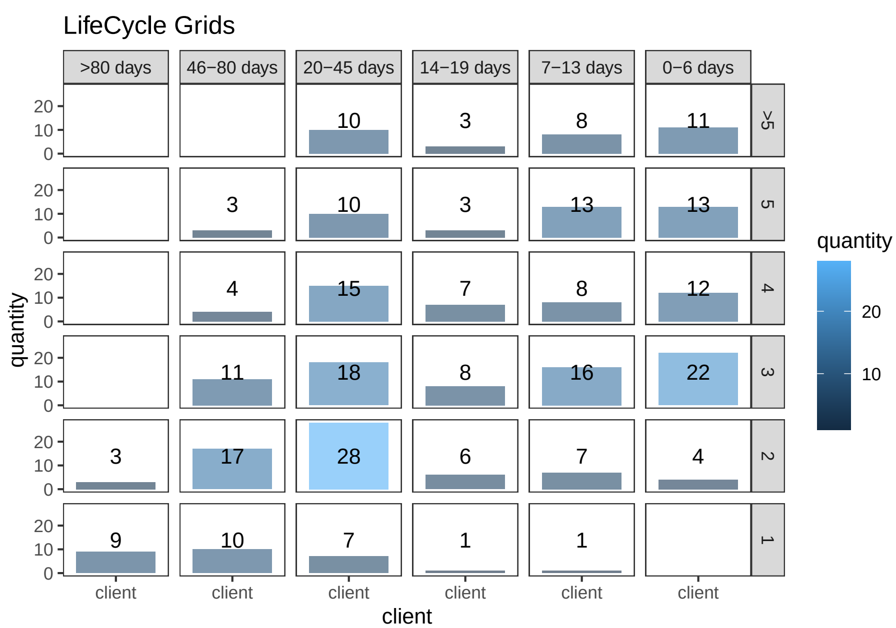

```
ggplot(orders, aes(x = recency)) +
    theme_bw() +
    geom_bar(alpha = 0.6, width = 1) +
    ggtitle("Dustribution by recency")
```



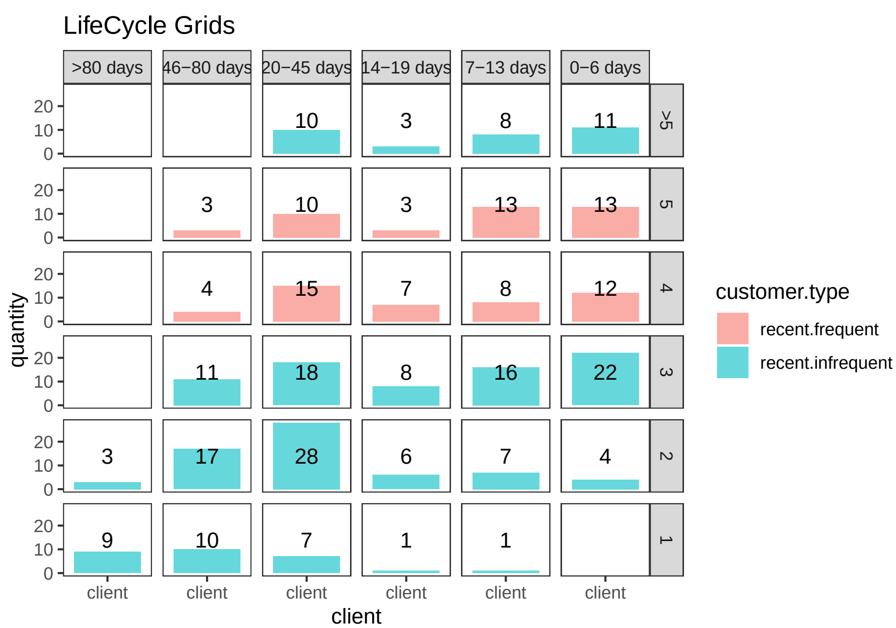

```
orders.segm <- orders %>%
    mutate(segm.freq = ifelse(between(frequency, 1, 1), '1',
                              ifelse(
                                  between(frequency, 2, 2), '2',
                                  ifelse(between(frequency, 3, 3), '3',
                                         ifelse(
                                             between(frequency, 4, 4), '4',
                                             ifelse(between(frequency, 5, 5), '5', '>5
                                         ))
                              ))) %>%
    mutate(segm.rec = ifelse(
        between(recency, 0, 6),
        '0-6 days',
        ifelse(
            between(recency, 7, 13),
            '7-13 days',
            ifelse(
                between(recency, 14, 19),
                '14-19 days',
                ifelse(
                    between(recency, 20, 45),
                    '20-45 days',
                    ifelse(between(recency, 46, 80), '46-80 days', '>80 days')
                )
            )
```

```
        )
    )) %>%
    # creating last cart feature
    mutate(cart = paste(
        ifelse(a != 0, 'a', ''),
        ifelse(b != 0, 'b', ''),
        ifelse(c != 0, 'c', ''),
        sep = ''
    )) %>%
    arrange(clientId)

# defining order of boundaries
orders.segm$segm.freq <-
    factor(orders.segm$segm.freq, levels = c('>5', '5', '4', '3', '2', '1'))
orders.segm$segm.rec <-
    factor(
        orders.segm$segm.rec,
        levels = c(
            '>80 days',
            '46-80 days',
            '20-45 days',
            '14-19 days',
            '7-13 days',
            '0-6 days'
        )
    )
```

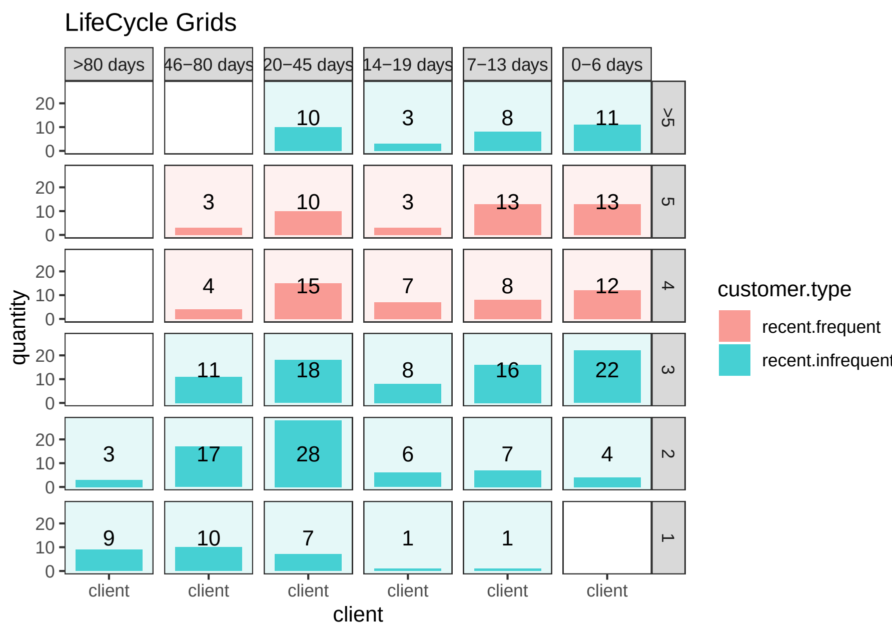
```
lcg <- orders.segm %>%
    group_by(segm.rec, segm.freq) %>%
    summarise(quantity = n()) %>%
    mutate(client = 'client') %>%
    ungroup()
lcg.matrix <-
    dcast(lcg,
          segm.freq ~ segm.rec,
          value.var = 'quantity',
          fun.aggregate = sum)

ggplot(lcg, aes(x = client, y = quantity, fill = quantity)) +
    theme_bw() +
    theme(panel.grid = element_blank()) +
    geom_bar(stat = 'identity', alpha = 0.6) +
    geom_text(aes(y = max(quantity) / 2, label = quantity), size = 4) +
    facet_grid(segm.freq ~ segm.rec) +
    ggtitle("LifeCycle Grids")
```



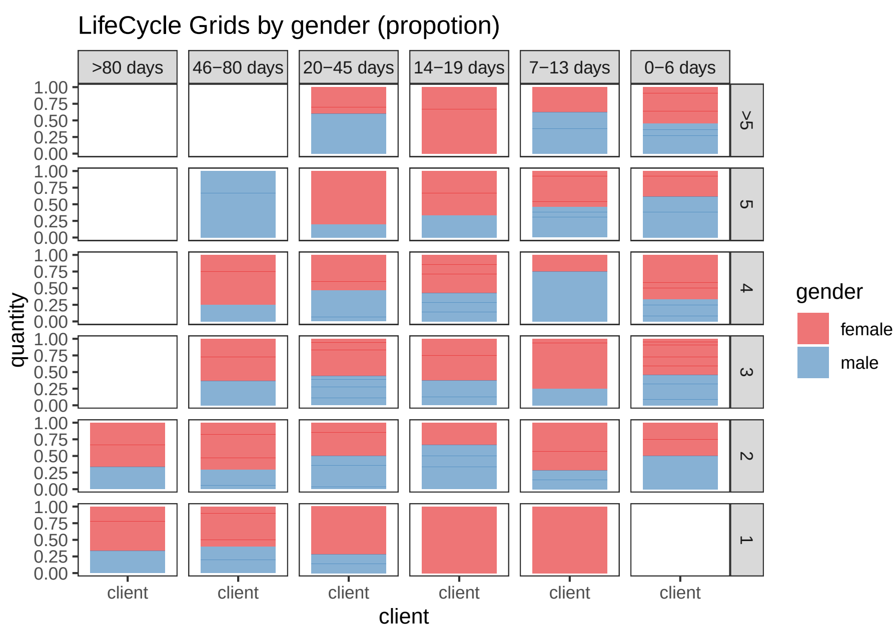

```
lcg.adv <- lcg %>%
    mutate(
        rec.type = ifelse(
            segm.rec %in% c("> 80 days", "46 - 80 days", "20 - 45 days"),
            "not recent",
            "recent"
        ),
        freq.type = ifelse(segm.freq %in% c(" >
          5", "5", "4"), "frequent", "infrequent"),
        customer.type = interaction(rec.type, freq.type)
    )

ggplot(lcg.adv, aes(x = client, y = quantity, fill = customer.type)) +
    theme_bw() +
    theme(panel.grid = element_blank()) +
    facet_grid(segm.freq ~ segm.rec) +
    geom_bar(stat = 'identity', alpha = 0.6) +
    geom_text(aes(y = max(quantity) / 2, label = quantity), size = 4) +
    ggtitle("LifeCycle Grids")
```



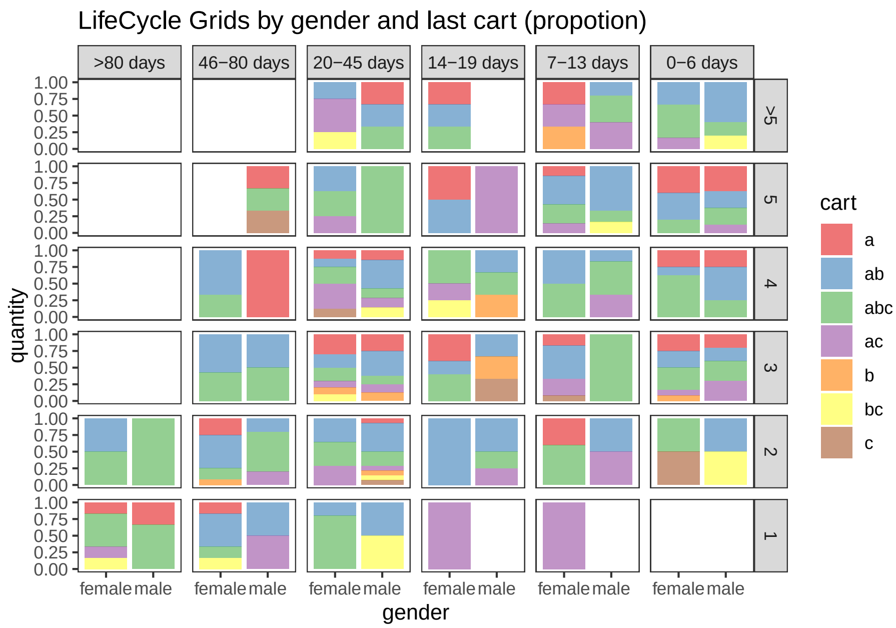

```
# with background
ggplot(lcg.adv, aes(x = client, y = quantity, fill = customer.type)) +
    theme_bw() +
    theme(panel.grid = element_blank()) +
    geom_rect(
        aes(fill = customer.type),
        xmin = -Inf,
        xmax = Inf,
        ymin = -Inf,
        ymax = Inf,
        alpha = 0.1
    ) +
    facet_grid(segm.freq ~ segm.rec) +
    geom_bar(stat = 'identity', alpha = 0.7) +
    geom_text(aes(y = max(quantity) / 2, label = quantity), size = 4) +
    ggtitle("LifeCycle Grids")
```




```
lcg.sub <- orders.segm %>%
    group_by(gender, cart, segm.rec, segm.freq) %>%
    summarise(quantity = n()) %>%
    mutate(client = 'client') %>%
    ungroup()

ggplot(lcg.sub, aes(x = client, y = quantity, fill = gender)) +
    theme_bw() +
    scale_fill_brewer(palette = 'Set1') +
    theme(panel.grid = element_blank()) +
    geom_bar(stat = 'identity',
             position = 'fill' ,
             alpha = 0.6) +
    facet_grid(segm.freq ~ segm.rec) +
    ggtitle("LifeCycle Grids by gender (propotion)")
```




```
ggplot(lcg.sub, aes(x = gender, y = quantity, fill = cart)) +
    theme_bw() +
    scale_fill_brewer(palette = 'Set1') +
    theme(panel.grid = element_blank()) +
    geom_bar(stat = 'identity',
             position = 'fill' ,
             alpha = 0.6) +
    facet_grid(segm.freq ~ segm.rec) +
    ggtitle("LifeCycle Grids by gender and last cart (propotion)")
```



15.11.2.2 CLV & CAC

calculate customer acquisition cost (CAC) and customer lifetime value (CLV)

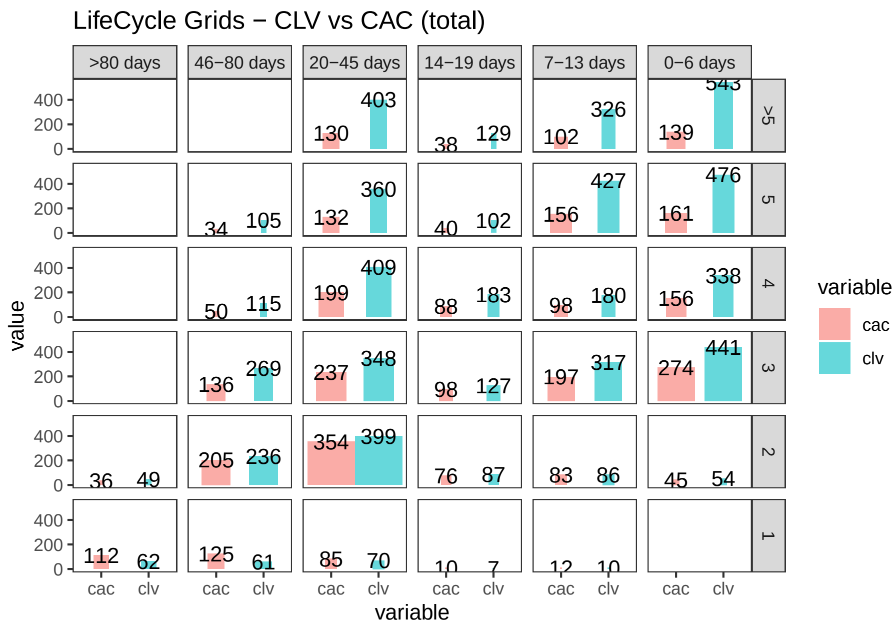
```
# loading libraries
library(dplyr)
library(reshape2)
library(ggplot2)

# creating data sample
set.seed(10)
data <- data.frame(
    orderId = sample(c(1:1000), 5000, replace = TRUE),
    product = sample(
        c('NULL', 'a', 'b', 'c'),
        5000,
        replace = TRUE,
        prob = c(0.15, 0.65, 0.3, 0.15)
    )
)
order <- data.frame(orderId = c(1:1000),
                    clientId = sample(c(1:300), 1000, replace = TRUE))
gender <- data.frame(clientId = c(1:300),
                     gender = sample(
                         c('male', 'female'),
```

```
                         300,
                         replace = TRUE,
                         prob = c(0.40, 0.60)
                     ))
date <- data.frame(orderId = c(1:1000),
                   orderdate = sample((1:100), 1000, replace = TRUE))
orders <- merge(data, order, by = 'orderId')
orders <- merge(orders, gender, by = 'clientId')
orders <- merge(orders, date, by = 'orderId')
orders <- orders[orders$product != 'NULL', ]
orders$orderdate <- as.Date(orders$orderdate, origin = "2012-01-01")

# creating data frames with CAC and Gross margin
cac <-
    data.frame(clientId = unique(orders$clientId),
               cac = sample(c(10:15), 288, replace = TRUE))
gr.margin <-
    data.frame(product = c('a', 'b', 'c'),
               grossmarg = c(1, 2, 3))

rm(data, date, order, gender)

# reporting date
today <- as.Date('2012-04-11', format = '%Y-%m-%d')

# calculating customer lifetime value
orders <- merge(orders, gr.margin, by = 'product')

clv <- orders %>%
    group_by(clientId) %>%
    summarise(clv = sum(grossmarg)) %>%
    ungroup()

# processing data
orders <-
    dcast(
        orders,
        orderId + clientId + gender + orderdate ~ product,
        value.var = 'product',
        fun.aggregate = length
    )

orders <- orders %>%
    group_by(clientId) %>%
    mutate(frequency = n(),
```

```
           recency = as.numeric(today - orderdate)) %>%
    filter(orderdate == max(orderdate)) %>%
    filter(orderId == max(orderId)) %>%
    ungroup()

orders.segm <- orders %>%
    mutate(segm.freq = ifelse(between(frequency, 1, 1), '1',
                              ifelse(
                                  between(frequency, 2, 2), '2',
                                  ifelse(between(frequency, 3, 3), '3',
                                         ifelse(
                                             between(frequency, 4, 4), '4',
                                             ifelse(between(frequency, 5, 5), '5', '>5
                                         ))
                              ))) %>%
    mutate(segm.rec = ifelse(
        between(recency, 0, 6),
        '0-6 days',
        ifelse(
            between(recency, 7, 13),
            '7-13 days',
            ifelse(
                between(recency, 14, 19),
                '14-19 days',
                ifelse(
                    between(recency, 20, 45),
                    '20-45 days',
                    ifelse(between(recency, 46, 80), '46-80 days', '>80 days')
                )
            )
        )
    )) %>%
    # creating last cart feature
    mutate(cart = paste(
        ifelse(a != 0, 'a', ''),
        ifelse(b != 0, 'b', ''),
        ifelse(c != 0, 'c', ''),
        sep = ''
    )) %>%
    arrange(clientId)

# defining order of boundaries
orders.segm$segm.freq <-
    factor(orders.segm$segm.freq, levels = c('>5', '5', '4', '3', '2', '1'))
orders.segm$segm.rec <-
```

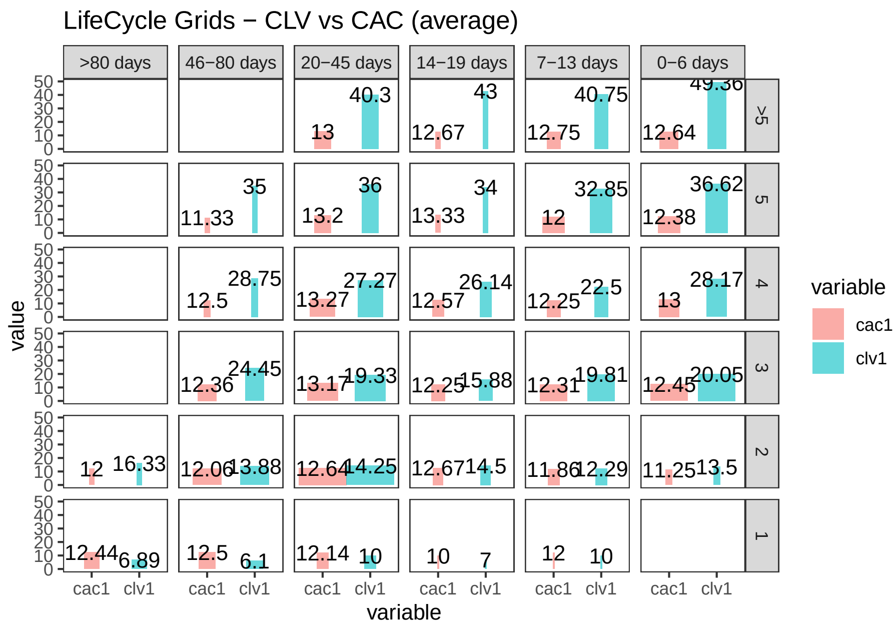
```
    factor(
        orders.segm$segm.rec,
        levels = c(
            '>80 days',
            '46-80 days',
            '20-45 days',
            '14-19 days',
            '7-13 days',
            '0-6 days'
        )
    )

orders.segm <- merge(orders.segm, cac, by = 'clientId')
orders.segm <- merge(orders.segm, clv, by = 'clientId')

lcg.clv <- orders.segm %>%
    group_by(segm.rec, segm.freq) %>%
    summarise(quantity = n(),
              # calculating cumulative CAC and CLV
              cac = sum(cac),
              clv = sum(clv)) %>%
    ungroup() %>%
    # calculating CAC and CLV per client
    mutate(cac1 = round(cac / quantity, 2),
           clv1 = round(clv / quantity, 2))

lcg.clv <-
    reshape2::melt(lcg.clv, id.vars = c('segm.rec', 'segm.freq', 'quantity'))

ggplot(lcg.clv[lcg.clv$variable %in% c('clv', 'cac'), ], aes(x = variable, y =
                                                                 value, fill = variable)) +
    theme_bw() +
    theme(panel.grid = element_blank()) +
    geom_bar(stat = 'identity', alpha = 0.6, aes(width = quantity / max(quantity))) +
    geom_text(aes(y = value, label = value), size = 4) +
    facet_grid(segm.freq ~ segm.rec) +
    ggtitle("LifeCycle Grids - CLV vs CAC (total)")
```

```
## Warning: Ignoring unknown aesthetics: width
```




```
ggplot(lcg.clv[lcg.clv$variable %in% c('clv1', 'cac1'), ], aes(x = variable, y =
                                                                  value, fill = varial
    theme_bw() +
    theme(panel.grid = element_blank()) +
    geom_bar(stat = 'identity', alpha = 0.6, aes(width = quantity / max(quantity))) +
    geom_text(aes(y = value, label = value), size = 4) +
    facet_grid(segm.freq ~ segm.rec) +
    ggtitle("LifeCycle Grids - CLV vs CAC (average)")
```

```
## Warning: Ignoring unknown aesthetics: width
```



15.11.2.3 Cohort Analysis

link

combine customers through common characteristics to split customers into homogeneous groups

```
# loading libraries
library(dplyr)
library(reshape2)
library(ggplot2)
library(googleVis)

set.seed(10)
# creating orders data sample
data <- data.frame(
    orderId = sample(c(1:5000), 25000, replace = TRUE),
    product = sample(
        c('NULL', 'a', 'b', 'c'),
        25000,
        replace = TRUE,
        prob = c(0.15, 0.65, 0.3, 0.15)
    )
)
order <- data.frame(orderId = c(1:5000),
                    clientId = sample(c(1:1500), 5000, replace = TRUE))
```

```
date <- data.frame(orderId = c(1:5000),
                   orderdate = sample((1:500), 5000, replace = TRUE))
orders <- merge(data, order, by = 'orderId')
orders <- merge(orders, date, by = 'orderId')
orders <- orders[orders$product != 'NULL',]
orders$orderdate <- as.Date(orders$orderdate, origin = "2012-01-01")
rm(data, date, order)
# creating data frames with CAC, Gross margin, Campaigns and Potential CLV
gr.margin <-
    data.frame(product = c('a', 'b', 'c'),
               grossmarg = c(1, 2, 3))
campaign <- data.frame(clientId = c(1:1500),
                       campaign = paste('campaign', sample(c(1:7), 1500, replace = TRU
                                                    ' '))
cac <-
    data.frame(campaign = unique(campaign$campaign),
               cac = sample(c(10:15), 7, replace = TRUE))
campaign <- merge(campaign, cac, by = 'campaign')
potential <- data.frame(clientId = c(1:1500),
                        clv.p = sample(c(0:50), 1500, replace = TRUE))
rm(cac)

# reporting date
today <- as.Date('2013-05-16', format = '%Y-%m-%d')
```

where

- campaign, which includes campaign name and customer acquisition cost for each customer,
- margin, which includes gross margin for each product,
- potential, which includes potential values / predicted CLV for each client,
- orders, which includes orders from our customers with products and order dates.

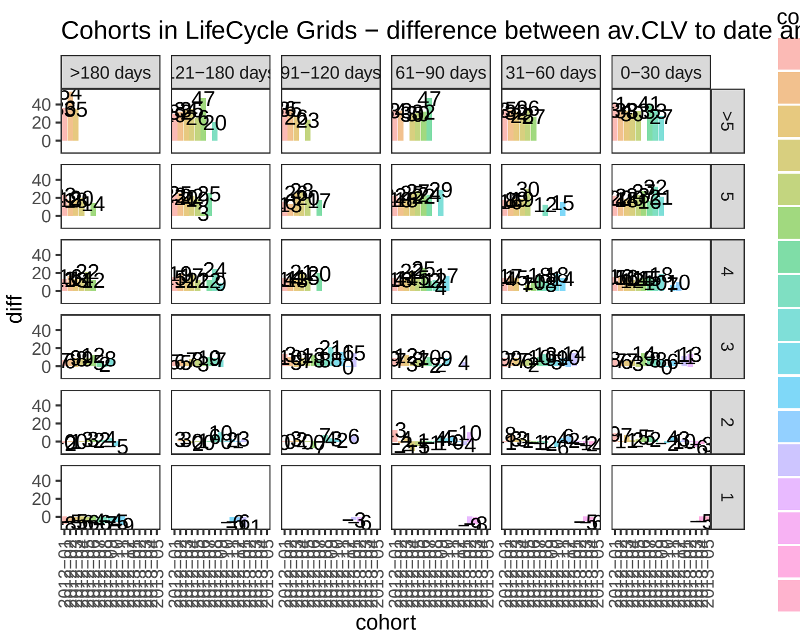
```
# calculating CLV, frequency, recency, average time lapses between purchases and defin

orders <- merge(orders, gr.margin, by = 'product')

customers <- orders %>%
    # combining products and summarising gross margin
    group_by(orderId, clientId, orderdate) %>%
    summarise(grossmarg = sum(grossmarg)) %>%
    ungroup() %>%
    # calculating frequency, recency, average time lapses between purchases and defini
    group_by(clientId) %>%
    mutate(
```

```
        frequency = n(),
        recency = as.numeric(today - max(orderdate)),
        av.gap = round(as.numeric(max(orderdate) - min(orderdate)) / frequency, 0),
        cohort = format(min(orderdate), format = '%Y-%m')
    ) %>%
    ungroup() %>%
    # calculating CLV to date
    group_by(clientId, cohort, frequency, recency, av.gap) %>%
    summarise(clv = sum(grossmarg)) %>%
    arrange(clientId) %>%
    ungroup()
# calculating potential CLV and CAC
customers <- merge(customers, campaign, by = 'clientId')
customers <- merge(customers, potential, by = 'clientId')
# leading the potential value to more or less real value
customers$clv.p <-
    round(customers$clv.p / sqrt(customers$recency) * customers$frequency,
          2)

rm(potential, gr.margin, today)
# adding segments
customers <- customers %>%
    mutate(segm.freq = ifelse(between(frequency, 1, 1), '1',
                              ifelse(
                                  between(frequency, 2, 2), '2',
                                  ifelse(between(frequency, 3, 3), '3',
                                         ifelse(
                                             between(frequency, 4, 4), '4',
                                             ifelse(between(frequency, 5, 5), '5', '>5')
                                         ))
                              ))) %>%
    mutate(segm.rec = ifelse(
        between(recency, 0, 30),
        '0-30 days',
        ifelse(
            between(recency, 31, 60),
            '31-60 days',
            ifelse(
                between(recency, 61, 90),
                '61-90 days',
                ifelse(
                    between(recency, 91, 120),
                    '91-120 days',
                    ifelse(between(recency, 121, 180), '121-180 days', '>180 days')
                )
```

```
            )
        )
    ))

# defining order of boundaries
customers$segm.freq <-
    factor(customers$segm.freq, levels = c('>5', '5', '4', '3', '2', '1'))
customers$segm.rec <-
    factor(
        customers$segm.rec,
        levels = c(
            '>180 days',
            '121-180 days',
            '91-120 days',
            '61-90 days',
            '31-60 days',
            '0-30 days'
        )
    )
```

```
lcg.coh <- customers %>%
    group_by(cohort, segm.rec, segm.freq) %>%
    # calculating cumulative values
    summarise(
        quantity = n(),
        cac = sum(cac),
        clv = sum(clv),
        clv.p = sum(clv.p),
        av.gap = sum(av.gap)
    ) %>%
    ungroup() %>%
    # calculating average values
    mutate(
        av.cac = round(cac / quantity, 2),
        av.clv = round(clv / quantity, 2),
        av.clv.p = round(clv.p / quantity, 2),
        av.clv.tot = av.clv + av.clv.p,
        av.gap = round(av.gap / quantity, 2),
        diff = av.clv - av.cac
    )

# 1. Structure of averages and comparison cohorts

ggplot(lcg.coh, aes(x = cohort, fill = cohort)) +
```

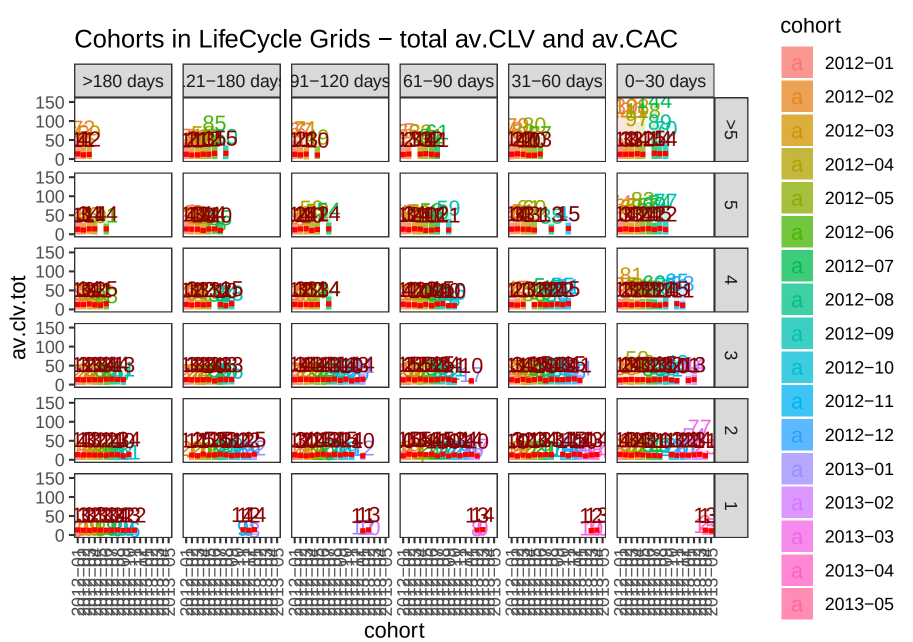
```
    theme_bw() +
    theme(panel.grid = element_blank()) +
    geom_bar(aes(y = diff), stat = 'identity', alpha = 0.5) +
    geom_text(aes(y = diff, label = round(diff, 0)), size = 4) +
    facet_grid(segm.freq ~ segm.rec) +
    theme(axis.text.x = element_text(
        angle = 90,
        hjust = .5,
        vjust = .5,
        face = "plain"
    )) +
    ggtitle("Cohorts in LifeCycle Grids - difference between av.CLV to date and av.CAC")
```



15.11.2.3.1 First-purchase date cohort

```
ggplot(lcg.coh, aes(x = cohort, fill = cohort)) +
    theme_bw() +
    theme(panel.grid = element_blank()) +
    geom_bar(aes(y = av.clv.tot), stat = 'identity', alpha = 0.2) +
    geom_text(aes(
        y = av.clv.tot + 10,
        label = round(av.clv.tot, 0),
        color = cohort
    ), size = 4) +
    geom_bar(aes(y = av.clv), stat = 'identity', alpha = 0.7) +
    geom_errorbar(aes(y = av.cac, ymax = av.cac, ymin = av.cac),
                  color = 'red',
```

```
                  size = 1.2) +
    geom_text(
        aes(y = av.cac, label = round(av.cac, 0)),
        size = 4,
        color = 'darkred',
        vjust = -.5
    ) +
    facet_grid(segm.freq ~ segm.rec) +
    theme(axis.text.x = element_text(
        angle = 90,
        hjust = .5,
        vjust = .5,
        face = "plain"
    )) +
    ggtitle("Cohorts in LifeCycle Grids - total av.CLV and av.CAC")
```

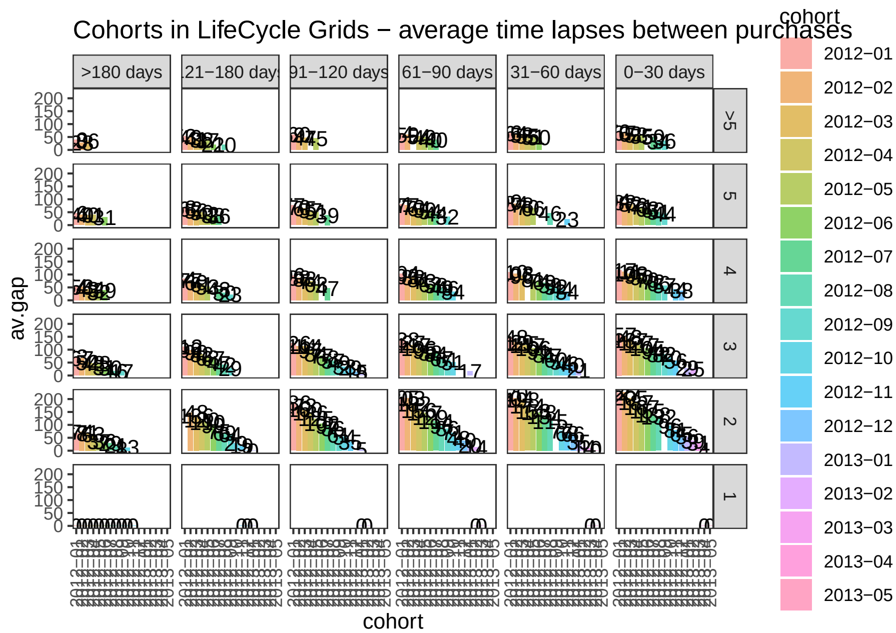



```
# 2. Analyzing customer flows
# customers flows analysis (FPD cohorts)

# defining cohort and reporting dates
coh <- '2012-09'
report.dates <- c('2012-10-01', '2013-01-01', '2013-04-01')
report.dates <- as.Date(report.dates, format = '%Y-%m-%d')

# defining segments for each cohort's customer for reporting dates
```

```
df.sankey <- data.frame()

for (i in 1:length(report.dates)) {
    orders.cache <- orders %>%
        filter(orderdate < report.dates[i])

    customers.cache <- orders.cache %>%
        select(-product,-grossmarg) %>%
        unique() %>%
        group_by(clientId) %>%
        mutate(
            frequency = n(),
            recency = as.numeric(report.dates[i] - max(orderdate)),
            cohort = format(min(orderdate), format = '%Y-%m')
        ) %>%
        ungroup() %>%
        select(clientId, frequency, recency, cohort) %>%
        unique() %>%
        filter(cohort == coh) %>%
        mutate(segm.freq = ifelse(
            between(frequency, 1, 1),
            '1 purch',
            ifelse(
                between(frequency, 2, 2),
                '2 purch',
                ifelse(
                    between(frequency, 3, 3),
                    '3 purch',
                    ifelse(
                        between(frequency, 4, 4),
                        '4 purch',
                        ifelse(between(frequency, 5, 5), '5 purch', '>5 purch')
                    )
                )
            )
        )) %>%
        mutate(segm.rec = ifelse(
            between(recency, 0, 30),
            '0-30 days',
            ifelse(
                between(recency, 31, 60),
                '31-60 days',
                ifelse(
                    between(recency, 61, 90),
                    '61-90 days',
```

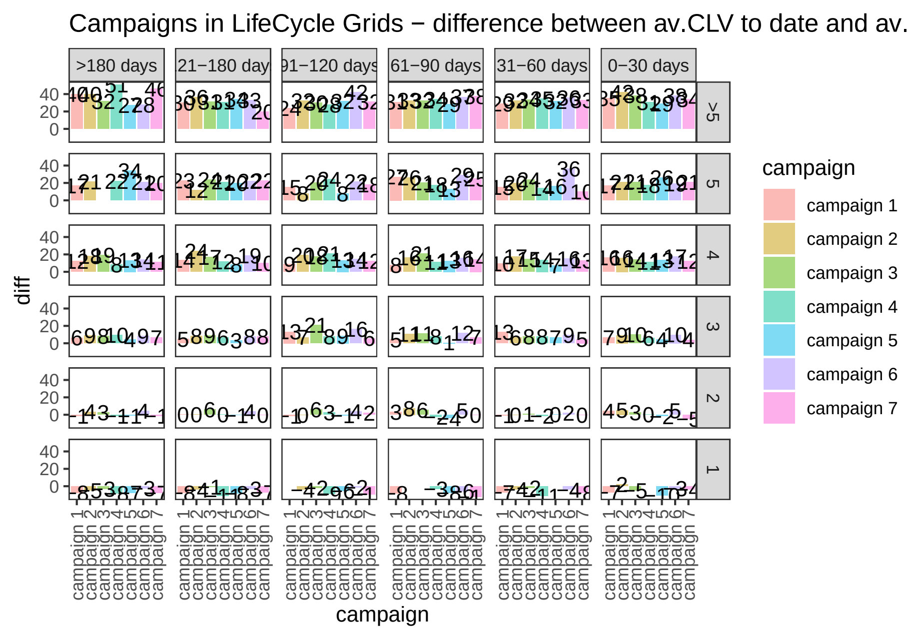
```
                    ifelse(
                        between(recency, 91, 120),
                        '91-120 days',
                        ifelse(between(recency, 121, 180), '121-180 days', '>180 days')
                    )
                )
            )
        )) %>%
        mutate(
            cohort.segm = paste(cohort, segm.rec, segm.freq, sep = ' : '),
            report.date = report.dates[i]
        ) %>%
        select(clientId, cohort.segm, report.date)

    df.sankey <- rbind(df.sankey, customers.cache)
}

# processing data for Sankey diagram format
df.sankey <-
    dcast(df.sankey,
          clientId ~ report.date,
          value.var = 'cohort.segm',
          fun.aggregate = NULL)
write.csv(df.sankey, 'customers_path.csv', row.names = FALSE)
df.sankey <- df.sankey %>% select(-clientId)

df.sankey.plot <- data.frame()
for (i in 2:ncol(df.sankey)) {
    df.sankey.cache <- df.sankey %>%
        group_by(df.sankey[, i - 1], df.sankey[, i]) %>%
        summarise(n = n()) %>%
        ungroup()

    colnames(df.sankey.cache)[1:2] <- c('from', 'to')

    df.sankey.cache$from <-
        paste(df.sankey.cache$from, ' (', report.dates[i - 1], ')', sep = '')
    df.sankey.cache$to <-
        paste(df.sankey.cache$to, ' (', report.dates[i], ')', sep = '')

    df.sankey.plot <- rbind(df.sankey.plot, df.sankey.cache)
}

# plotting
plot(gvisSankey(
```

```
    df.sankey.plot,
    from = 'from',
    to = 'to',
    weight = 'n',
    options = list(
        height = 900,
        width = 1800,
        sankey = "{link:{color:{fill:'lightblue'}}}"
    )
))

# purchasing pace

ggplot(lcg.coh, aes(x = cohort, fill = cohort)) +
    theme_bw() +
    theme(panel.grid = element_blank()) +
    geom_bar(aes(y = av.gap), stat = 'identity', alpha = 0.6) +
    geom_text(aes(y = av.gap, label = round(av.gap, 0)), size = 4) +
    facet_grid(segm.freq ~ segm.rec) +
    theme(axis.text.x = element_text(
        angle = 90,
        hjust = .5,
        vjust = .5,
        face = "plain"
    )) +
    ggtitle("Cohorts in LifeCycle Grids - average time lapses between purchases")
```




```r
# campaign cohorts
lcg.camp <- customers %>%
    group_by(campaign, segm.rec, segm.freq) %>%
    # calculating cumulative values
    summarise(
        quantity = n(),
        cac = sum(cac),
        clv = sum(clv),
        clv.p = sum(clv.p),
        av.gap = sum(av.gap)
    ) %>%
    ungroup() %>%
    # calculating average values
    mutate(
        av.cac = round(cac / quantity, 2),
        av.clv = round(clv / quantity, 2),
        av.clv.p = round(clv.p / quantity, 2),
        av.clv.tot = av.clv + av.clv.p,
        av.gap = round(av.gap / quantity, 2),
        diff = av.clv - av.cac
    )

ggplot(lcg.camp, aes(x = campaign, fill = campaign)) +
```

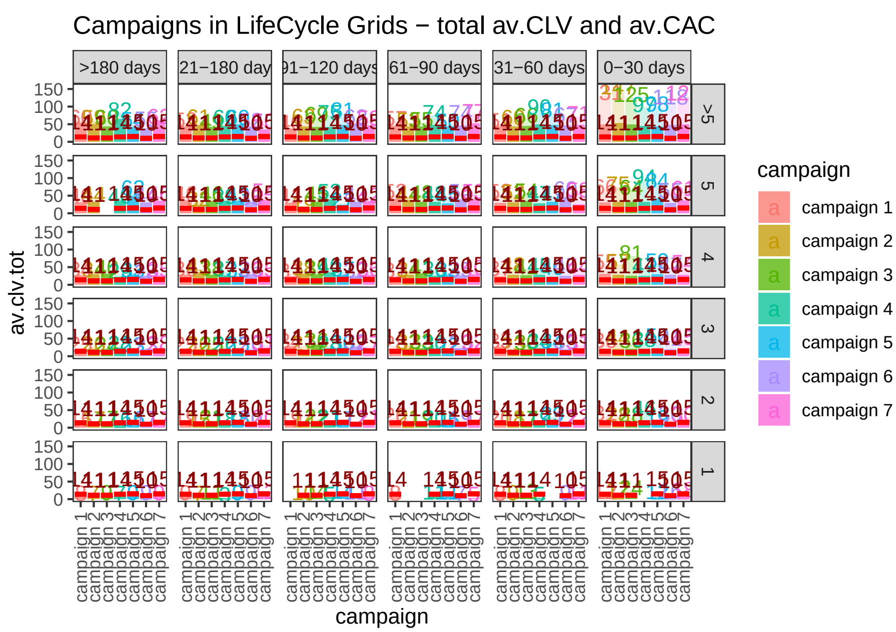
```
    theme_bw() +
    theme(panel.grid = element_blank()) +
    geom_bar(aes(y = diff), stat = 'identity', alpha = 0.5) +
    geom_text(aes(y = diff, label = round(diff, 0)), size = 4) +
    facet_grid(segm.freq ~ segm.rec) +
    theme(axis.text.x = element_text(
        angle = 90,
        hjust = .5,
        vjust = .5,
        face = "plain"
    )) +
    ggtitle("Campaigns in LifeCycle Grids - difference between av.CLV to date and av.CAC")
```



15.11.2.3.2 Campaign Cohorts

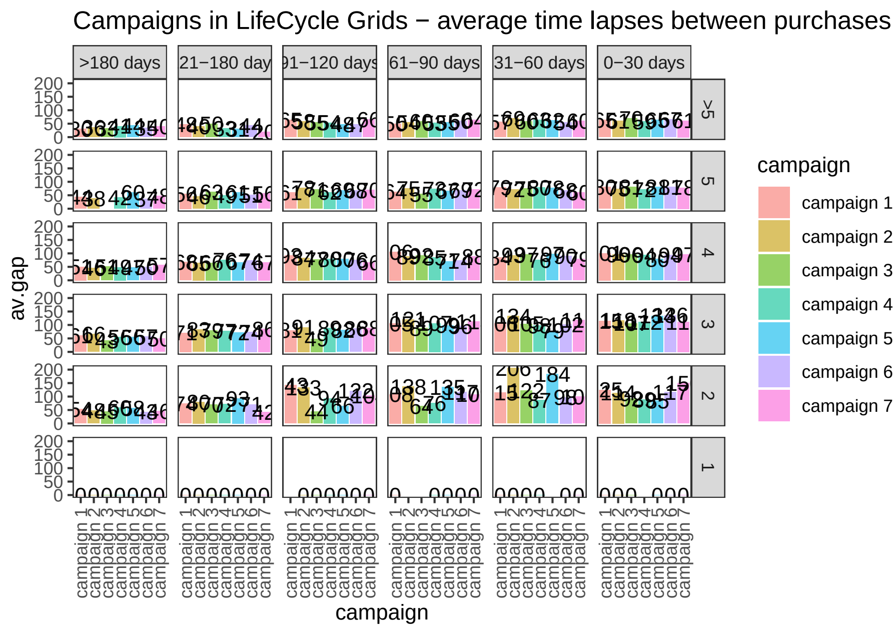
```
ggplot(lcg.camp, aes(x = campaign, fill = campaign)) +
    theme_bw() +
    theme(panel.grid = element_blank()) +
    geom_bar(aes(y = av.clv.tot), stat = 'identity', alpha = 0.2) +
    geom_text(aes(
        y = av.clv.tot + 10,
        label = round(av.clv.tot, 0),
        color = campaign
    ), size = 4) +
    geom_bar(aes(y = av.clv), stat = 'identity', alpha = 0.7) +
    geom_errorbar(aes(y = av.cac, ymax = av.cac, ymin = av.cac),
                  color = 'red',
```

```
                  size = 1.2) +
    geom_text(
        aes(y = av.cac, label = round(av.cac, 0)),
        size = 4,
        color = 'darkred',
        vjust = -.5
    ) +
    facet_grid(segm.freq ~ segm.rec) +
    theme(axis.text.x = element_text(
        angle = 90,
        hjust = .5,
        vjust = .5,
        face = "plain"
    )) +
    ggtitle("Campaigns in LifeCycle Grids - total av.CLV and av.CAC")
```




```
ggplot(lcg.camp, aes(x = campaign, fill = campaign)) +
    theme_bw() +
    theme(panel.grid = element_blank()) +
    geom_bar(aes(y = av.gap), stat = 'identity', alpha = 0.6) +
    geom_text(aes(y = av.gap, label = round(av.gap, 0)), size = 4) +
    facet_grid(segm.freq ~ segm.rec) +
    theme(axis.text.x = element_text(
        angle = 90,
        hjust = .5,
```

```
        vjust = .5,
        face = "plain"
    )) +
    ggtitle("Campaigns in LifeCycle Grids - average time lapses between purchases")
```



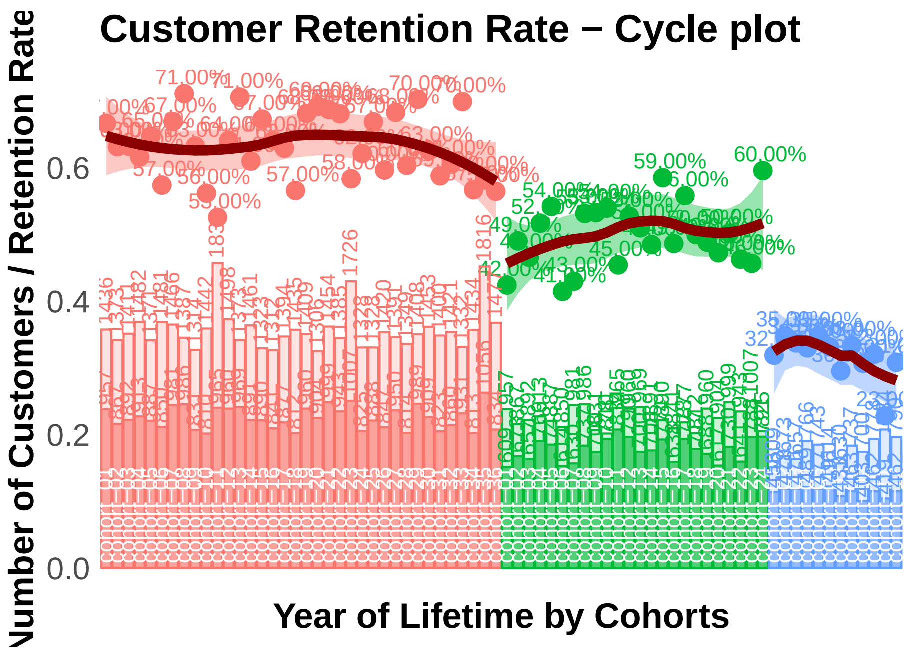
15.11.2.3.3 Retention Rate Customer Retention Rate

```
# loading libraries
library(dplyr)
library(reshape2)
library(ggplot2)
library(scales)
library(gridExtra)
# creating data sample
set.seed(10)
cohorts <-
    data.frame(
        cohort = paste('cohort', formatC(
            c(1:36),
            width = 2,
            format = 'd',
            flag = '0'
        ), sep = '_'),
        Y_00 = sample(c(1300:1500), 36, replace = TRUE),
        Y_01 = c(sample(c(800:1000), 36, replace = TRUE)),
```

```
        Y_02 = c(sample(c(600:800), 24, replace = TRUE), rep(NA, 12)),
        Y_03 = c(sample(c(400:500), 12, replace = TRUE), rep(NA, 24))
    )
# simulating seasonality (Black Friday)
cohorts[c(11, 23, 35), 2] <-
    as.integer(cohorts[c(11, 23, 35), 2] * 1.25)
cohorts[c(11, 23, 35), 3] <-
    as.integer(cohorts[c(11, 23, 35), 3] * 1.10)
cohorts[c(11, 23, 35), 4] <-
    as.integer(cohorts[c(11, 23, 35), 4] * 1.07)

# calculating retention rate and preparing data for plotting
df_plot <-
    reshape2::melt(
        cohorts,
        id.vars = 'cohort',
        value.name = 'number',
        variable.name = "year_of_LT"
    )

df_plot <- df_plot %>%
    group_by(cohort) %>%
    arrange(year_of_LT) %>%
    mutate(number_prev_year = lag(number),
           number_Y_00 = number[which(year_of_LT == 'Y_00')]) %>%
    ungroup() %>%
    mutate(
        ret_rate_prev_year = number / number_prev_year,
        ret_rate = number / number_Y_00,
        year_cohort = paste(year_of_LT, cohort, sep = '-')
    )

##### The first way for plotting cycle plot via scaling
# calculating the coefficient for scaling 2nd axis
k <-
    max(df_plot$number_prev_year[df_plot$year_of_LT == 'Y_01'] * 1.15) / min(df_plot$re

# retention rate cycle plot
ggplot(
    na.omit(df_plot),
    aes(
        x = year_cohort,
        y = ret_rate,
        group = year_of_LT,
        color = year_of_LT
```

```
    )
) +
    theme_bw() +
    geom_point(size = 4) +
    geom_text(
        aes(label = percent(round(ret_rate, 2))),
        size = 4,
        hjust = 0.4,
        vjust = -0.6,
        fontface = "plain"
    ) +
    # smooth method can be changed (e.g. for "lm")
    geom_smooth(
        size = 2.5,
        method = 'loess',
        color = 'darkred',
        aes(fill = year_of_LT)
    ) +
    geom_bar(aes(y = number_prev_year / k, fill = year_of_LT),
             alpha = 0.2,
             stat = 'identity') +
    geom_bar(aes(y = number / k, fill = year_of_LT),
             alpha = 0.6,
             stat = 'identity') +
    geom_text(
        aes(y = 0, label = cohort),
        color = 'white',
        angle = 90,
        size = 4,
        hjust = -0.05,
        vjust = 0.4
    ) +
    geom_text(
        aes(y = number_prev_year / k, label = number_prev_year),
        angle = 90,
        size = 4,
        hjust = -0.1,
        vjust = 0.4
    ) +
    geom_text(
        aes(y = number / k, label = number),
        angle = 90,
        size = 4,
        hjust = -0.1,
        vjust = 0.4
```

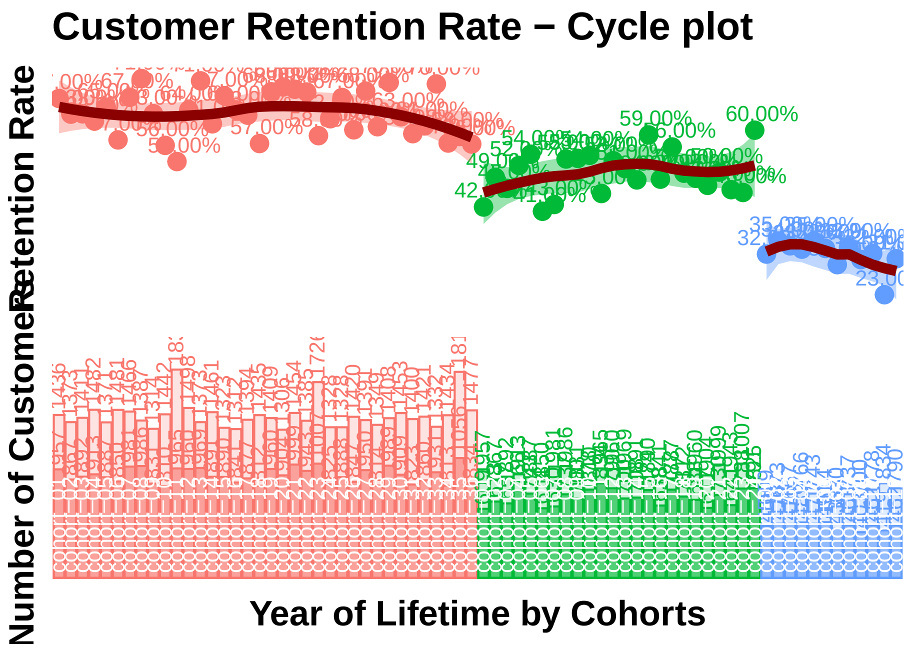
```
    ) +
    theme(
        legend.position = 'none',
        plot.title = element_text(size = 20, face = "bold", vjust = 2),
        axis.title.x = element_text(size = 18, face = "bold"),
        axis.title.y = element_text(size = 18, face = "bold"),
        axis.text = element_text(size = 16),
        axis.text.x = element_blank(),
        axis.ticks.x = element_blank(),
        axis.ticks.y = element_blank(),
        panel.border = element_blank(),
        panel.grid.major = element_blank(),
        panel.grid.minor = element_blank()
    ) +
    labs(x = 'Year of Lifetime by Cohorts', y = 'Number of Customers / Retention Rate')
    ggtitle("Customer Retention Rate - Cycle plot")
```




```
##### The second way for plotting cycle plot via multi-plotting
# plot #1 - Retention rate
p1 <-
    ggplot(
        na.omit(df_plot),
        aes(
            x = year_cohort,
            y = ret_rate,
```

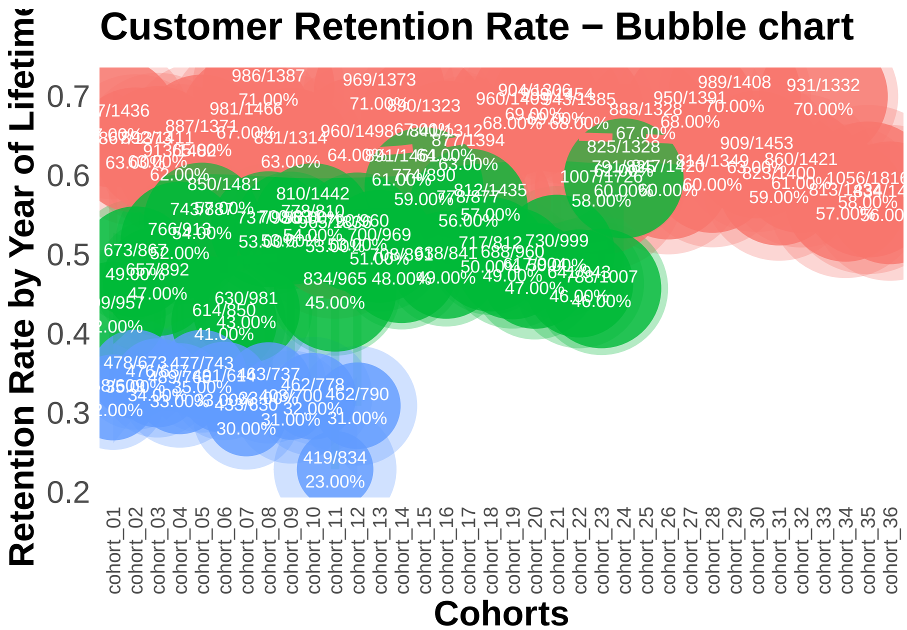
```
            group = year_of_LT,
            color = year_of_LT
        )
    ) +
    theme_bw() +
    geom_point(size = 4) +
    geom_text(
        aes(label = percent(round(ret_rate, 2))),
        size = 4,
        hjust = 0.4,
        vjust = -0.6,
        fontface = "plain"
    ) +
    geom_smooth(
        size = 2.5,
        method = 'loess',
        color = 'darkred',
        aes(fill = year_of_LT)
    ) +
    theme(
        legend.position = 'none',
        plot.title = element_text(size = 20, face = "bold", vjust = 2),
        axis.title.x = element_blank(),
        axis.title.y = element_text(size = 18, face = "bold"),
        axis.text = element_blank(),
        axis.ticks.x = element_blank(),
        axis.ticks.y = element_blank(),
        panel.border = element_blank(),
        panel.grid.major = element_blank(),
        panel.grid.minor = element_blank()
    ) +
    labs(y = 'Retention Rate') +
    ggtitle("Customer Retention Rate - Cycle plot")

# plot #2 - number of customers
p2 <-
    ggplot(na.omit(df_plot),
           aes(x = year_cohort, group = year_of_LT, color = year_of_LT)) +
    theme_bw() +
    geom_bar(aes(y = number_prev_year, fill = year_of_LT),
             alpha = 0.2,
             stat = 'identity') +
    geom_bar(aes(y = number, fill = year_of_LT),
             alpha = 0.6,
             stat = 'identity') +
```

```
    geom_text(
        aes(y = number_prev_year, label = number_prev_year),
        angle = 90,
        size = 4,
        hjust = -0.1,
        vjust = 0.4
    ) +
    geom_text(
        aes(y = number, label = number),
        angle = 90,
        size = 4,
        hjust = -0.1,
        vjust = 0.4
    ) +
    geom_text(
        aes(y = 0, label = cohort),
        color = 'white',
        angle = 90,
        size = 4,
        hjust = -0.05,
        vjust = 0.4
    ) +
    theme(
        legend.position = 'none',
        plot.title = element_text(size = 20, face = "bold", vjust = 2),
        axis.title.x = element_text(size = 18, face = "bold"),
        axis.title.y = element_text(size = 18, face = "bold"),
        axis.text = element_blank(),
        axis.ticks.x = element_blank(),
        axis.ticks.y = element_blank(),
        panel.border = element_blank(),
        panel.grid.major = element_blank(),
        panel.grid.minor = element_blank()
    ) +
    scale_y_continuous(limits = c(0, max(df_plot$number_Y_00 * 1.1))) +
    labs(x = 'Year of Lifetime by Cohorts', y = 'Number of Customers')

# multiplot
grid.arrange(p1, p2, ncol = 1)
```




```
# retention rate bubble chart
ggplot(na.omit(df_plot),
       aes(
           x = cohort,
           y = ret_rate,
           group = cohort,
           color = year_of_LT
       )) +
    theme_bw() +
    scale_size(range = c(15, 40)) +
    geom_line(size = 2, alpha = 0.3) +
    geom_point(aes(size = number_prev_year), alpha = 0.3) +
    geom_point(aes(size = number), alpha = 0.8) +
    geom_smooth(
        linetype = 2,
        size = 2,
        method = 'loess',
        aes(group = year_of_LT, fill = year_of_LT),
        alpha = 0.2
    ) +
    geom_text(
        aes(label = paste0(
            number, '/', number_prev_year, '\n', percent(round(ret_rate, 2))
        )),
        color = 'white',
```

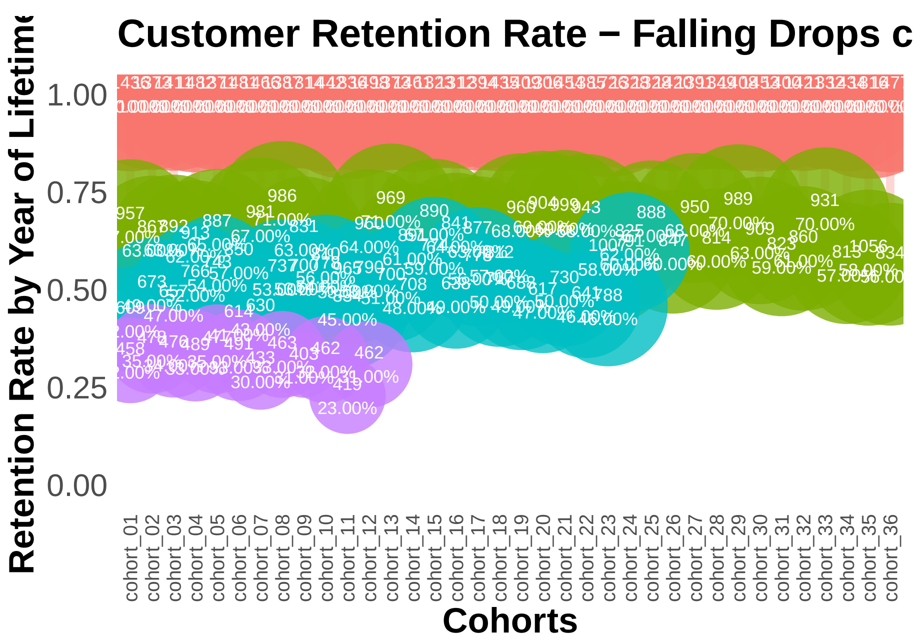
```
        size = 3,
        hjust = 0.5,
        vjust = 0.5,
        fontface = "plain"
    ) +
    theme(
        legend.position = 'none',
        plot.title = element_text(size = 20, face = "bold", vjust = 2),
        axis.title.x = element_text(size = 18, face = "bold"),
        axis.title.y = element_text(size = 18, face = "bold"),
        axis.text = element_text(size = 16),
        axis.text.x = element_text(
            size = 10,
            angle = 90,
            hjust = .5,
            vjust = .5,
            face = "plain"
        ),
        axis.ticks.x = element_blank(),
        axis.ticks.y = element_blank(),
        panel.border = element_blank(),
        panel.grid.major = element_blank(),
        panel.grid.minor = element_blank()
    ) +
    labs(x = 'Cohorts', y = 'Retention Rate by Year of Lifetime') +
    ggtitle("Customer Retention Rate - Bubble chart")
```




```r
# retention rate falling drops chart
ggplot(df_plot,
       aes(
           x = cohort,
           y = ret_rate,
           group = cohort,
           color = year_of_LT
       )) +
    theme_bw() +
    scale_size(range = c(15, 40)) +
    scale_y_continuous(limits = c(0, 1)) +
    geom_line(size = 2, alpha = 0.3) +
    geom_point(aes(size = number), alpha = 0.8) +
    geom_text(
        aes(label = paste0(number, '\n', percent(round(
            ret_rate, 2
        )))),
        color = 'white',
        size = 3,
        hjust = 0.5,
        vjust = 0.5,
        fontface = "plain"
    ) +
    theme(
        legend.position = 'none',
```

```
        plot.title = element_text(size = 20, face = "bold", vjust = 2),
        axis.title.x = element_text(size = 18, face = "bold"),
        axis.title.y = element_text(size = 18, face = "bold"),
        axis.text = element_text(size = 16),
        axis.text.x = element_text(
            size = 10,
            angle = 90,
            hjust = .5,
            vjust = .5,
            face = "plain"
        ),
        axis.ticks.x = element_blank(),
        axis.ticks.y = element_blank(),
        panel.border = element_blank(),
        panel.grid.major = element_blank(),
        panel.grid.minor = element_blank()
    ) +
    labs(x = 'Cohorts', y = 'Retention Rate by Year of Lifetime') +
    ggtitle("Customer Retention Rate - Falling Drops chart")
```

```
## Warning: Removed 36 row(s) containing missing values (geom_path).

## Warning: Removed 36 rows containing missing values (geom_point).

## Warning: Removed 36 rows containing missing values (geom_text).
```



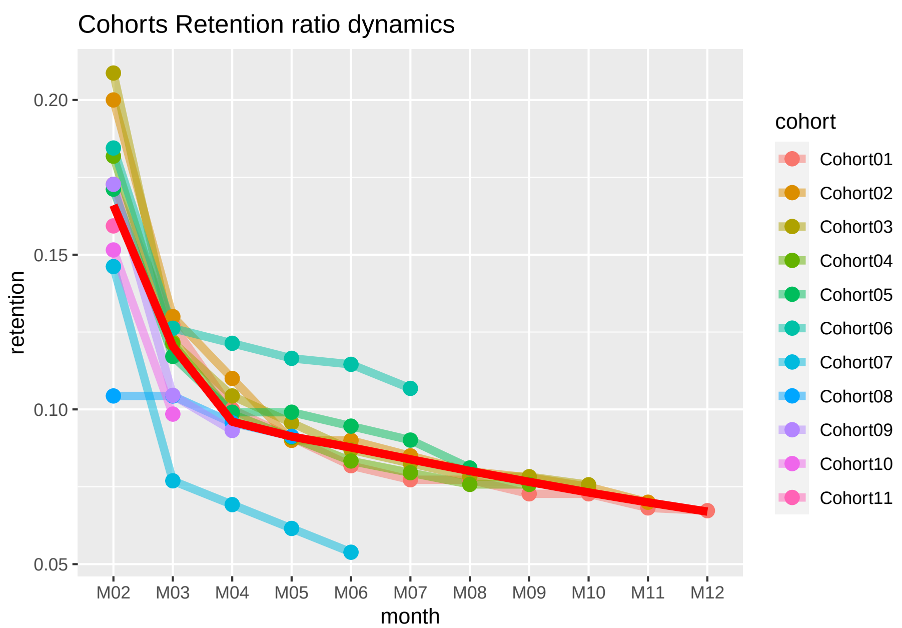
15.11.2.3.4 Retention Charts

Retention charts

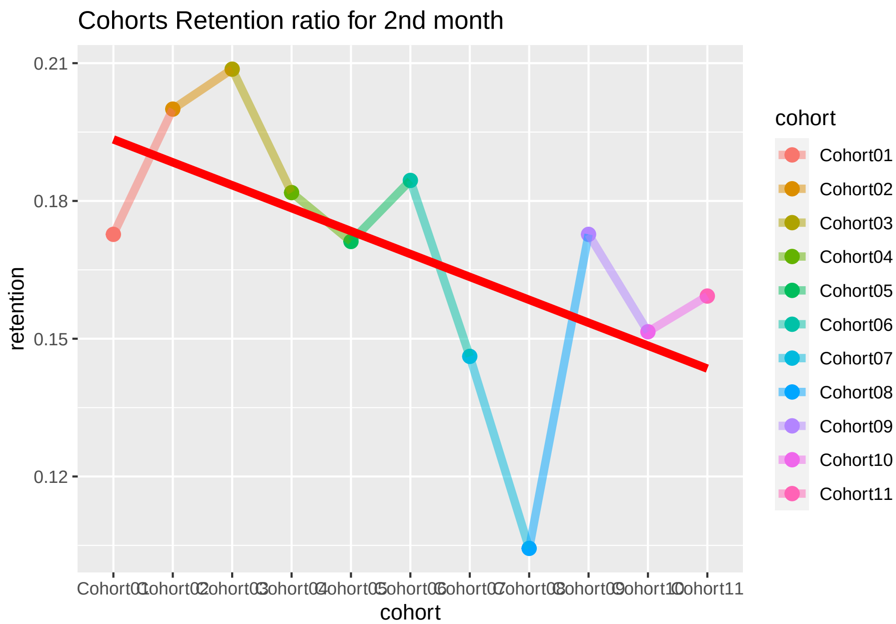
```
# libraries
library(dplyr)
library(ggplot2)
library(reshape2)

cohort.clients <- data.frame(
    cohort = c(
        'Cohort01',
        'Cohort02',
        'Cohort03',
        'Cohort04',
        'Cohort05',
        'Cohort06',
        'Cohort07',
        'Cohort08',
        'Cohort09',
        'Cohort10',
        'Cohort11',
        'Cohort12'
    ),
    M01 = c(11000, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0),
    M02 = c(1900, 10000, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0),
    M03 = c(1400, 2000, 11500, 0, 0, 0, 0, 0, 0, 0, 0, 0),
    M04 = c(1100, 1300, 2400, 13200, 0, 0, 0, 0, 0, 0, 0, 0),
    M05 = c(1000, 1100, 1400, 2400, 11100, 0, 0, 0, 0, 0, 0, 0),
    M06 = c(900, 900, 1200, 1600, 1900, 10300, 0, 0, 0, 0, 0, 0),
    M07 = c(850, 900, 1100, 1300, 1300, 1900, 13000, 0, 0, 0, 0, 0),
    M08 = c(850, 850, 1000, 1200, 1100, 1300, 1900, 11500, 0, 0, 0, 0),
    M09 = c(800, 800, 950, 1100, 1100, 1250, 1000, 1200, 11000, 0, 0, 0),
    M10 = c(800, 780, 900, 1050, 1050, 1200, 900, 1200, 1900, 13200, 0, 0),
    M11 = c(750, 750, 900, 1000, 1000, 1180, 800, 1100, 1150, 2000, 11300, 0),
    M12 = c(740, 700, 870, 1000, 900, 1100, 700, 1050, 1025, 1300, 1800, 20000)
)

cohort.clients.r <- cohort.clients #create new data frame
totcols <-
    ncol(cohort.clients.r) #count number of columns in data set
for (i in 1:nrow(cohort.clients.r)) {
    #for loop for shifting each row
    df <- cohort.clients.r[i,] #select row from data frame
    df <- df[, !df[] == 0] #remove columns with zeros
    partcols <-
        ncol(df) #count number of columns in row (w/o zeros)
    #fill columns after values by zeros
```

```
    if (partcols < totcols)
        df[, c((partcols + 1):totcols)] <- 0
    cohort.clients.r[i,] <- df #replace initial row by new one
}
# Retention ratio = # clients in particular month / # clients in 1st month of life-tim

#calculate retention (1)
x <- cohort.clients.r[, c(2:13)]
y <- cohort.clients.r[, 2]
reten.r <- apply(x, 2, function(x)
    x / y)
reten.r <- data.frame(cohort = (cohort.clients.r$cohort), reten.r)

#calculate retention (2)
c <- ncol(cohort.clients.r)
reten.r <- cohort.clients.r
for (i in 2:c) {
    reten.r[, (c + i - 1)] <- reten.r[, i] / reten.r[, 2]
}
reten.r <- reten.r[,-c(2:c)]
colnames(reten.r) <- colnames(cohort.clients.r)




#charts
reten.r <- reten.r[,-2] #remove M01 data because it is always 100%
#dynamics analysis chart
cohort.chart1 <- melt(reten.r, id.vars = 'cohort')
colnames(cohort.chart1) <- c('cohort', 'month', 'retention')
cohort.chart1 <- filter(cohort.chart1, retention != 0)
p <-
    ggplot(cohort.chart1,
           aes(
               x = month,
               y = retention,
               group = cohort,
               colour = cohort
           ))
p + geom_line(size = 2, alpha = 1 / 2) +
    geom_point(size = 3, alpha = 1) +
    geom_smooth(
        aes(group = 1),
        method = 'loess',
        size = 2,
```

```
        colour = 'red',
        se = FALSE
    ) +
    labs(title = "Cohorts Retention ratio dynamics")
```

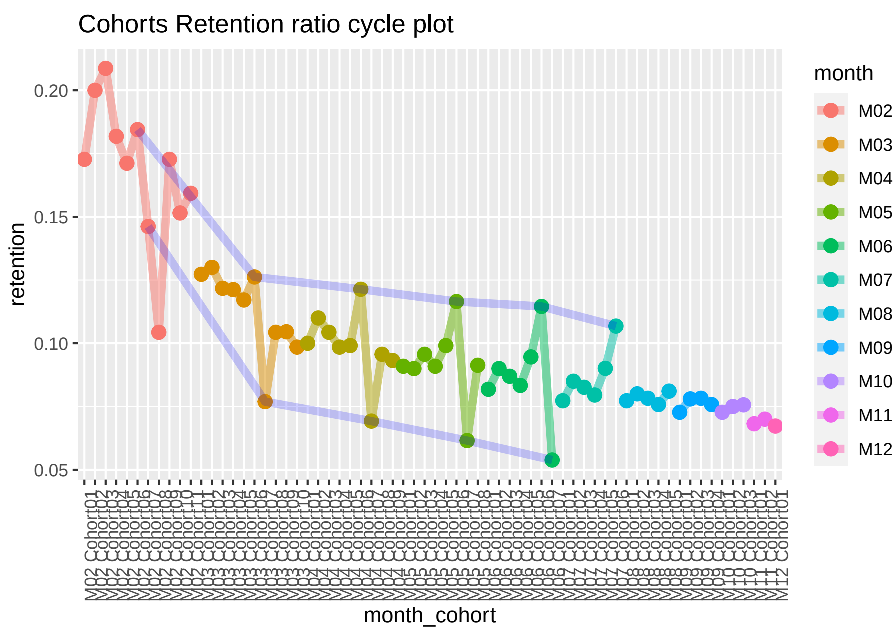



```
#second month analysis chart
cohort.chart2 <-
    filter(cohort.chart1, month == 'M02') #choose any month instead of M02
p <-
    ggplot(cohort.chart2, aes(x = cohort, y = retention, colour = cohort))
p + geom_point(size = 3) +
    geom_line(aes(group = 1), size = 2, alpha = 1 / 2) +
    geom_smooth(
        aes(group = 1),
        size = 2,
        colour = 'red',
        method = 'lm',
        se = FALSE
    ) +
    labs(title = "Cohorts Retention ratio for 2nd month")
```




```
#cycle plot
cohort.chart3 <- cohort.chart1
cohort.chart3 <-
    mutate(cohort.chart3, month_cohort = paste(month, cohort))
p <-
    ggplot(cohort.chart3,
           aes(
               x = month_cohort,
               y = retention,
               group = month,
               colour = month
           ))
#choose any cohorts instead of Cohort07 and Cohort06
m1 <- filter(cohort.chart3, cohort == 'Cohort07')
m2 <- filter(cohort.chart3, cohort == 'Cohort06')
p + geom_point(size = 3) +
    geom_line(aes(group = month), size = 2, alpha = 1 / 2) +
    labs(title = "Cohorts Retention ratio cycle plot") +
    geom_line(
        data = m1,
        aes(group = 1),
        colour = 'blue',
        size = 2,
        alpha = 1 / 5
    ) +
```

```
    geom_line(
        data = m2,
        aes(group = 1),
        colour = 'blue',
        size = 2,
        alpha = 1 / 5
    ) +
    theme(axis.text.x = element_text(angle = 90, hjust = 1))
```



15.11.2.4 Lifecycle phase sequential analysis

- analyze the path patterns of each cohort
- identify cohorts that attracted customers with the path we prefer to make offers.

```
library(TraMineR)

min.date <- min(orders$orderdate)
max.date <- max(orders$orderdate)

l <-
    c(seq(0, as.numeric(max.date - min.date), 10), as.numeric(max.date - min.date))

df <- data.frame()
```

```
for (i in l) {
    cur.date <- min.date + i
    print(cur.date)

    orders.cache <- orders %>%
        filter(orderdate <= cur.date)

    customers.cache <- orders.cache %>%
        select(-product,-grossmarg) %>%
        unique() %>%
        group_by(clientId) %>%
        mutate(frequency = n(),
               recency = as.numeric(cur.date - max(orderdate))) %>%
        ungroup() %>%
        select(clientId, frequency, recency) %>%
        unique() %>%

        mutate(segm =
                   ifelse(
                       between(frequency, 1, 2) & between(recency, 0, 60),
                       'new customer',
                       ifelse(
                           between(frequency, 1, 2) &
                               between(recency, 61, 180),
                           'under risk new customer',
                           ifelse(
                               between(frequency, 1, 2) & recency > 180,
                               '1x buyer',

                               ifelse(
                                   between(frequency, 3, 4) &
                                       between(recency, 0, 60),
                                   'engaged customer',
                                   ifelse(
                                       between(frequency, 3, 4) &
                                           between(recency, 61, 180),
                                       'under risk engaged customer',
                                       ifelse(
                                           between(frequency, 3, 4) & recency > 180,
                                           'former engaged customer',

                                           ifelse(
                                               frequency > 4 & between(recency, 0, 60)
                                               'best customer',
                                               ifelse(
```

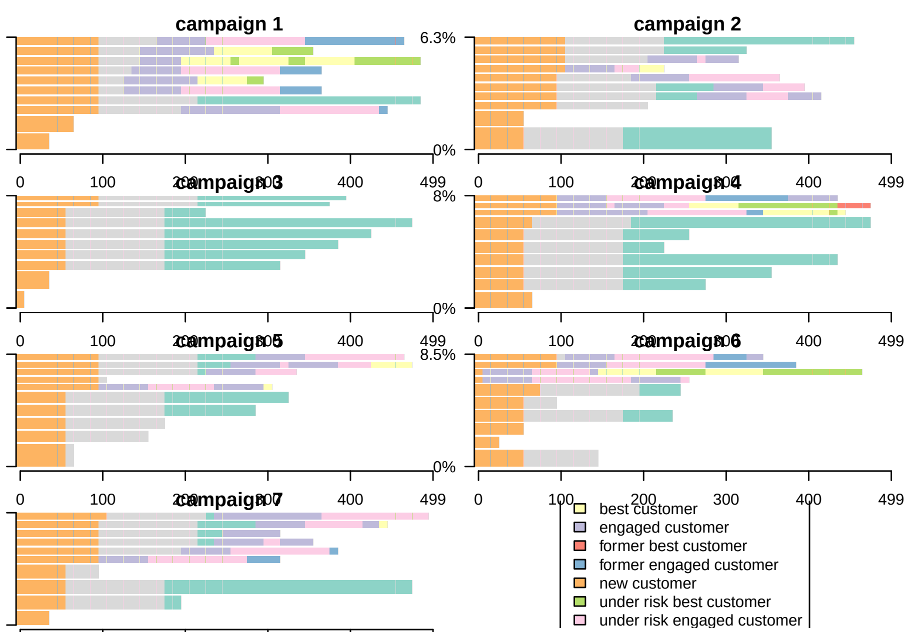
```
                                                        frequency > 4 &
                                                            between(recency, 61, 180),
                                                        'under risk best customer',
                                                        ifelse(frequency > 4 &
                                                                   recency > 180, 'former best custome
                                                    )
                                                )
                                            )
                                        )
                                    )
                                )
                            )
                        )) %>%

        mutate(report.date = i) %>%
        select(clientId, segm, report.date)

    df <- rbind(df, customers.cache)
}
```

```
## [1] "2012-01-02"
## [1] "2012-01-12"
## [1] "2012-01-22"
## [1] "2012-02-01"
## [1] "2012-02-11"
## [1] "2012-02-21"
## [1] "2012-03-02"
## [1] "2012-03-12"
## [1] "2012-03-22"
## [1] "2012-04-01"
## [1] "2012-04-11"
## [1] "2012-04-21"
## [1] "2012-05-01"
## [1] "2012-05-11"
## [1] "2012-05-21"
## [1] "2012-05-31"
## [1] "2012-06-10"
## [1] "2012-06-20"
## [1] "2012-06-30"
## [1] "2012-07-10"
## [1] "2012-07-20"
## [1] "2012-07-30"
## [1] "2012-08-09"
## [1] "2012-08-19"
## [1] "2012-08-29"
```

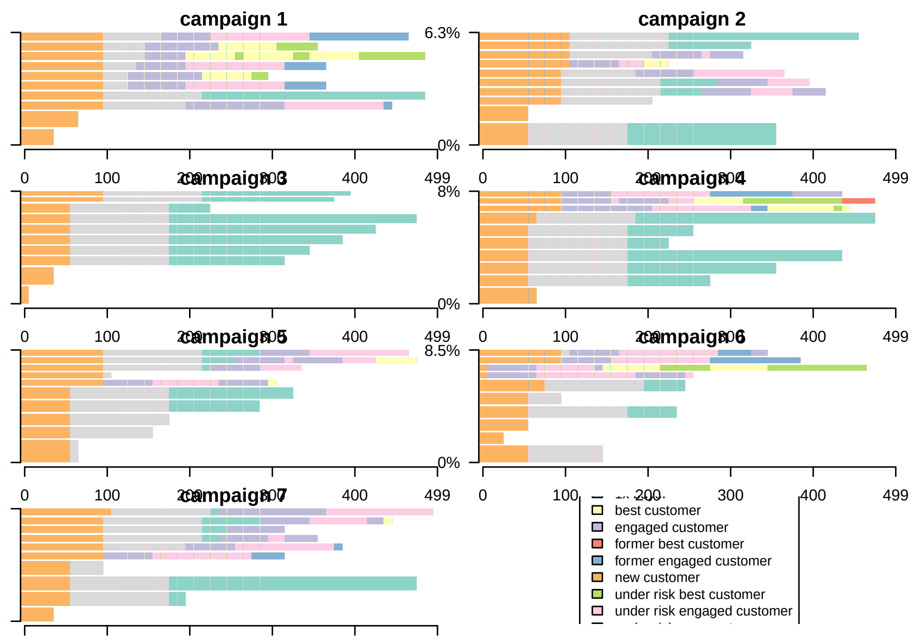
```
## [1] "2012-09-08"
## [1] "2012-09-18"
## [1] "2012-09-28"
## [1] "2012-10-08"
## [1] "2012-10-18"
## [1] "2012-10-28"
## [1] "2012-11-07"
## [1] "2012-11-17"
## [1] "2012-11-27"
## [1] "2012-12-07"
## [1] "2012-12-17"
## [1] "2012-12-27"
## [1] "2013-01-06"
## [1] "2013-01-16"
## [1] "2013-01-26"
## [1] "2013-02-05"
## [1] "2013-02-15"
## [1] "2013-02-25"
## [1] "2013-03-07"
## [1] "2013-03-17"
## [1] "2013-03-27"
## [1] "2013-04-06"
## [1] "2013-04-16"
## [1] "2013-04-26"
## [1] "2013-05-06"
## [1] "2013-05-15"
```

```
# converting data to the sequence format
df <-
    dcast(df,
          clientId ~ report.date,
          value.var = 'segm',
          fun.aggregate = NULL)
df.seq <- seqdef(df,
                 2:ncol(df),
                 left = 'DEL',
                 right = 'DEL',
                 xtstep = 10)

# creating df with first purch.date and campaign cohort features
feat <- df %>% select(clientId)
feat <- merge(feat, campaign[, 1:2], by = 'clientId')
feat <- merge(feat, customers[, 1:2], by = 'clientId')

par(mar = c(1, 1, 1, 1))
```

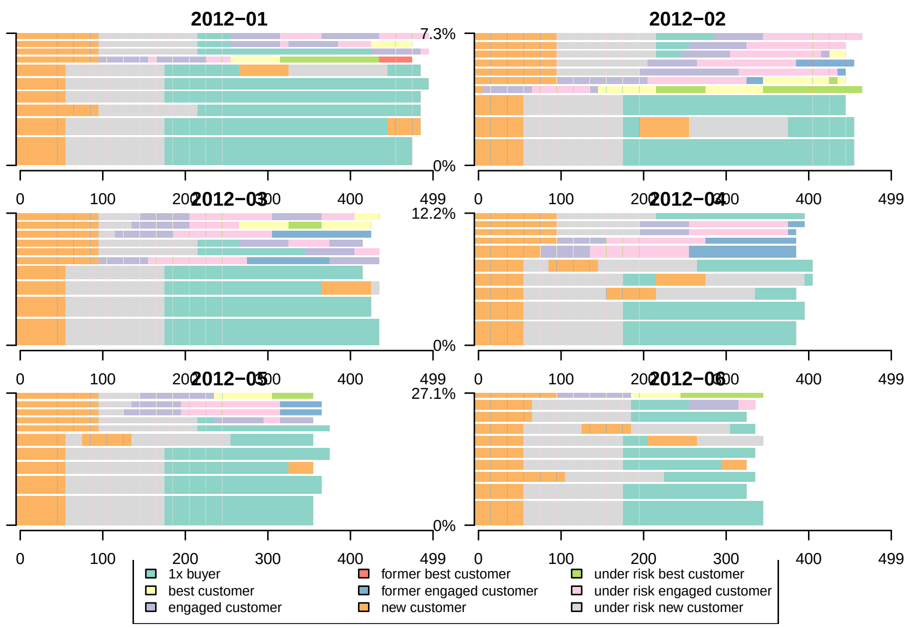
```
# plotting the 10 most frequent sequences based on campaign
seqfplot(df.seq, border = NA, group = feat$campaign)
```




```
# plotting the 10 most frequent sequences based on campaign
seqfplot(
    df.seq,
    border = NA,
    group = feat$campaign,
    cex.legend = 0.9
)
```




```
# plotting the 10 most frequent sequences based on first purch.date cohort
coh.list <- sort(unique(feat$cohort))
# defining cohorts for plotting
feat.coh.list <- feat[feat$cohort %in% coh.list[1:6] ,]
df.coh <- df %>% filter(clientId %in% c(feat.coh.list$clientId))
df.seq.coh <-
    seqdef(
        df.coh,
        2:ncol(df.coh),
        left = 'DEL',
        right = 'DEL',
        xtstep = 10
    )
seqfplot(
    df.seq.coh,
    border = NA,
    group = feat.coh.list$cohort,
    cex.legend = 0.9
)
```


2012−01
2012−02
2012−03
2012−04
2012−05
2012−06
7.3%
12.2%
27.1%
0%
0
100
200
300
400
499
1x buyer
best customer
engaged customer
former best customer
former engaged customer
new customer
under risk best customer
under risk engaged customer
under risk new customer

Chapter 16

Nudges

(Garbinsky et al., 2020) uncovers that people under save because they have positive illusion of financial responsibility (i.e., they thought they are financially more responsible than the average).

Chapter 17

Visualization

Bibliography

Aaker, D. (1996). *Building Strong Brands.*

Akpinar, E. and Berger, J. (2017). Valuable virality. *Journal of Marketing Research*, 54(2):318–330.

Berger, J. (2014). Word of mouth and interpersonal communication: A review and directions for future research. *Journal of Consumer Psychology*, 24(4):586–607.

Berger, J. and Fitzsimons, G. (2008). Dogs on the street, pumas on your feet: How cues in the environment influence product evaluation and choice. *Journal of Marketing Research*, 45(1):1–14.

Berlyne, D. E. (1960). Toward a theory of exploratory behavior: I. arousal and drive. In *Conflict, arousal, and curiosity.*, pages 163–192. McGraw-Hill Book Company.

Carhart, M. M. (1997). On persistence in mutual fund performance. *The Journal of Finance*, 52(1):57–82.

Chakravarty, A. and Grewal, R. (2011). The stock market in the driver's seat! implications for r&d and marketing. *Management Science*, 57(9):1594–1609.

Chakravarty, A. and Grewal, R. (2016). Analyst earning forecasts and advertising and r&d budgets: Role of agency theoretic monitoring and bonding costs. *Journal of Marketing Research*, 53(4):580–596.

Chandrasekaran, D., Tellis, G. J., and James, G. M. (2020). Leapfrogging, cannibalization, and survival during disruptive technological change: The critical role of rate of disengagement. *Journal of Marketing*, page 002224292096791.

Chen, A. H. and Siems, T. F. (2004). The effects of terrorism on global capital markets. *European Journal of Political Economy*, 20(2):349–366.

Connors, S., Khamitov, M., Thomson, M., and Perkins, A. (2020). EXPRESS: They’re just not that into you: How to leverage existing consumer-brand relationships through social psychological distance. *Journal of Marketing*, page 002224292098449.

Fama, E. F. and French, K. R. (1993). Common risk factors in the returns on stocks and bonds. *Journal of Financial Economics*, 33(1):3–56.

Fama, E. F. and French, K. R. (2015). A five-factor asset pricing model. *Journal of Financial Economics*, 116(1):1–22.

Garbinsky, E. N., Mead, N. L., and Gregg, D. (2020). EXPRESS: Popping the positive illusion of financial responsibility can increase personal savings: Applications in emerging and western markets. *Journal of Marketing*, page 002224292097964.

Gordon, B. R., Jerath, K., Katona, Z., Narayanan, S., Shin, J., and Wilbur, K. C. (2019). Inefficiencies in digital advertising markets. *SSRN Electronic Journal*.

Gordon, B. R., Zettelmeyer, F., Bhargava, N., and Chapsky, D. (2017). A comparison of approaches to advertising measurement: Evidence from big field experiments at facebook. *SSRN Electronic Journal*.

Grewal, R., Chakravarty, A., Ding, M., and Liechty, J. (2008). Counting chickens before the eggs hatch: Associating new product development portfolios with shareholder expectations in the pharmaceutical sector. *International Journal of Research in Marketing*, 25(4):261–272.

Grewal, R., Chandrashekaran, M., and Citrin, A. V. (2010). Customer satisfaction heterogeneity and shareholder value. *Journal of Marketing Research*, 47(4):612–626.

Holt, D. B. (1998). Does cultural capital structure american consumption? *Journal of Consumer Research*, 25(1):1–25.

Kashmiri, S. and Mahajan, V. (2017). Values that shape marketing decisions: Influence of chief executive officers' political ideologies on innovation propensity, shareholder value, and risk. *Journal of Marketing Research*, 54(2):260–278.

Kim, M. and McAlister, L. M. (2011). Stock market reaction to unexpected growth in marketing expenditure: Negative for sales force, contingent on spending level for advertising. *Journal of Marketing*, 75(4):68–85.

Kumar, V., Petersen, A., and P.Leone, R. (2007). How valuable is word of mouth? *Harvard Business Review*.

Kumar, V., Venkatesan, R., Bohling, T., and Beckmann, D. (2008). Practice prize report—the power of CLV: Managing customer lifetime value at IBM. *Marketing Science*, 27(4):585–599.

Lewis, R. A. and Rao, J. M. (2015). The unfavorable economics of measuring the returns to advertising $*$. *The Quarterly Journal of Economics*, 130(4):1941–1973.

Libert, K. (2014). Age and gender matter in viral marketing. *Harvard Business Review*.

Little, J. D. C. (1970). Models and managers: The concept of a decision calculus. *Management Science*, 16(8):B–466–B–485.

Little, J. D. C. (1976). Models and managers: The concept of a decision calculus. In *Lecture Notes in Economics and Mathematical Systems*, pages 208–209. Springer Berlin Heidelberg.

Liu, Y., Shankar, V., and Yun, W. (2017). Crisis management strategies and the long-term effects of product recalls on firm value. *Journal of Marketing*, 81(5):30–48.

Malshe, A. and Agarwal, M. (2015). From finance to marketing: Impact of financial leverage on customer satisfaction. *Journal of Marketing*, page 150626124337002.

Manchanda, P., Rossi, P. E., and Chintagunta, P. K. (2004). Response modeling with nonrandom marketing-mix variables. *Journal of Marketing Research*, 41(4):467–478.

Mantrala, M. K., Sinha, P., and Zoltners, A. A. (1992). Impact of resource allocation rules on marketing investment-level decisions and profitability. *Journal of Marketing Research*, 29(2):162.

Markovitch, D. G., Steckel, J. H., and Yeung, B. (2005). Using capital markets as market intelligence: Evidence from the pharmaceutical industry. *Management Science*, 51(10):1467–1480.

McAlister, L., Srinivasan, R., Jindal, N., and Cannella, A. A. (2016). Advertising effectiveness: The moderating effect of firm strategy. *Journal of Marketing Research*, 53(2):207–224.

Morgan, N. A. and Rego, L. L. (2009). Brand portfolio strategy and firm performance. *Journal of Marketing*, 73(1):59–74.

Nagle, F. and Riedl, C. (2014). Online word of mouth and product quality disagreement. *Academy of Management Proceedings*, 2014(1):15681.

Nezami, M., Worm, S., and Palmatier, R. W. (2018). Disentangling the effect of services on b2b firm value: Trade-offs of sales, profits, and earnings volatility. *International Journal of Research in Marketing*, 35(2):205–223.

Rao, V. R., Agarwal, M. K., and Dahlhoff, D. (2004). How is manifest branding strategy related to the intangible value of a corporation? *Journal of Marketing*, 68(4):126–141.

Rust, R. T., Ambler, T., Carpenter, G. S., Kumar, V., and Srivastava, R. K. (2004). Measuring marketing productivity: Current knowledge and future directions. *Journal of Marketing*, 68(4):76–89.

Shannon, C. E. (1948). A mathematical theory of communication. *Bell System Technical Journal*, 27(3):379–423.

Shapiro, B., Hitsch, G. J., and Tuchman, A. (2018). Generalizable and robust TV ad effects. *SSRN Electronic Journal*.

Srivastava, R. K., Shervani, T. A., and Fahey, L. (1998). Market-based assets and shareholder value: A framework for analysis. *Journal of Marketing*, 62(1):2–18.

Tybout, A. M. and calkins, T. (2005). *Kellogg on Branding*.

Wies, S., Hoffmann, A. O. I., Aspara, J., and Pennings, J. M. (2019). Can advertising investments counter the negative impact of shareholder complaints on firm value? *Journal of Marketing*, 83(4):58–80.

Winterich, K. P., Gangwar, M., and Grewal, R. (2018). When celebrities count: Power distance beliefs and celebrity endorsements. *Journal of Marketing*, 82(3):70–86.

Zhang, Y., Bradlow, E. T., and Small, D. S. (2013). New measures of clumpiness for incidence data. *Journal of Applied Statistics*, 40(11):2533–2548.

Zhang, Y., Bradlow, E. T., and Small, D. S. (2015). Predicting customer value using clumpiness: From RFM to RFMC. *Marketing Science*, 34(2):195–208.

Zhao, K., Mahboobi, S. H., and Bagheri, S. R. (2018). Revenue-based attribution modeling for online advertising. *International Journal of Market Research*, 61(2):195–209.

Zwillinger, D. (1995). *CRC Standard Mathematical Tables and Formulae, 30th Edition*. CRC Press.